-

Iowa Research Online

The design of a natural language modeling environment for manufacturing system simulation

Su, Horng-Ming

https://iro.uiowa.edu/esploro/outputs/doctoral/The-design-of-a-natural-language/9985153720802771/filesAndLinks?index=0

Su, H.-M. (1988). The design of a natural language modeling environment for manufacturing system simulation [University of Iowa].
https://iro.uiowa.edu/esploro/outputs/doctoral/The-design-of-a-natural-language/9985153720802771

https://iro.uiowa.edu
Free to read and download
No known copyright restrictions
Downloaded on 2026/11/07 07:19:22 -0600

-

INFORMATION TO USERS

The most advanced technology has been used to photograph and reproduce this manuscript from the microfilm master. UMI films the original text directly from the copy submitted. Thus, some dissertation copies are in typewriter face, while others may be from a computer printer.

In the unlikely event that the author did not send UMI a complete manuscript and there are missing pages, these will be noted. Also, if unauthorized copyrighted material had to be removed, a note will indicate the deletion.

Oversize materials (e.g., maps, drawings, charts) are reproduced by sectioning the original, beginning at the upper left-hand corner and continuing from left to right in equal sections with small overlaps. Each oversize page is available as one exposure on a standard 35 mm slide or as a 17″ × 23″ black and white photographic print for an additional charge.

Photographs included in the original manuscript have been reproduced xerographically in this copy. 35 mm slides or 6″ × 9″ black and white photographic prints are available for any photographs or illustrations appearing in this copy for an additional charge. Contact UMI directly to order.



U·M·I

Accessing the World's Information since 1938

300 North Zeeb Road, Ann Arbor, MI 48106-1346 USA

Order Number 8815141

The design of a Natural Language Modeling Environment for manufacturing system simulation

Su, Horng-Ming, Ph.D.

The University of Iowa, 1988

U·M·I
300 N. Zeeb Rd.
Ann Arbor, MI 48106

PLEASE NOTE:

In all cases this material has been filmed in the best possible way from the available copy. Problems encountered with this document have been identified here with a check mark ___✓___.

1. Glossy photographs or pages _____
2. Colored illustrations, paper or print _____
3. Photographs with dark background _____
4. Illustrations are poor copy _____
5. Pages with black marks, not original copy ___✓___
6. Print shows through as there is text on both sides of page _____
7. Indistinct, broken or small print on several pages ___✓___
8. Print exceeds margin requirements _____
9. Tightly bound copy with print lost in spine _____
10. Computer printout pages with indistinct print _____
11. Page(s) _____ lacking when material received, and not available from school or author.
12. Page(s) _____ seem to be missing in numbering only as text follows.
13. Two pages numbered _____. Text follows.
14. Curling and wrinkled pages _____
15. Dissertation contains pages with print at a slant, filmed as received _____
16. Other _____

_____

_____

U·M·I

THE DESIGN OF
A NATURAL LANGUAGE MODELING ENVIRONMENT
FOR
MANUFACTURING SYSTEM SIMULATION

by

Horng-Ming Su

A thesis submitted in partial fulfillment of the
requirements for the Doctor of Philosophy degree
in Industrial and Management Engineering
in the Graduate College of
The University of Iowa

May 1988

Thesis supervisor: Assistant Professor Voratas Kachitvichyanukul

Graduate College
The University of Iowa
Iowa City, Iowa

CERTIFICATE OF APPROVAL

PH.D. THESIS

This is to certify that the Ph.D. thesis of

Horng-Ming Su

has been approved by the Examining Committee for the thesis requirement for the Doctor of Philosophy degree in Industrial and Management Engineering at the May 1988 graduation.

Thesis committee:

Thesis supervisor

Member

Member

Member

Member

To My Parents :

Mr. Chen-Pi Su and Mrs. Lin-Mei Su

ACKNOWLEDGEMENTS

There are many people to whom I would like to express my sincere appreciation. First and foremost, I would like to thank my thesis advisor, Professor Voratas Kachitvichyanukul, for suggesting the idea of applying natural language in the area of model formulation to me and for his trust and confidence in me. I would also like to thank Professor Gary W. Fischer, James R. Buck, Tzvi Raz, and Joseph Kearney for their valuable suggestions and for serving on my committee.

I would also like to take this opportunity to give a word of appreciation to Jim Lee and his wife, Liang-Fang, for providing a place where I could go anytime and was always welcome during the years in Iowa City. A warm thanks goes as well to Sarina Chen. Through her, I had more of a chance to explore myself with people from different cultural backgrounds, which should be another purpose for studying abroad.

A special note of thanks also goes to Marilyn Schippers Doyle for editing this thesis.

Finally, there are my parents, Mr. Chen-Pi Su and Mrs. Lin-Mei Su. Without their consistent inspiration and support, my life would have led in other directions. Without them, my life would not be as meaningful as it is. Even there is no way for me to adequately express my appreciation, I would still like to express my deep gratitude to them, and very, very much more.

TABLE OF CONTENTS

Page

LIST OF TABLES ---------- vii

LIST OF FIGURES ---------- viii

CHAPTER

I. INTRODUCTION. ---------- 1

1.1. The Problem to Be Solved. ---------- 1
1.2. Goals to Achieve and Foreseeable Contributions. ---------- 3
1.3. The Basic Framework of
the Natural Language Modeling Environment. ---------- 6
1.4. Research Plan. ---------- 8
1.5. Overview of Thesis. ---------- 9

II. RELEVANT LITERATURE. ---------- 11

2.1. Natural Language Processing Strategies. ---------- 11
2.1.1. Structures of English Statements in
System Descriptions. ---------- 12
2.1.2. Semantic Analysis. ---------- 14
2.1.3. Syntactic Analysis. ---------- 17
2.1.4. Ambiguity,
the Problem of Natural Language Understanding. 23
2.2. Knowledge Representation Schemes. ---------- 24
2.2.1. Production Rules. ---------- 27
2.2.2. Frames. ---------- 29
2.3. Modular Modeling Techniques. ---------- 32
2.4. Applications of Natural Language Processing. ---------- 33

III. AN OVERVIEW OF THE NATURAL LANGUAGE MODELING ENVIRONMENT(NLME). ---------- 36

3.1. The Application Domain: Manufacturing Systems. ---------- 36
3.2. System Description: The Input Data. ---------- 37
3.2.1. Modeling Concepts for Manufacturing Systems. ------ 39
3.2.2. Words, Phrases, Sentences, and Modeling Concepts. 40
3.3. How the System Works: A General Description of the NLME. --- 43

IV. THE STRUCTURE AND FUNCTIONS OF THE LANGUAGE PROCESSOR. ---------- 47

4.1. Case Grammar: The Theory Behind the Language Processor. ---- 47
4.2. Lexicon: The Knowledge Base of the Language Processor. ------- 54
4.2.1. Syntactic Knowledge. ---------- 56
4.2.2. Semantic and World Knowledge. ---------- 57
4.2.2.1. Semantic Representation Structure. ----- 57
4.2.2.2. Features. ---------- 64
4.2.2.3. Modifiers. ---------- 64
4.2.3. The Inheritance Property of the Lexicon. ---------- 64
4.3. Syntactic Processor. ---------- 69
4.3.1. PreProcessor: The Frontier of the System. ---------- 69
4.3.2. Syntactic Analyzer: An Augmented Transition Network (ATN) Based Parser. ---------- 69
4.4. Semantic Processor. ---------- 76
4.4.1. Semantic Synthesizer. ---------- 76
4.4.2. Short Term Memory. ---------- 79
4.4.3. Context Applier: Contextual Rules and Contextual Memory. -------- 80

V. THE STRUCTURE AND FUNCTIONS OF THE CENTRAL UNIT. ------ 88

5.1. Working Memory (WM): The Heart of the Central Unit ---------- 90
5.2. Semantic Representation Interpreter. ---------- 95
5.3. Program Generator: The Program Writer of the System. --------- 101
5.4. Outputs of the Central Unit. ---------- 105

VI. THE NATURAL LANGUAGE MODELING ENVIRONMENT PROTOTYPE. ---- 108

6.1. Current Implementation. ---- 108

6.1.1. Sentence Structures. ---- 109

6.1.2. Modeling Capabilities. ---- 109

6.1.3. System Descriptions. ---- 113

6.1.4. Users' Interface. ---- 115

6.2. An Annotated Example. ---- 120

VII. AN EMPIRICAL STUDY OF THE SYSTEM ---- 137

7.1. The Experiment. ---- 137

7.2. Experimental Result. ---- 143

7.3. Some Remarks About the Experiment. ---- 144

VIII. CONCLUDING REMARKS. ---- 147

8.1. Conclusions. ---- 147

8.2. Directions for the Further Research. ---- 148

REFERENCES. ---- 150

APPENDIX I: THE DEFINITIONS OF THE AUGMENTED TRANSITION NETWORKS GIVEN IN FIGURE 17. ---- 154

APPENDIX II :. A CONTINUATION OF THE TRACE GIVEN IN CHAPTER SIX. ---- 174

APPENDIX III : SUMMARY REPORTS, MODEL TEMPLATES AND PROGRAMS CORRESPONDING TO EXPERIMENT PROBLEMS. ---- 187

LIST OF TABLES

Tables Page

1. Suggestions for Improving the Use of Simulation in Organization. ---------- 2
2. A Frame for a Space Drill Process to Create a Hole H. ---------- 30
3. An Example of a System Description. ---------- 38
4. Case Frames Used in the NLME Prototype. ---------- 51
5. A Summary of the Syntactic Information Stored under each Type of Part of Speech. ---------- 58
6. An Example of the Lexicon. ---------- 67
7. An Example of Contextual Rules. ---------- 84
8. The Contents of the Initial Structure of the Working Memory. ---------- 96
9. An Example of Store Rules for a Transfer Frame. ---------- 99
10. An Example of Defining Modules in SLAM. ---------- 103
11. The Acceptable Sentence Structures by the NLME Prototype. ---------- 110
12. A Summary Report Corresponding to the System Description in Table 3. ----- 133
13. A SLAM Program Corresponding to the System Description in Table 3. ------ 135
14. Experiment Problem Number One. ---------- 139
15. Experiment Problem Number Two. ---------- 140
16. Experiment Problem Number Three. ---------- 141
17. Experimental Result. ---------- 143
18. The Average Time Spent on each Problem by the Two Approaches. ---------- 144

LIST OF FIGURES

Figures Page

1. The Life Cycle of Model Development. ---------- 5
2. An Example of Augmented Transition Networks. ---------- 19
3. A Syntactic Structure Tree for the Sentence "The boy kicks a ball ." ---------- 20
4. The Augmented Transition Networks Used in LUNAR. ---------- 21
5. Four Different Interpretations for the Sentence "I saw the man on the hill with a telescope." ---------- 25
6. The Control Flow of the Natural Language Modeling Environment. ---------- 44
7. The Structure of the Natural Language Modeling Environment. ---------- 45
8. The Structure of the Language Processor. ---------- 48
9. The Control Flow of the Language Processor. ---------- 49
10. The Control Flow of the Dictionary Search Process. ---------- 66
11. An Example of Augmented Transition Networks with Their Definitions. ------- 74
12. A Framework of Transforming a Sentence into Its Semantic Representation. -- 78
13. The Control Flow of the Context Applier. ---------- 86
14. The Structure of the Central Unit. ---------- 88
15. The Control Flow of the Central Unit. ---------- 89
16. The Working Memory Structure Corresponding to the System Description in Table 3. ---------- 91
17. The Augmented Transition Networks Corresponding to the Sentence Structures in Table 11. ---------- 111
18. Main Menu. ---------- 116
19. Language Processor Menu. ---------- 117
20. Trace Utility Menu. ---------- 118

21. Working Memory Utility Menu. ---------- 119

22. Report Utility Menu. ---------- 119

23. Program Generator Menu. ---------- 120

24. Manual Approach Vs NLME Approach. ---------- 145

CHAPTER I

INTRODUCTION

1.1 The Problem to Be Solved

In recent years, because of the successful expert system applications, Artificial Intelligence (AI) techniques have attracted a great deal of attention from both research and application communities. Among all AI techniques, the knowledge representation schemes and AI languages such as LISP and PROLOG are most widely used to design new simulation languages[Sathi et. al., 1986; Buller and Schultz, 1986; Klahr and Faught, 1985; Ruiz-Mier, Talavage and Ben-Arieh, 1985; Reddy et. al., 1983; Nielsen, 1986]. Basically, these simulation languages provide a more flexible modeling capability and the most important thing of all is that they attempt to incorporate human decisions into these models.

Although these improvements indeed enhance the power of simulation itself, the user still has not been relieved from the tedious model formulation process. As usual, the system analysts first need to learn the modeling concepts of a language and use them in the formulation of models and second, need to learn the language syntax and convert their conceptual models into program in order to perform the experiments. From time to time, the whole process generates a great deal of frustration.

Moreover, currently there are quite a few commercially available simulation languages. For different languages, functional supports and the information provided can be quite different, and one language can be more suitable for one type of application than the others. Furthermore, the modeling concepts and syntax of these languages can be

quite different. Therefore, if a system analyst wants to apply different simulation languages to different applications, the effort spent on mastering these concepts and syntax can be tremendous.

According to the survey of industrial practice concerning simulation technique by Christy and Watson [1983], increasing the knowledge of the technique by end users and managers, as well as making the software easier to use and less expensive are the two most important factors among the recommendations for improving the use of simulation in organizations. (A complete listing of the recommendations is shown in Table 1.) However, the knowledge of simulation techniques can be promoted through the easy-to-use and low cost simulation system. In addition, the continuous growth of the personal computer population in industrial and academic communities in the last two years reveals that the hardware and software resources have already become less and less expensive. Therefore, the only main concern currently is to make the simulation software easier to use; in other words, to reduce the difficulty in the modeling process.

Table 1 : Suggestions for Improving the Use of Simulation in Organization (reproduced from [Christy and Watson, 1983]).

Recommendation	%
Knowledge of the technique (uses, limitations, and potential) by end users and older managers	28
Easier to use, less expensive software	26
More accessible, larger, better quality data bases	20
Better communication between manager / analyst / client	8
Easier interface with data bases; integrated models	6
Improves understanding of the technique and its philosophy by modelers	4
Lower costs	3

1.2. Goals to Achieve and Foreseeable Contributions

Regarding the great amount of effort and time spent on modeling a system, a Natural Language Modeling Environment (NLME) is designed to assist the system analyst to minimize the amount of time and effort spent on the formulation process. The prototype system of NLME will be implemented in the domain of manufacturing.

The first goal that the NLME tries to achieve is to allow a user to communicate with a computer using his or her own native language, English, as opposed to confined computer languages. Thus, an analyst can just sit down in front of a terminal and communicate with a computer about his or her problem and the computer will follow his or her problem description to formulate a model and further to generate a simulation program automatically.

The advantages of such a system are quite obvious. First, it is easier to use. A system that is easy to use can usually reduce the time and expertise required for system analysts to build models. Consequently, the productivity of analysts can be increased and the cost of simulation studies can be decreased. Secondly, the model as well as the problem description can be easily used to communicate among people. Generally, poor communication is one of the problems in modeling projects [Balci, 1986]. That is, especially when projects involve persons with different backgrounds and areas of expertise. Thus, it is essential to have effective communications tools in order for those persons to exchange their ideas. On the other hand, if models are written in simulation languages, it is hard to discuss the models among a group of people with different levels of knowledge of simulation and the languages. However, if models are written in their natural language, such a problem can be reduced to the minimum.

The second goal to be accomplished in this research is to design an internal model representation scheme, for storing the information about models, which is independent of

simulation languages. The third goal is to design the language drivers (code generator) which can compile the internal model representations of systems into simulation programs.

Actually, the internal representation scheme is very essential to the whole system since it is nearly impossible to generate simulation programs directly from system descriptions. Here, a system description is composed of statements used to describe a manufacturing system. The major difficulties are both that there is no one to one relationship between a descriptive statement and a program statement, and the events/activities described in system descriptions can be out of logical sequence. Therefore, it is necessary to have an intermediate representation device between a description and a program. If the representation structure is properly designed, not only can the information be easily stored, but simulation programs for different languages can also be generated using different language drivers. By and large, users can take the advantage of the special features of the languages.

Finally, the proposed system can shorten the life-cycle of model development. According to the development cycle [Balci, 1986] given in Figure 1, the modeling process is started from formulating the conceptual model of the system under study. Basically, the conceptual model exists in the mind of the modeler. Then, the conceptual model is presented by the communication model used to discuss among people. Finally, the simulation program is coded based on the communication model and used to perform experiments.

Generally speaking, the system description of the NLME can be regarded as the communication model in the model development cycle. However, the additional benefit of the system description, is that it can be converted into a program automatically. Traditionally, the process of converting a communication model into a programmed model is a manual and tedious task. The process starts with the coding of the program, followed

by the compilation and debugging. Also, the whole process needs to be cycled repeatedly until a correct program is obtained. Unlike the traditional approach, system descriptions are transformed into programs directly by a computer. In other words, the process from the communication model to the programmed model can be eliminated.

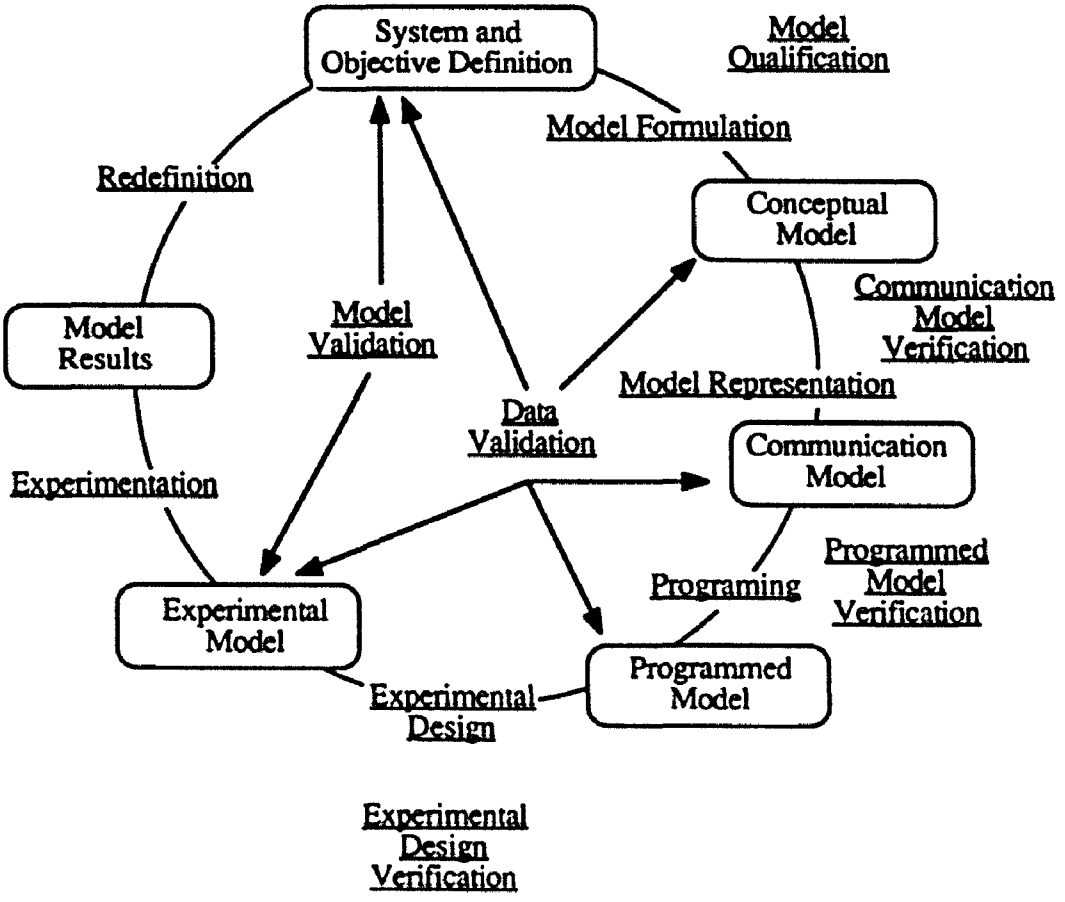


Figure 1. The Life Cycle of Model Development.
(reproduced from [Balci, 1986])

1.3. The Basic Framework of the Natural Language Modeling Environment

When a simulation analyst receives a problem description, the procedure used to formulate the problem description into a simulation program can be described as follows:

1. Identifying entities, events/activities, states and resource engaged in the system, and relations among them. This process should include[Prisker and Pegden, 1979] (a) defining the changes in states that occur at each event time, (b) describing the activities in which the entities in the system engage, and (c) describing the processes through which the entities in the system flow.
2. According to the above information, laying out the overall structure of the system with the help of a diagram or a flow chart.
3. Converting the structure of the conceptual model from last step into specific program statements.

In order to accomplish the above task, the analyst should possess several kinds of knowledge.

1. Knowledge about words and structures of sentences.

   It should include both the syntactic and semantic characteristics of words and sentences. This is the basic knowledge that a person uses to interpret the meaning of words and sentences.
2. Knowledge about the problem domain.

   The domain knowledge serves three purposes. First, the knowledge is used to constrain the number of possible interpretations of words and sentences. Frequently, a word in a sentence can have different possible interpretations. The correct meaning of the word usually can be determined by the surrounding words

and the domain knowledge. For instance, in the domain of manufacturing, the word "move" represents the meaning of "change position" instead of others such as "arouse." Second, it can provide the context information of activities, which is not explicitly revealed in a problem description. For example, when a work piece is completed, the subsequent activity is expected to be either transferring the work piece to the next station or sending it to a storage area. Finally, the domain knowledge can also be applied to check for the completeness of the information collected. For example, a transportation activity should be participated by parts and material handling equipment; and a destination should be specified.

3. Knowledge about the simulation modeling concepts.

   A simulation model is composed in terms of entities, events/activities, states and resources, and their relationships. The major concern in processing input sentences is to identify these features from the sentences. In other words, it provides a way to determine the meaning of words in terms of the simulation modeling concepts. For example, under the interpretation of modeling concepts, the noun " part" means entity and the verb "transfer" implies activity. This work usually requires knowledge about specific domain and the simulation modeling concepts.

4. Knowledge about the simulation language.

   The knowledge is applied to convert the conceptual model into a specific simulation language program.

Basically, the framework of the Natural Language Modeling Environment coincides with the process which analysts employ to formulate their problems. The system consists of two major components: the Language Processor and the Central Unit. The Language Processor, the frontier of the system, is used to process input sentences. The process is

similar to the modeling procedure step one described above. Language Processor is used to perform both the syntactic and semantic analysis with the help of the prestored knowledge of words, sentence structures, modeling concepts and the problem domain. The syntactic analysis decides the grammatical structures of sentences in the system descriptions and resolves the referential ambiguities caused by pronouns and determiners. Based on the syntactic information, the semantic analysis is to construct the semantic representations of input sentences. However, if the meanings of the sentences are ambiguous, contextual analysis is also initiated to decide the proper interpretations of the sentences.

The functions of the Central Unit are to keep the information acquired from system descriptions, to generate summary reports, and to construct programs based on the information. The process of the Central Unit starts from scratching out a model in its working memory piece by piece from the information gathered from the Language Processor. Basically, the model is constructed according to the pre-defined internal representation scheme. After the model is complete, utility programs can be called to generate reports and programs.

1.4. Research Plan

In the course of developing the Natural Language Modeling Environment, the following research tasks will provide guidance of accomplishments.

1. Determine the specific knowledge required for each component of the Natural Language Modeling Environment.
2. Determine the acceptable range of sentence structures and the mechanism to be used to store them. The mechanism should be flexible enough to allow the sentence structures to be easily expanded.

3. Design a language processor to process input sentences which is capable of performing both syntactic and semantic analysis.
4. Develop the structure of a lexicon which will be used to store the linguistic information such as part-of-speech, usage, meaning, and deambiguity approaches. The lexicon can also be used to store the interpretation of a word in simulation modeling concepts.
5. Design the structure of the working memory which is used to store the information collected from a system description and associated memory manipulation functions.
6. Determine strategies which can be used to map the information in the working memory to program statements.
7. Develop an overall controlling strategy which can be employed to integrate all the system components together. This work should emphasize more on designing flexible interfaces among components.

1.5. Overview of Thesis

There are seven chapters in this thesis. Following the introduction, review of relevant literature will be provided in chapter two. The literature review will cover the Artificial Intelligence techniques, knowledge representation schemes, natural language processing approaches, and the modeling technique as well as a brief review of the applications of natural language understanding system. Chapter three is to define the system description used in the NLME and an overview of the proposed system will be also provided. Chapter four and five will be an in-depth discussion of the functions and structures of the Language Processor and the Central Unit of the NLME. Following these two chapters, the current implementation of the NLME prototype is discussed in chapter six. A complete trace of processing an example system description will also be enclosed in

this chapter. Finally, the last chapter will be the contributions of this research and directions for future research.

CHAPTER II

RELEVANT LITERATURE

This chapter focuses mainly on the supporting theories for the design of the proposed Natural Language Modeling Environment(NLME). These theories include the natural language processing strategies, the knowledge representation structures, and modular modeling techniques. In addition, a brief review of applications of natural language processing will be presented in the last section.

2.1 Natural Language Processing Strategies

In the Natural Language Modeling Environment, Language Processor is designed to parse the input sentences of a problem description for the purpose of extracting semantic information from them. The strategies employed by the Processor are to perform both syntactic and semantic analyses. More specifically, Augmented Transition Networks are used to parse a sentence into a syntactic structure tree, and then based on the structure tree, the theory of case grammar is applied to perform the semantic analysis. These two approaches will be discussed in two of the following subsections. First of all, there will be a discussion about the sentence structures of the problem description statements. Also, at the end of this section, the issue concerning the difficulties of natural language understanding will be presented.

2.1.1. Structures of English Statements in System Descriptions

In a system description, each sentence provides information about the system to be modeled. The information provided can be about activities, states, or attributes of the system. Heidorn [1973 and 1974] classified these sentences into two categories: the action sentence and the attribute sentence. Basically, an action sentence contains an action verb and is used to describe an activity in a system. For example, the sentence, "The cargo is unloaded at the dock," is an action sentence and "unloaded" is the action verb. An attribute sentence, however, is used to specify the value of an attribute such as the service time of an activity.

In contrast, Paul and Doukidis [1986] provide a structural perspective to analyze sentences. They claim that a sentence in a problem description is mainly composed of one or more activity phrases. Each activity phrase can be further divided into two (or maybe three) parts :

1. The main entity phrase, which describes the main entity.
2. The action phrase, which describes the action that the main entity is involved in.
3. The additional entity phrases, which describe the additional entities (if any) involved in the action. An additional entity could be an agent, object, or instrument of the action.

In the sentence, "The barmaid serves a customer using a glass or she washes the glasses," "barmaid" is the main entity, "serve" is the activity, and "customer" and "glass" are the additional entities. In Paul and Doukidis's system, using Heidorn's categories, we can find that only the action sentence is used to describe a model. There is no attribute sentence to specify the values of attributes.

In short, the common characteristics of these two systems are the use of the verb in an action sentence to signal an activity in a model, and the use of nouns, especially the subject and/or the object, to identify as entities. However, this is not always true. For example, in the following problem description:

The part flows from Station-1 to Station-2. (2.1)
The service time of Station-1 is exponentially distributed with a mean of 10 minutes. (2.2)
The inspection time of Station-2 is normally distributed with a mean of 5 minutes. (2.3)
A part is provided to Station-1 every 15 minutes. (2.4)

Although, there is no action sentence to describe the activities corresponding to Station-1 and Station-2, these two activities can be identified by the adjective "service" in sentence 2.2 and the noun "inspection" as adjective in sentence 2.3. Therefore, a more appropriate method to analyze a sentence that describes a model should be focused on examining what kind of information might be carried by a sentence. Generally speaking, a sentence in a problem description conveys one or several of the following messages.

1. Indicating an event or activity.
2. Specifying the values of attributes.
3. Indicating entities or instruments, which involve in an event or an activity.
4. Describing the required system status to cause an event or activity to happen.

Thus, the major concern in designing the mechanism of semantic analysis is how to extract the above information from a sentence.

2.1.2 Semantic Analysis

Case Grammar Theory [Bruce, 1975].

The objective of semantic analysis is to relate symbols to concepts. In the Natural Language Modeling Environment, the process is to extract the information regarding modeling concepts from a problem description. As we know, most of the grammatical rules encountered are used to interpret the syntactic structure of a sentence such as the subject, verb, object, etc. However, Case Grammar provides a method to combine syntactic structure with the semantic interpretation. The central idea of Case Grammar can be illustrated as follows [Barr and Feigenbaum, 1981]:

> The proposition embodied in a simple sentence has a deep structure consisting of a verb (the central component) and one or more noun phrases. Each noun phrase is associated with the verb in a particular relationship. Those relationships, which Fillmore characterized as "semantically relevant syntactic relationships " are called cases.

For Example, Fillmore [1971] proposed a case system with the following cases :

1. Agent (A) -- the instigator of the event.
2. Counter-Agent (C) -- the force or resistance against which the action is carried out.
3. Object (O) -- the entity that moves or changes or whose position or existence is in consideration.
4. Result (R) -- the entity that comes into existence as a result of the action.
6. Instrument (I) -- the stimulus or immediate physical cause of an event.
7. Goal (G) -- the place to which something moves.
8. Experience (E) -- entity which receives or accepts or experiences or undergoes the effect of an action.

In the sentence,

The worker moves a part from Station-1 to Station-2 with a truck, (2.4)

"worker" is the agent of the verb "moves," "part" is the object, "Station-1" and "Station-2" are the source and the goal, while "truck" is the instrument.

In addition, Fillmore suggested that verbs can be categorized according to "sentence types" or "case frames." A case frame indicates what case relationships may exist between a verb and its nouns. For example, the verb "move" may have a case frame as follows:

Verb : Move
Case Frame : Agent (A).
Object (O).
Instrument (I).
Source (S).
Goal (G).

The cases corresponding to a verb will not necessary appear in every sentence. Some cases are optional, while others are required. For instance, the verb "move" can be used in the following ways:

A part is moved from Station-1 to Station-2. [O,S,G] (2.6)

A part is moved to Station-2 with a truck. [O,G,I] (2.7)

The worker moves a part to Station-2 with a truck. [A,O,G,I] (2.8)

A more important aspect of the case system is that it provides the feasibility to have several verbs share the same case frame structure. For example, Schank and Abelson [1977] uses eleven different primitive acts to represent all the verbs. The advantage of such

an approach is that when two sentences describe the same event in such a way that descriptions have the same overall meaning, but are quite different forms, then the representations in the knowledge base for both description are identical [Schank and Riesbeck, 1981]. In addition, this tactic also reduces the inference problem in Artificial Intelligence, since, inference rules need to be written only once for any primitive acts rather than many times for each verb referring to that primitive act.

Currently, there is no case system which is universally acceptable. The variations of cases with different application systems are numerous. They can range from 30 cases [Martin, 1973] to 5 cases [Celce, 1972]. Basically, the underlying principle used to determine the types of cases in a case system is based on the application domain and the information required. For example, the following case system can be used in the domain of the manufacturing system:

1. Event / Activity : the events or activities associated with parts or facilities in the system.
2. Part .
3. Station .
4. Material Handling Equipment.
5. Attribute : attribute type corresponding to an event / activity or a part or a station or material handling equipment or a system.
6. Source : the place from which a part moves.
7. Destination : the place to which a part moves.
8. Status : the status corresponding to a facility or system.
9. Value : the value of an attribute.

In the sentence,

The service time of Station-1 is exponentially distributed
with a mean of 5 minutes, (2.9)

the "service" is the event and the "Station-1" is the station; whereas, the "time" is the attribute and the "exponential ..." is the value of the attribute.

Finally, the basic rules applied to identify cases from a sentence are the following [Simmons, 1973; Bobrow,et.al., 1977]:

1. Mapping between syntactic roles and cases. Most of the cases can be identified from the syntactic structure of a sentence and prepositions. For instance, the subject of the verb "move" (in active voice) should be the agent, and the preposition "by" and "with" usually signal the instrument case.

2. Selection restriction. For most cases, there are only some things with certain features that can be assigned to a specific case. For example, things assigned to the goal and source cases of the verb "move" should have the feature of place, and its agent should have the feature of animate, human being or animal, or machine.

2.1.3. Syntactic Analysis

The role of the syntactic analysis in a natural language understanding system is to support the semantic analysis. The main function of the syntactic analysis is to parse a sentence into a syntactic structure tree. Although, there are several parsing techniques currently available, one of the most widely used methods in analyzing the syntax of a sentence will be discussed in this subsection. This method is Augmented Transition Networks [Bolc, 1983].

Augmented Transition Networks (ATN) were first developed by Woods [1970]. The basic idea is to represent a grammar as a transition network with a set of nodes (or states) and arcs between the nodes. The arcs are labeled by word classes or syntactic constructs. In addition, there can be conditions at each arcs, and they may be traversed only if the conditions are true. Furthermore, actions may be attached to an arc to be executed whenever the arc is taken, and a set of registers can also be applied to store information. Usually, nodes are grouped into subsets and each subset is responsible for parsing a particular kind of syntactic constructions such as noun phrase, verb phrase, and preposition phrase.

Figure 2 gives an example of Augmented Transition Networks with two subsets, the sentence and the noun phrase. The ATN can accept a sentence which is composed of a noun phrase followed by a verb, and then another noun phrase. Each noun phrase in the sentence should consist of either a noun with a determiner or only a pronoun.

A sentence or a phrase is said to be acceptable if it starts from the beginning state and reaches the final state. While the parsing process is proceeding, the required information can be collected and manipulated using registers and actions on arcs to generate a syntactic structure tree such as the one shown in Figure 3.

Up to now, the ATN parsing technique has been successfully used in a variety of applications with quite complicated sentence structures. Figure 4 is a transition network diagram representing the ATN syntactic grammar used in the LUNAR system [Woods, Kaplan, and Nash-Webber, 1972].

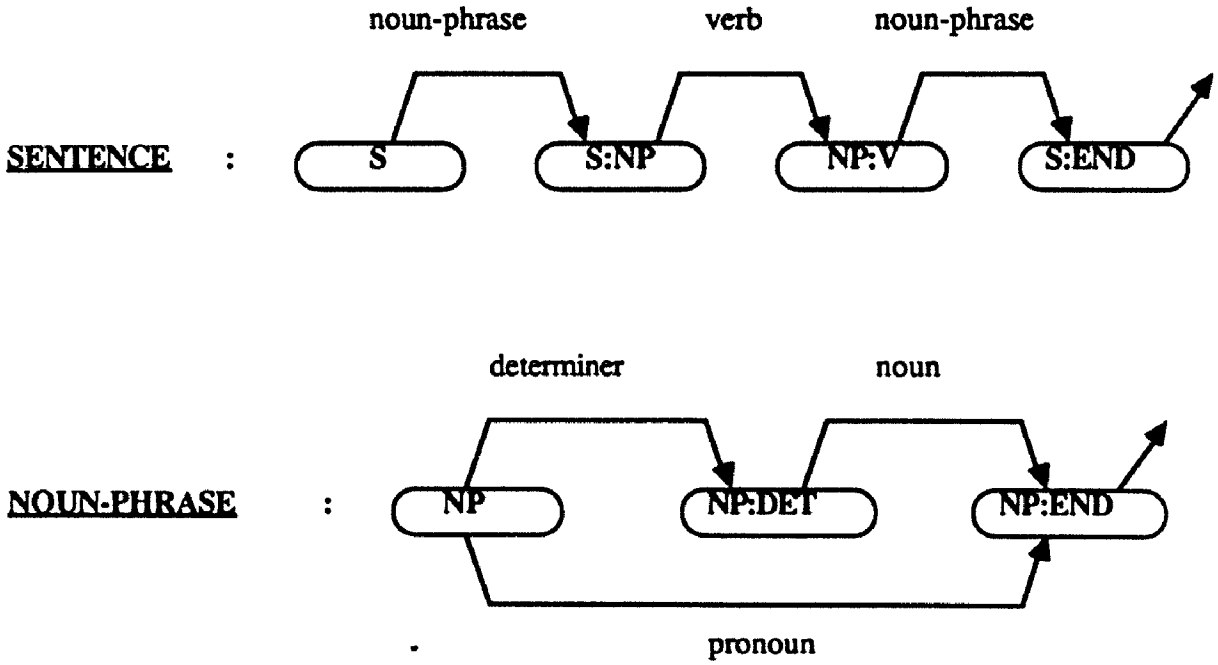


Figure 2 : An Example of Augmented Transition Networks

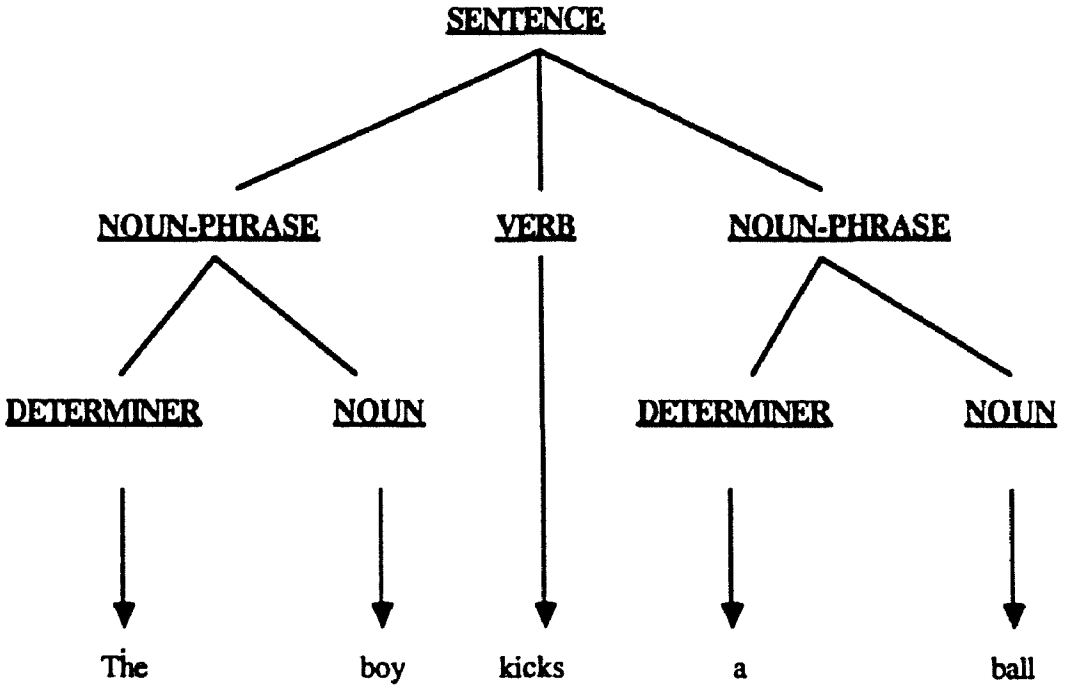


Figure. 3 A Syntactic Structure Tree for the Sentence,
"The boy kicks a ball."

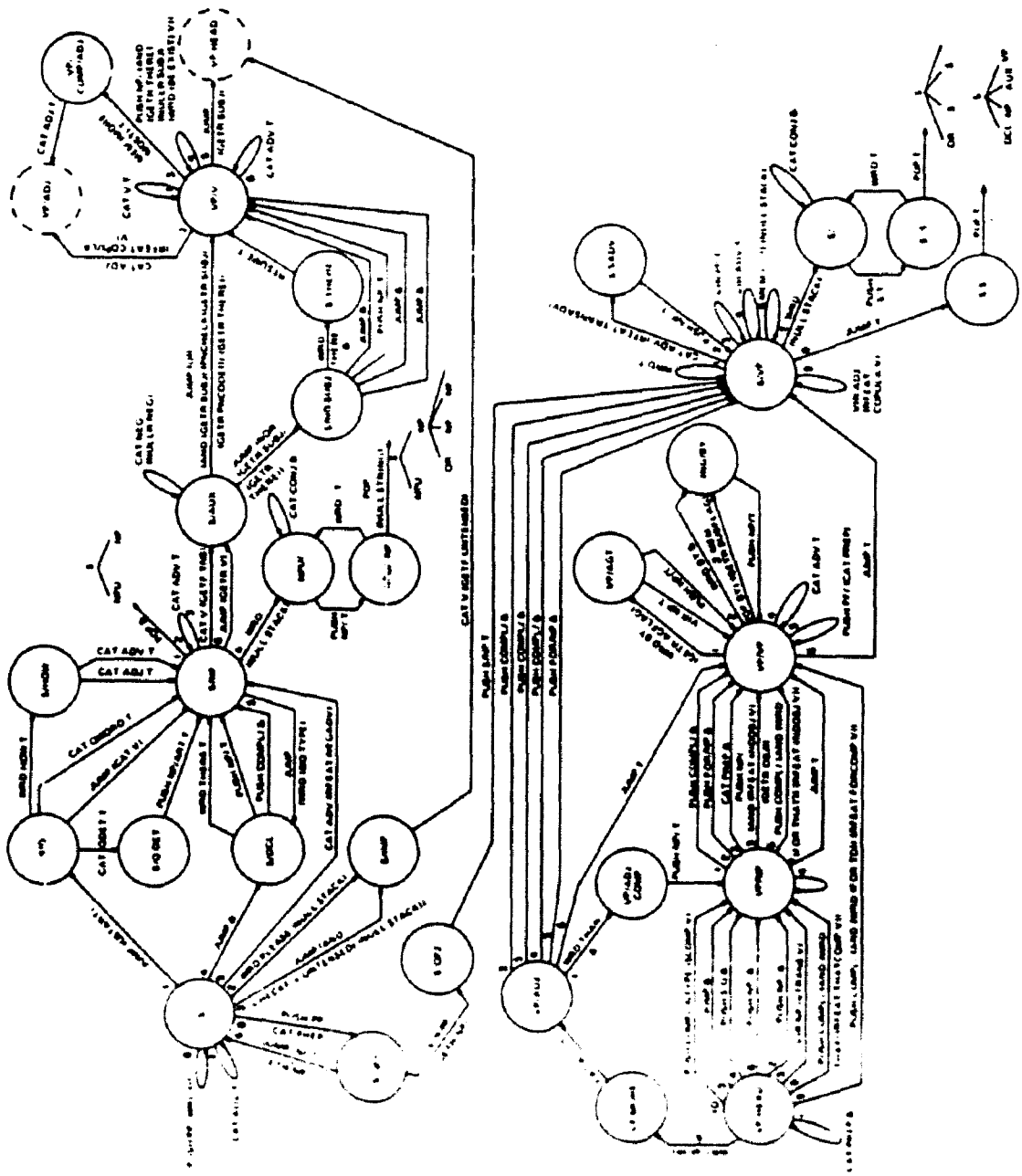


Figure 4. The Augmented Transition Networks
Used in LUNAR

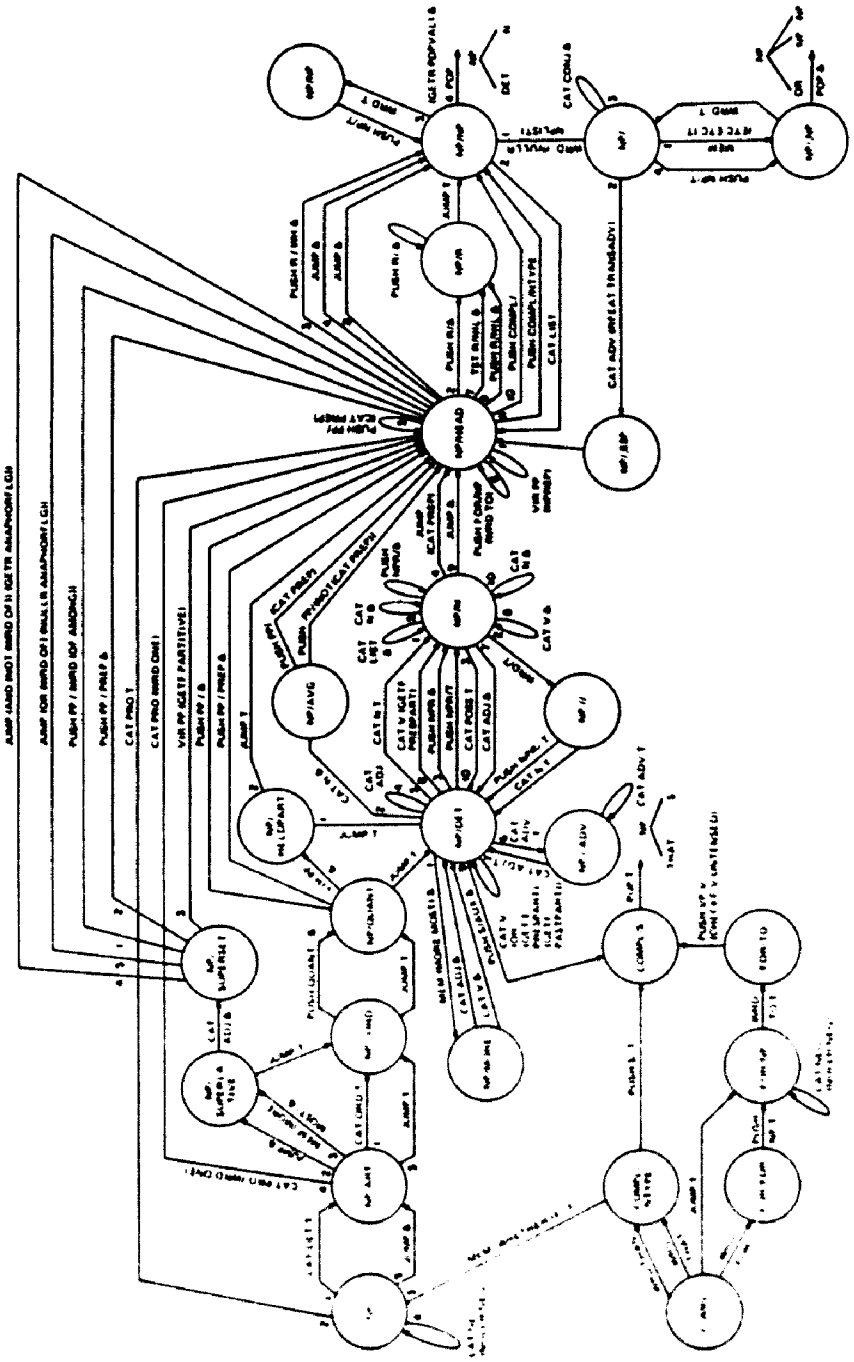


Figure 4. (Continued)

2.1.4. Ambiguity, the Problem of Natural Language Understanding

The research of natural language understanding starting from the machine translation in 1950's has been going on for more than 30 years. However, most of the current natural language understanding systems are still at the research stage. Even though there are a few commercially available systems, their functions are still limited because of syntactic and semantic constraints. The difficulty of language understanding is mainly the ambiguity, as described by Cater [1983] and Charniak and McDermott [1985] as follows:

1. Category ambiguity : This is because the part-of-speech of a word is not unique. For instance, the word "saw" can be a noun or a verb. Usually, these type of problems can be solved by identifying the syntactic role of a world in a sentence.
2. Structural ambiguity : Sometimes a sentence can be parsed into more than one syntactic structures. For example, in the sentence

   The part has been processed for 20 minutes at Station-1, (2.10)

   the preposition phrase "at Station-1" can be understood as modifying "processed" or "minutes." However, from the semantics, the word "minutes" can not be modified by the phrase "at Station-1." Thus, the deambiguity process requires world knowledge such as common sense.
3. Word sense ambiguity : This is the case when a word can have different interpretations. For instance, the meaning of the verb "take" in the phrase "take a bus" and "take a pill" is different. Usually, this type of problem can be resolved by receiving more information from the surrounding words; and the knowledge of the application domain can also contribute to reducing the number of interpretations.

4. Semantic ambiguity : This is the worst situation of all. It happens when a sentence has different syntactic structures, and each one of them does have a correct semantic interpretation of its own. The most well-known example is the sentence, "I saw the man on the hill with a telescope." Figure 5 gives its four different possible interpretations. This kind of ambiguity can only be solved by acquiring more knowledge from the context of the text. For instance, if there is a sentence before the example sentence saying that, "I brought a telescope," then we know that the preposition phrase "with a telescope" is used to modify the word "I" in this context.
5. Referential ambiguity : It is about pronouns and words with definite articles. The deambiguity tactic is to search through previous sentences to locate the corresponding word. In addition, the selection restriction strategy can also be applied here. For example, the word "he" corresponds to a word with the male feature.

Thus, a general natural language system is required not only to store information about the syntax of a language and vocabularies, but also to represent and manipulate a huge amount of real world knowledge; that is, semantic knowledge. And, this is why natural language understanding system is so difficult to develop.

2.2. Knowledge Representation Schemes

The knowledge stored in the Natural Language Modeling Environment generally can be classified into two major categories: the internal knowledge and the external knowledge. The internal knowledge is applied to process input sentences and to generate simulation programs in NLME. Whereas, the external knowledge is analysts' understanding of their

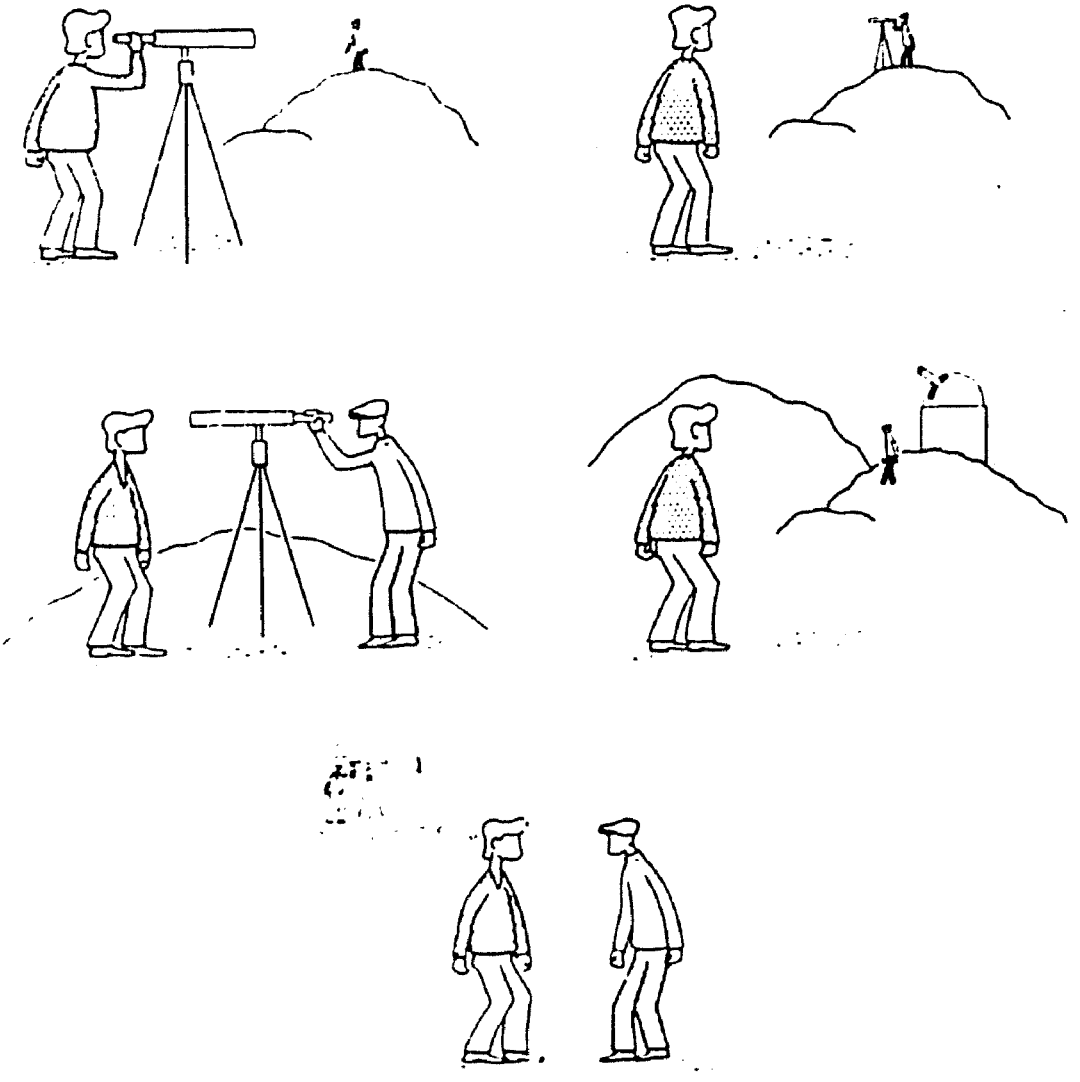


Figure. 5 Four Different Interpretations of the Sentence
" I saw the man on the hill with a telescope."
(reproduced from [Waltz,1982])

problems which are described in problem descriptions and captured by NLME. Both the internal and external knowledge are required to be represented in a computer implementable form.

In Artificial Intelligence (AI), knowledge representation schemes provide systematic ways of codifying knowledge about some domain. A knowledge representation scheme is a combination of the data structure used to store the knowledge about a particular problem domain, and procedures used to apply the stored knowledge. In AI systems, typical knowledge, which needs to be represented in systems, can be classified into four types [Barr and Feigenbaum, 1981]:

1. Object : Facts correspond to an object in a system. For example, "Machine 1 can only be used to perform a drilling operation" is a fact about the object machine 1.
2. Event : Knowledge about actions and events which have taken place or will taken place in the world. For instance, "Part A was transferred to station 2 by cart 1" is the event knowledge about part A. For this type of knowledge, it is important to have a representation formalism to indicate the sequence of events and their cause-and-effect relations.
3. Performance : Knowledge concerning how to do things, and also, performance skills such as the knowledge used by an experienced process engineer to decide the operational processes of a newly designed part. However, it is often difficult to draw a line between the object knowledge and the performance knowledge.
4. Meta-knowledge : Knowledge about the previously mentioned knowledge. In other words, this is the knowledge about what we have known. For example, if there are several rules that can be used to decide which machine would be the best to process a certain job, the knowledge of which rule is the most suitable one to be

applied is the meta-knowledge. This kind of knowledge includes knowing about the extent and origin of the knowledge concerning a particular subject, about the reliability of certain information, or about the relative importance of specific facts about the world.

Currently, several knowledge representation structures are available. However, some of the structures can be more appropriate to represent certain kinds of knowledge than others. In the following subsections, two of the well-known structures, production rules and frames, will be described along with their pros and cons.

2.2.1. Production Rules

Production rules are also called "condition-action" rules. The format of a rule is a condition-action pair as such:

IF C_1 & C_2 & C_3 . . .,
THEN A_1 & A_2 & A_3 . . ..

It means that if conditions, C_1, C_2, C_3 . . ., are true; then perform action A_1, A_2, A_3 . . .. Usually, the condition part of a rule is a conjunction of predicates that test properties of the current state of a system, and the action part of a rule is commands applied to change the state of that system. For example, a production rule could be:

IF station A is free
and station A is capable of processing job X,
THEN
load job X into the shop,

which can be applied in controlling a work station to represent the principle of selecting a job into a system.

The system equipped with production rules along with a working memory and a rule interpreter is currently the most widely employed structure by researchers of various disciplines to construct expert systems. The major reason is because the production rule is capable of expounding various information which includes [Hayes-Roth, 1985] :

1. Specific inferences that follow from specific observations;
2. Abstraction, generalizations, and categorization of given data;
3. Necessary and sufficient condition for achieving some goal;
4. Likeliest process to look for relevant information;
5. Preferred strategies for eliminating uncertainty or minimizing other risks;
6. Likely consequences of hypothetical situations;
7. Probable causes of symptoms.

In addition, other advantages [O'Hare and Bell, 1985] to use production rules to represent knowledge exist. First, the " if ... then ...." structure is one of the most natural ways of encoding human knowledge. Secondly, each production rule represents a small independent piece of information. The modularity of the structure enables a rule base to be easily expanded and contracted.

On the other hand, there are also some drawbacks concerning production rules. As the number of rules grows, the performance of the corresponding system becomes less and less efficient. Since all the rules are grouped into one pool without any partition, the inference engine needs to scan the whole rule base whenever the engine needs to decide which rule to apply. Moreover, in spite of the expressive power of production rules, they are inadequate for defining terms and for describing domain objects and static relationships among objects [Fikes and Kehler, 1985]. Because of the simple and restricted format of production rules, they are handicapped in representing objects and associated attributes.

Furthermore, their independent and modular formalism is incapable of organizing knowledge in a hierarchical structure.

However, the problem domain being studied, manufacturing system, consists of parts and facilities such as work stations and material handling equipment. It is essential for the knowledge base structure of the Natural Language Modeling Environment to be able to describe these objects and their relationships. This is because the simulation study of a manufacturing system required that the characteristics of the examined system's components and the ways that the components interact with one another be incorporated into a model. Therefore, the ability to represent relationships among components of the system under study should be the indispensable property of the knowledge base structure of NLME.

2.2.2 Frames

Frames are data structures used to group the knowledge about a particular object or event in terms of a record of "slots" and "fillers" [Minsky, 1975]. A frame can be considered as a complex node in a network, which has several storages or slots. These storages (slots) are used to store the information (fillers) including the type and attributes of the node and the names of the related nodes. An example frame for a spade drill process is shown in Table 2. Generally speaking, the frame scheme provides a structured means to describe domain objects which a system will have to build.

In addition, several special features exist which make a frame system different from other representation schemes. These features include [Bonnet, 1984; Jackson, 1986; Stefik,1979]:

Table 2. : A Frame for a Space Drill Process to Create a Hole H.
(reproduced from [Nau and Chang, 1983])

---

frame for a process to create a hole H
process name : spade-drill

---

list of restrictions :

1. diameter(H) > 0.75
2. diameter(H) < 4
3. length(H) < 4 * diameter(H)
4. pos-diam-to(H) > (.005 * diameter(H))$^{0.5}$ + 0.03
5. neg-diam-to(H) > (.004*length(H))$^{0.5}$ + 0.25
6. loc-to(H) > .008
7. straightness(H) > (.0004 * length(H)/diameter(H))^{3} + 0.002
8. parallelism(H) > (.0008 * length(H)/diameter(H))^{3} + 0.003
10. surface-finish(H) > 100

---

list of preconditions :

1. one of
   flat-surface S
   x-angle(S) = x-angle(H)
   z-angle(S) = z-angle(H)
   spot-face F
   hole(F) = H
2. hole-workspace C
   x-location(C) = x-location(H)
   y-location(C) = y-location(H)
   z-location(C) = z-location(H)
   x-angle(S) = x-angle(H)
   z-angle(S) = z-angle(H)

---

1. All the data about a particular object or event, and the corresponding constrains can be stored together.
2. A default value can be specified for a slot.

3. A frame system allows a builder to describe a prototype object, and to create instance objects according to this prototype. The relationship between a prototype and the instances can be [Brachman, 1983] : (a). member, (b). predication, (c). membership in kind, (c). conceptual containment, (d). description, (e). similarity to prototype, (f). specification of role filler, and (g). abstraction. Additionally, when these instance objects are created, they will automatically inherit the properties of a prototype object.
4. Each slot can have procedures attached to it. The procedures can be activated every time the value of the slot is stored or retrieved. This allows the value of a slot to be dynamically determined by the state of a system, or the state of a system to be modified whenever a slot is being instantiated. Particular procedures attached to a frame are IF-NEEDED, IF-ADDED, and IF-REMOVED.

Consequently, the frame structure provides a modular and hierarchical approach to organize knowledge and increase its accessibility.

Up to the present day, several Artificial Intelligence systems in industrial engineering area have been constructed based on frame structures. Their areas of application include process selection [Nau and Chang, 1983], job shop scheduling [Fox and Smith, 1984], fault diagnosis [Finin, McAdams and Kleinosky, 1984], and simulation [Nielsen, 1986], etc. The popularity of frame structures are because of their ability to provide an object-oriented structure to arrange information. The structure is most suitable for describing systems in the domain of manufacturing.

In a manufacturing system, a single frame can be used to denote a part or a process or a machine or a controller, and the corresponding information can be defined by using slots in the frame. For instance, if a frame is used to represent one of the operations of a

part, the information associated with the operation such as the required tools and fixtures, candidate machines, and previous and successive operations can be grouped and stored in the frame. Furthermore, a manufacturing system is hierarchical in nature, the whole system is organized from the highest factory computer controller level to the middle cell or station level, and then to the lowest machine level. Each component of the upper level is composed of several units of its lower level. For example, a manufacturing cell consists of machines, robots, and other requirement. This kind of hierarchical structure and membership relations can be easily described by a frame system.

Actually, the frame system is not foreign to the simulation community. For instance, a simulation network model consists of a variety of nodes to represent the characteristics of the system to be simulated, and is to some degree similar to a frame system. In the model, there is also a set of attributes corresponding to a node, and each node in some way relates to one another. Furthermore, each node in the model can be treated as a piece of knowledge about the system to be simulated; and this condition is just like the situation that each frame in a knowledge base would function. Just as a knowledge-based system can be used to predicate the state change of a system, a simulation model can also be used to inference the behavior of a simulated system. In addition, the case-frame structure described in Semantic Analysis can be treated as a variant of frame structures as well.

2.3. Modular Modeling Techniques

Modular modeling techniques provide a way to model a system using a set of pre-defined modules. A module can be treated as a macro instruction where each type of modules corresponds to a set of pre-defined simulation functions. These functions can be used either to change the states of a system, to perform a sequence of activities, or to cause the happening of events.

The modular modeling technique has been suggested as a method of modeling manufacturing systems[Medeiros, et al.,1980]. Since it is relatively simple to split a manufacturing system up into modules, and each type of module can correspond to a set of operations performed by a facility in the system, a model can be easily constructed by grouping different types of modules together. Besides, this method can also reduce the amount of written codes required and minimize the interaction between different functions when a model is modified [Crite, Mills, and Talavage 1984].

The modular modeling techniques have been employed to design several manufacturing simulators. For instance, PATHSIM [Crite, Mills, and Talavage, 1984] provides four types of modules: loading station, processing unit, unloading station and central tool warehouse. A MAP/1 [Miner and Rolston, 1986] model is basically composed of six types of modules -- PART, STATION, TRANSPORTER, CONVEYOR, OPERATOR and FIXTURE. SIMAN [Pegden, 1982] also provides the STATION block to group a set of blocks corresponding to the processing functions of a machine as a module.

2.4. Applications of Natural Language Processing

In this section, four of the leading examples of natural language understanding systems will be briefly reviewed to illustrate current applications of natural language processing. These four systems are SHRDLU, LUNAR, LIFTER, and GUS; and each of them has its own unique approach to deal with the problem of language processing under its own application domain.

1. SHRDLU [Winograd, 1972].

   Winograd's SHRDLU is one of the most widely known natural language processors. Its domain of discourse is the block world which consists of a table with a number of blocks of various sizes, shapes and colors. Moreover, SHRDLU is able to receive orders from users to manipulate these blocks. The key of the system's success is the integration of reasoning procedures, semantic analysis, and syntactic analysis. However, this system has been criticized that it can only operate in a very restricted domain of the block world.

2. LUNAR [Woods, Kaplan, and Nash-Webber, 1972].

   LUNAR was designed as an interface to a database that described the moon rock samples brought back by Apollo astronauts. Its language processing strategy operates in three steps. The first step is to perform the syntactic analysis based on Augmented Transition Networks, and a sentence structure tree will be produced at this step. The second step is to carry out the semantic interpretation of the structure tree to generate a query language expression. The query is then executed in the last step.

3. LIFTER [Hendrix, 1977].

   LIFTER, Language Interface Facility with Ellipsis and Recursion, is part of the internal research and development program of SRI international. It was designed to be an off-the-shelf utility for building "natural language front-ends" for applications in any domain. LIFTER is a semantic grammar-based parser. The design of LIFTER is to match a question received from a user to pre-defined sentence templates; and whenever a template is matched, a query expression is generated.

D. GUS [Bobrow, et al., 1977].

GUS, Genial Understanding System, is a natural language program written to study cases of frames in natural language processing, and its domain of discourse consists of airline reservations. The characteristic of GUS is that its frame embodies expectation of what the dialog will be during the process. GUS will go through its expectations and ask a user for each piece of information, one after another.

CHAPTER III

AN OVERVIEW OF

THE NATURAL LANGUAGE MODELING ENVIRONMENT (NLME)

The objective of this chapter is to present the strategy used in the Natural Language Modeling Environment (NLME) to attack the problem. The focus will be on the relationship between the manufacturing system descriptions and modeling concepts. In the last section, a general description of the NLME is also presented.

3.1. The Application Domain : Manufacturing Systems

One of the keys in developing a successful natural language processing system is to focus on a narrow application area. Theoretically, a general purpose natural language processing system can be constructed. However, it requires tremendous computational resources in practice. This is because of the ambiguity associated with words and sentences. For different problem domains, the interpretation of a word, a phrase, or a sentence may vary. Such an interpretation process requires a great deal of computational power and memory to store the information to support the process.

The domain of interest is the discrete part manufacturing system. A manufacturing system can be defined as a production facility that consists of a group of process equipment units and they are linked to one another with a material handling system. Parts which flow through the system are transformed by the process equipment and are transferred between units by a material handling system. The processing of the parts generally consists of a series of operations that must be performed in a specific order and on designated equipment units.

3.2. System Description : The Input Data

The function of the proposed NLME system is to simplify the modeling process of a simulation experiment and also to provide system analysts with the option of using alternative simulation systems. The proposed simplification process is through a natural language interface which allows analysts to describe their manufacturing systems in terms of everyday English. Consequently, the system description can be transformed into programs of different simulation systems.

A system description, such as shown in Table 3, consists of sentences used to describe the physical objects, activities, and attributes of the manufacturing system under study. The physical objects of a system include facility such as machine tools, material handling equipments, and parts. The activities taking place in a manufacturing system can be processing, inspection, or transportation activity. Furthermore, the attributes of the objects or activities include the lot size of a part, the machine number in a station, the buffer size of a station, and the duration of an activity

In a system description, each sentence represents pieces of information about the system to be modeled. However, a piece of information can usually be represented in more than one form. The information can be described by using the same words but in different syntax, or even by using different words with the same meanings. For instance, the next three sentences are all equivalent:

The time to service a part at Station-1 is 10 minutes.	(3.1)
The service time of a part at Station-1 is 10 minutes.	(3.2)
The operation time of a workpiece at Station-1 is 10 minutes.	(3.3)

Since the number of possible combinations of words and sentence structures is quite large, it is impossible to design rules to accommodate each one of them and, consequently, to

Table 3 . An Example of a System Description.

The cast part enters the CIM work center.
The part arrives at the milling station according to a Poisson process with a mean of 3 minutes.
The station processes it for 5 minutes.
The milling station has a waiting space for 1 part.
The station has 2 machines.
Then the cast part goes to the drilling station.
The transfer time is uniformly distributed from 1.5 to 2 minutes.
The process time at the drilling station is normally distributed with a mean of 2 and a variance of 0.5 minutes
There are 2 machines in the drilling station.
Also, the station has a waiting space for 2 parts.
Next, the cast part is moved to the polishing station.
The transfer time is exponentially distributed with a mean of 2 minutes.
The part is processed for 3 minutes.
Then the cast part leaves the CIM work center.
The polishing station has 1 machine.
Also, there is a waiting space for 1 part in the station.
The CIM work center is to be simulated for 24000 minutes.

---

generate simulation programs. Therefore, to reach a resolution, those statements using different words and sentence structures, but with the same meaning, need to be converted into the same internal representation. In order to serve the NLME purpose, to generate simulation program, the design principle of the internal representation is based on the simulation modeling concepts which will be discussed in the next section.

3.2.1. Modeling Concepts for Manufacturing Systems

Generally, a simulation model set up to study a system is to reproduce activities in which the entities engaged in, and to move the simulated system from state to state based on the occurrences of the activities. For instance, to study the utilization of a machine station, we could generate part arrival events, part in service activities, and part departure events. At the same time, the state of the machine station can be changed from idle to busy or from busy to idle. Statistics can then be collected by observing the changes in the system states.

Evidently, the event / activity, the entity, and the state of a system are three major elements of the corresponding simulation model. Moreover, the relations between the model elements and the manufacturing system are described as follows.

1. Entity : There are three major types of entity in the domain of manufacturing. Two of them with mobility are the part and the transporter, and the third one, stationary entity is the station.
2. Event/Activity : There are three basic types of event/activity in a manufacturing system. One is the arrival event which is about the entering of workpieces into the system; the second one is the operation activity which denotes the process of a part being served in a station; and the last one is the transportation activity which

represents the movement of a part from one station to another with the help of a material handling equipment.

3. State : States of a manufacturing system are usually the status of station and material handling equipments such as busy, idle, or breakdown.

In addition, for each entity, event, and activity there is a set of attributes associated with it. For example, a part entity has attributes such as lot size, operation sequence,..., and a station entity has attributes such as machine number, buffer size,... . Furthermore, for an operation activity , a duration attribute is associated with it.

3.2.2. Words, Phrases, Sentences, and Modeling Concepts

As it was discussed previously, several ways can be applied to represent a piece of information. It can be presented using different words or different sentence structures. In order to simplify the NLME language processing procedures, different words or different sentence structures with the same meaning are converted to a standard internal representation using the modeling concepts.

"How do words in a sentence relate to modeling concepts ?" This issue is discussed according to the part-of-speech of words as follows:

1. Noun : Nouns and noun phrases are most frequently used to describe entities. For instance, words such as workpiece, item, and unit all refer to the same modeling concept, the part type entity; and words such as truck, AGV, conveyor, and crane are used to describe the transporter type entity. Moreover, words such as cell, station, and machine center are used to represent station type entity. In addition, attributes are usually illustrated by nouns or noun phrases such as duration, lot size, and machine number.

2. Verb : Events and activities in a sentence are mainly depicted by verbs such as arrive, process, serve, inspect, transfer, and move. Usually, the verb in the sentence is to indicate an event or activity and the noun phrases associated with the verb in a sentence are to describe the entities engaging in the activity or the event. For example, in the sentence, "A pcb is moved from the TDK station to the VCD station," the verb "move" is used to specify the transportation activity and the noun phrases "the pcb," "the TDK station," and "the VCD station" are to describe the entities involving the activity. Furthermore, verbs such as "be" and "have" are usually used to assign values to attributes such as "the station has 2 machines." and "the process time is 10 minutes."
3. Adjective : Adjectives such as busy, free, available, idle, and full are used to indicate the state of a system or entity. In addition, adjectives are also used to serve the purpose of distinguishing the same type of model elements. For example, both "milling station" and "drilling station" belong to the same type of station entity; however, they can be distinguished by their adjectives drilling and milling.
4. Article and pronoun : The definite article and pronoun are used to serve the referential purpose.
5. Preposition : A preposition in a sentence can be used to signal the function of the subsequent noun phrase. Basically, a preposition in a statement of a problem description is used to indicate :

    a. Location such as "before," "after," "in," "at" and "between,"
    b. Duration such as "for" in "for 10 minutes,"
    c. Instrument such as "by" and "with,"
    d. Size such as "for" in "a space for 10 items,"
    e. Starting point such as "from,"
    f. Ending point such as "to."

6. Conjunction : There are two types of conjunction that require special attention in terms of modeling concepts. One is the conjunction which is used to indicate sequences of events and/or activities, such as "before" and" after," and the other is to indicate states of conditions such as "if," "when," "otherwise" and "or."

To summarize, the strategy of the NLME is basically to map the external representations, system description statements, to the internal target representations, modeling concepts. For each statement in the description, the operation process is first to identify individual model elements such as entities, activities, and attributes from the statement. Then these model elements are grouped in a predefined format.

As to how these elements are organized and how they are related to one another, that is determined by the information from other words in the sentence, by the sentence structure, and by the function of the statement. However, the underlying principle is based on the case grammar which has been discussed in the previous chapter. Furthermore, the function of each statement in a system description under the domain of manufacturing can be categorized as follows:

1. Specifying parts and facilities in a system.
2. Describing an event/activity in a model and the participant parts and/or facilities.
3. Specifying the value of attributes of events, activities, parts or facilities.
4. Indicating the status of facilities or system, which corresponds to the happening of an event or activity.

Finally, a more detailed illustration of the language processing tactics will be provided in the next chapter.

3.3. How the System Works : A General Description of the NLME

In the next two chapters, chapters four and five, each component of the NLME will be discussed in detail, The discussion will include its role in the system and the structure and the control logic. However, before going into individual component, a general view of how the whole system works will be provided in this section. The overall flow of control in NLME is summarized in Figure 6, and its structure is presented in Figure 7.

Figure 6 presents the order in which the NLME processes an input. Each statement in a system description file is first read into the NLME system and feed into the language processor. The objective of the language processor is to examine the input sentence and to generate the corresponding semantic representation. In order to achieve the objective, each component in the language processor is designed to be responsible for specific functions.

1. PreProcessor : To check whether words in an input sentence exist in the Lexicon or not, and to combine words into phrases.
2. Augmented Transition Networks (ATN) : To define the acceptable sentence structures.
3. Syntactic Analyzer : To decide the structure of an input sentence using the ATN and the Lexicon.
4. Semantic Synthesizer : To generate the semantic representation of an input sentence using the information from the Syntactic Analyzer and the Lexicon.
5. Lexicon : To store both the syntactic and semantic information about words, which are used by the Syntactic Analyzer and the Semantic Synthesizer.
6. Short-Term-Memory : To resolve the referential ambiguity which is caused by the pronoun and the definite determiner.

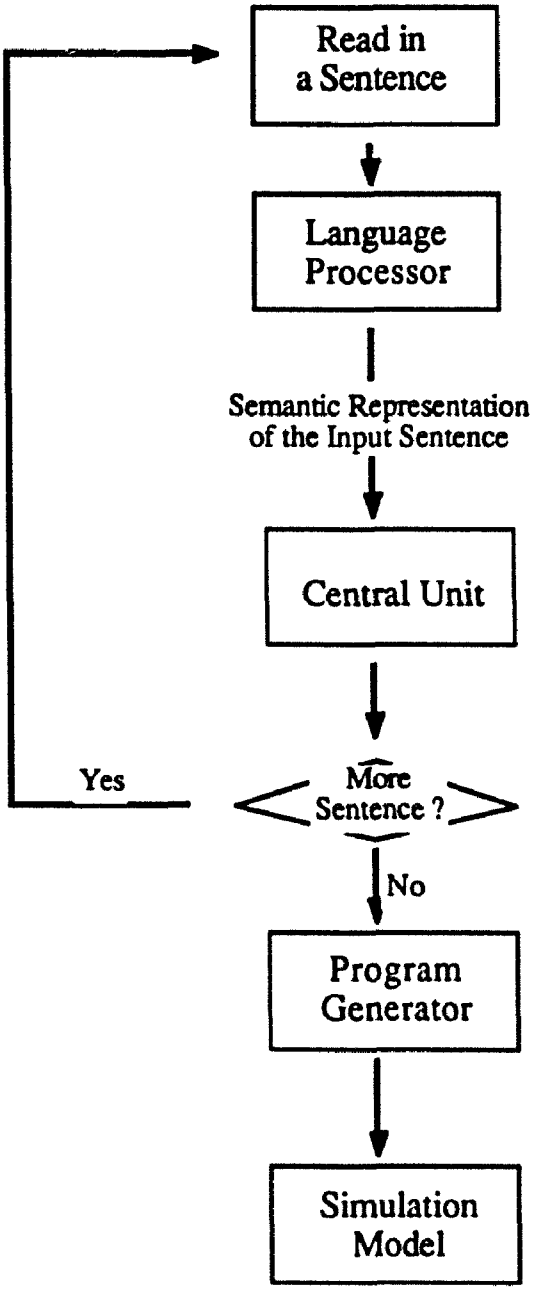


Figure 6. The Control Flow of the Natural Language Modeling Environment

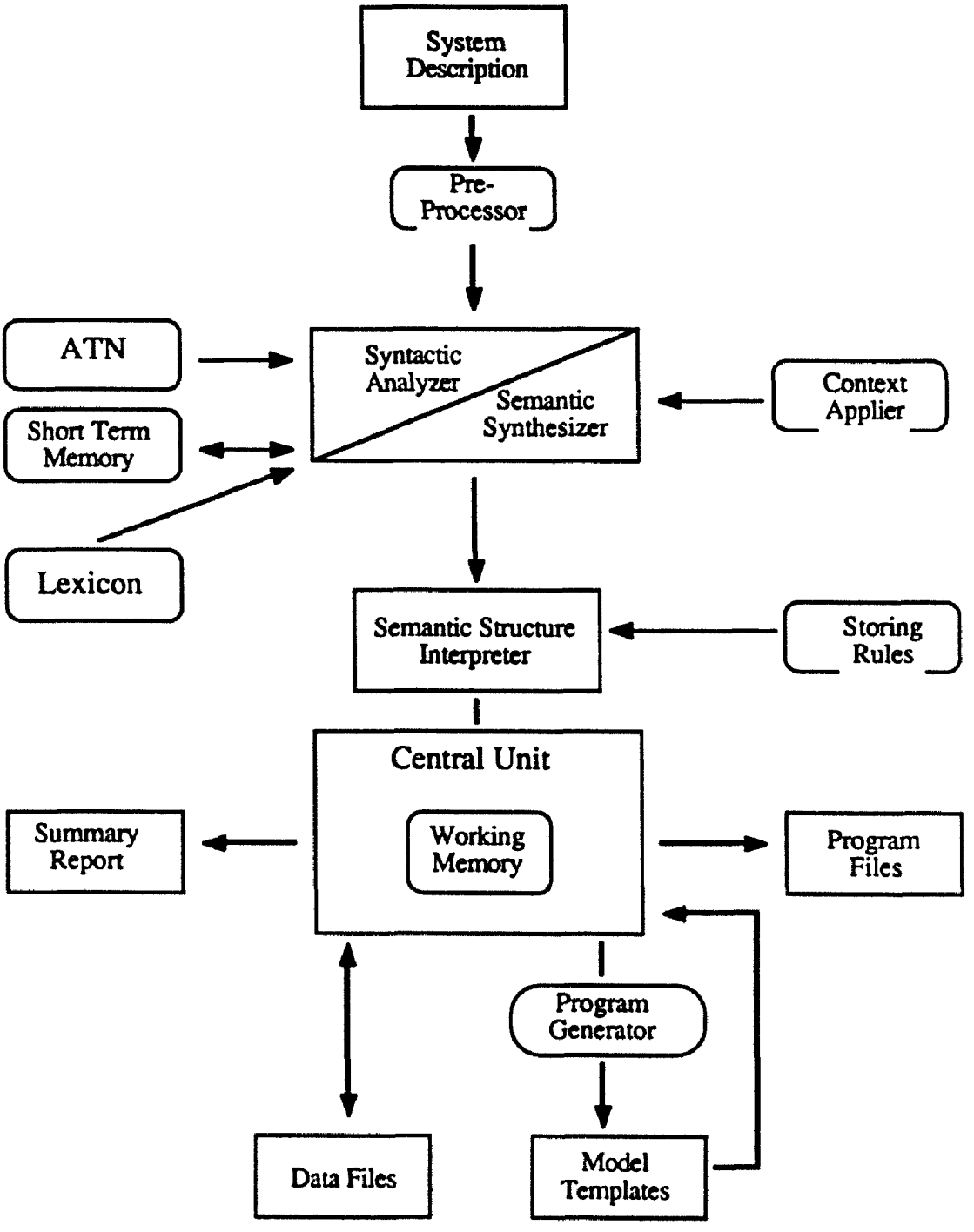


Figure 7. The Structure of the Natural Language Modeling Environment

7. Context Applier : To resolve the semantic ambiguity, when the information carries by a sentence is incomplete and it can only be understood from previous input sentences.

Furthermore, the semantic representation from the language processor is sent to the Central Unit. The Central Unit examines the representation and decides how to store it according to the organization of the central unit's working memory. The above processes, from reading a sentence to storing the semantic representation of it , are repeated until the input file is exhausted. Then, a summary report of the information collected from the input file of a system description can be generated and the corresponding simulation model can also be constructed automatically.

Basically, the central unit consists of three major components. The function of each component is described as follows.

1. Semantic Structure Interpreter : To decide how to store the semantic representations of input sentences into the working memory of the Central Unit.
2. Working Memory : To store the information from semantic representations of a system description.
3. Program Generator : To construct a simulation model using the information stored in the Working Memory.

CHAPTER IV
THE STRUCTURE AND FUNCTIONS OF THE LANGUAGE PROCESSOR

The Language processor is basically divided into three parts, the Syntactic Processor, the Semantic Processor, and the Lexicon. The structure of the Language Processor and its control flow are presented in Figure 8 and Figure 9. The two processors are to perform syntactic and semantic analysis of input sentences, and the required information is stored in the Lexicon. Furthermore, the Syntactic processor consists of two components. The PreProcessor is to check the correctness of words in sentences, and the Syntactic Analyzer is to delinearize sentences into their grammatical constructions. Moreover, each construction is sent to the Semantic Synthesizer to form a semantic representation. Lastly, all the representation of constructions in a sentence are combined to form the sentence's representation. In addition to the Semantic Synthesizer in the Semantic processor, the Short Term Memory is used to resolve the referential ambiguities, and contextual rules and the contextual memory are applied by the Context Applier to resolve semantic ambiguities.

4.1. Case Grammar : The Theory Behind the Language Processor

Despite that most of the grammatical rules only provide ways to interpret the syntactic structure of a sentence, the case grammar uniquely offers a method of combining syntactic structure with the semantic interpretation. Generally speaking, most of the grammatical rules only allow one to examine a sentence in terms of its syntactic structure. The grammar itself does not reveal the information on how the syntactic components of a sentence semantically relate to one another.

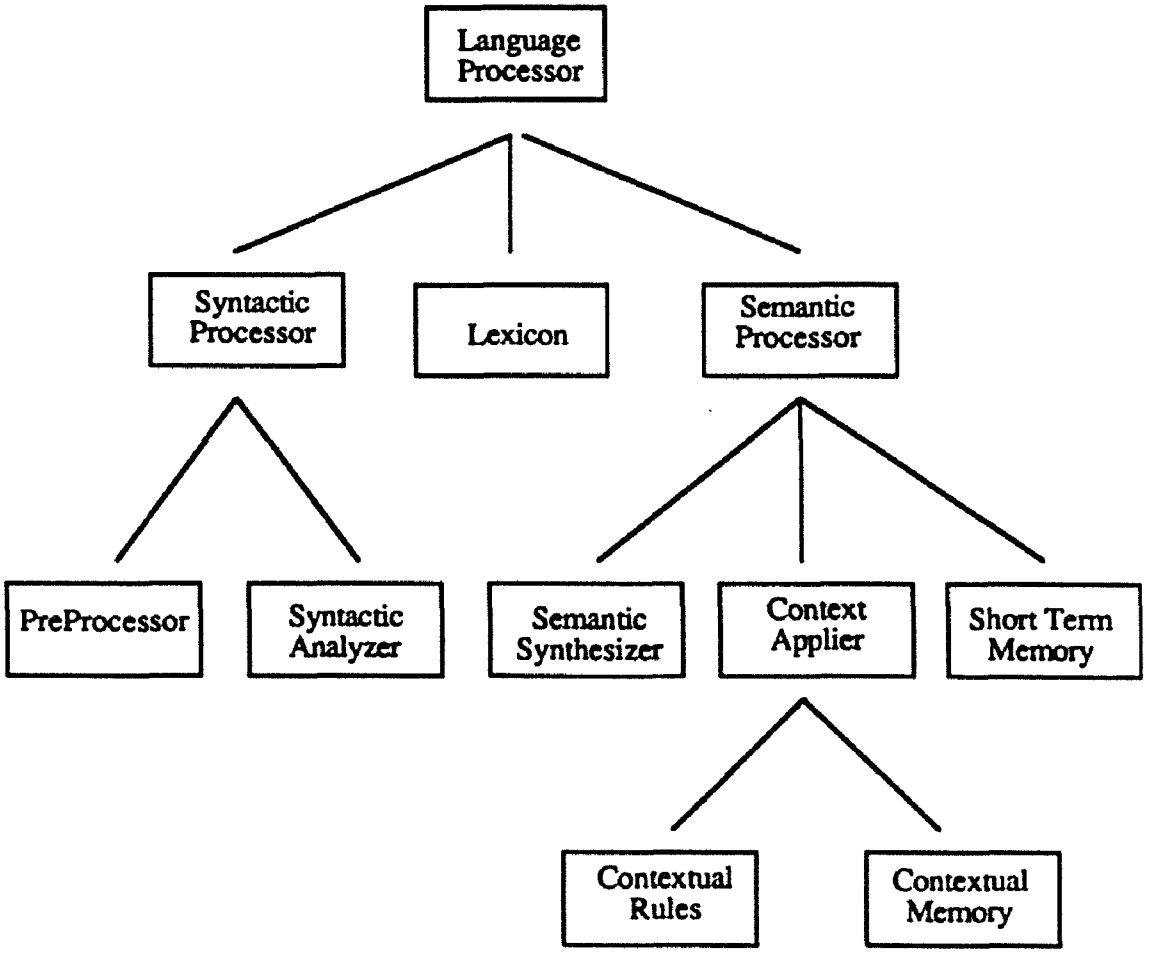


Figure 8. The Structure of the Language Processor

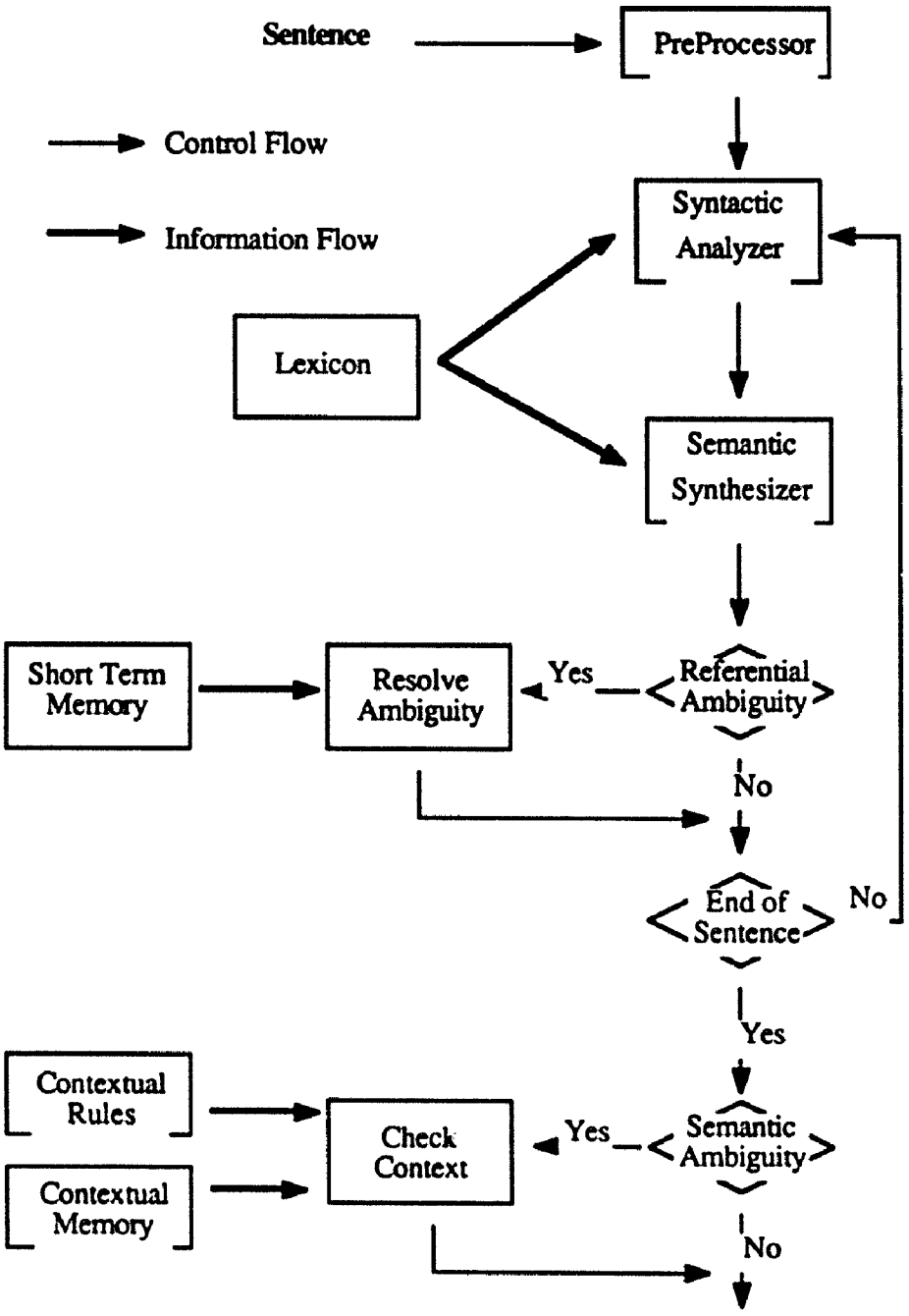


Figure 9. The Control Flow of the Language Processor

However, unlike regular grammar, the case grammar says that the proposition under a simple sentence has a deep structure that consists of a verb and one or more noun phrases . Furthermore the noun phrases are associated with the verb in some particular relationships, and they are called cases. These semantically relevant syntactic relationships in a sentence are essential to the design of a natural language processing system. They are not only to provide a bridge between syntactic processing and semantic processing, but also to offer efficient and thoughtful ways to identify simulation model elements and their relationships from the system descriptions.

As was discussed in the previous chapter, a simulation model is used to recreate the events and activities which take place in a system under study. Therefore, it is crucial to be able to identify the events and activities, and what and how entities undertake them from the system descriptions. Generally, activities can easily be recognized from the action verbs such as process, move, and arrive, in the sentences. In addition, the case grammar also gives a method to categorize verbs according to sentence types, and for each sentence type, a case frame exist which indicates what case relationships may exist between a verb and noun phrases. Here, the relationships between a verb and its noun phrases are just as the relationships between an activity and its entities.

For example, in an operation activity, there should be a part entity to receive the operation, and a station entity to perform the operation. The condition is just the same as an operation type verb which has the part, station and duration cases. Here the duration case is to indicate the time required to perform the operation. Furthermore, for those verbs which correspond to the transportation activity such as transfer and move, they can have the same cases such as part, starting station, destination station, and condition. Most of these cases are precisely the entities required to participate in a transfer activity. Currently,

several types of case frames have been implemented in the prototype NLME system, and they are given in Table 4.

Table 4. Case Frames Used in the NLME Prototype

Case Frame : Transfer
Case Part.
Case From (Starting Station).
Case To (Destination Station).
Case Duration.
Case Condition.

Case Frame : Arrive.
Case Part.
Case To.
Case Duration.

Case Frame : Operation.
Case Part
Case Station.
Case Duration.

Case Frame : Attribute.
Case Type : Duration.
Case Event : Operation.
Case Part.
Case Station.
Case Duration.

Case Frame : Attribute.
Case Type : Duration.
Case Event : Transfer.
Case Part.
Case From.
Case To.
Case Duration.

Table 4. Continued.

Case Frame : Attribute.
Case Type : Duration.
Case Event : Arrive.
Case Part.
Case To.
Case Duration.

Case Frame : Attribute.
Case Type : Machine-No.
Case Station.
Case Value.

Case Frame : Attribute.
Case Type : Buffer-Size.
Case Station.
Case Value.
Case State.

The case grammar also furnishes a link between syntactic constructs and semantic components. Generally, the cases of a verb are conformity to specific types of syntactic component or to specific types of preposition phrases in a sentence. If the cases are embodied in preposition phrases, the beginning prepositions in the phrases are to signify the types of case. For example, in the sentence,

The cast part is transferred from the milling station to the drilling station, (4.1)

the connections between cases and syntactic components are :

Case Frame : Transfer

Case	Value	Syntactic Construct
Part :	The cast part	Subject
From : (starting station)	The milling station	Preposition Phrase (from)
To : (destination station).	The drilling station	Preposition Phrase (to)

In addition to locating cases from syntactic components of a sentence, the map between syntactic roles to cases also allows the semantic verification of a sentence. This is because cases in a case frame can only be instantiated by objects with specific natures. For instance, the cases corresponding to the verb process are part , station, and duration. The part case, the station case, and the duration case should be filled by objects with work piece, machine, and time characteristics. Therefore, the syntactically correct sentence "The PCB is processed by FMC" can be semantically incorrect, if the nature of the PCB is not work piece, or if the nature of the FMC is not machine or an object with processing capability. Thus, if the features of cases do not agree with the natures of objects described by the words under the corresponding syntactic components in a sentence, the sentence can be semantically incorrect despite its syntactic correctness.

Finally, besides the characteristics of the case grammar described above, the case grammar presents the potential usage of the proposed system in languages other then English. Because the case grammar provides a base which can be applied to any language employing the apparent linguistic universals, nouns and verbs. The relations defined in the base do not seem to be language specific, .ie., they apply to many languages and ultimately prove to be universal relations [Baron, 1974].

4.2. Lexicon : The Knowledge Base of the Language Processor

The language understanding process is complicated and requires the support of a variety knowledge. Let's examine the following sentence:

The cast part is processed at the polishing station for 10 minutes. (4.2)

In order to understand the sentence (4.2), we first need to know the structure of the sentence. The sentence is in passive voice and the object of the verb should be the noun phrase,"the cast part." Also, there are two preposition phrases and they are used to modify the verb "process." Furthermore, the word "cast" is served as an adjective to modify the noun "part," and so is the word "milling" to the noun "station."

Based on the sentence structure and the meaning and usage of words in the sentence, we can interpret the sentence as :

A part which is made of cast iron is transformed at a station,
the station is capable of polishing process,
and the time required to perform the operation is 10 minutes. (4.3)

Moreover, if we examine the sentence in terms of modeling concepts, we can conclude that it is used to described an operation activity. There are two entities, the cast part and the polishing station, participating the activity and the duration of the activity is 10 minutes.

From the above interpretation procedures, basically we can conclude that understanding sentences requires two types of knowledge. One is the syntactic knowledge, and the other one is the semantic knowledge, if we treat modeling concepts as part of the semantic knowledge. The syntactic knowledge is about grammatical characteristics of words, phrases and sentences, and the semantic knowledge is about the meaning and usage of words.

In addition to the syntactic and semantic knowledges, the world knowledge and the contextual knowledge are also essential to the understanding process. The world knowledge can be described as our common sense and the knowing about the problem domain. The world knowledge can be very helpful in interpreting a sentence, when it is ambiguous. For instance, in the sentence (4.2), the preposition phrase, "for 10 minutes" can be used to modify the noun "station" or the verb "process," and both interpretations are syntactically correct. However, common sense tells us that a station cannot accept a duration as a modifier. Therefore, the phrase "for 10 minutes" is used to modify the verb "process."

As for the contextual knowledge, it allows us to fully understand a sentence by looking through the context of a description. The contextual knowledge is generally used to resolve the referential and semantic ambiguities. For instance, for the sentence :

The part is processed for 10 minutes, (4.4)

if we would like to know what part is processed for 10 minutes and where the operation is performed. These two questions can only be answered from the context. If we look through the context and find that its previous sentence is:

The cast part is transferred to the drilling station, (4.5)

we can conclude that the part is a cast part and the station is the drilling station.

To summarize, the process of understanding sentences generally requires the assistance of four different kinds of knowledge. They include the following [Gershman, 1979]:

1. Syntactic Knowledge
2.Semantic Knowledge

3. World Knowledge
4. Contextual Knowledge

The Lexicon in the Language Processor is basically designed to store the first three kinds of knowledge in order to support the operations of the Syntactic Analyzer and the Semantic Synthesizer. As to the mechanism used to represent and apply the contextual knowledge, it will be discussed in the section Contextual Rules and the Contextual Memory.

4.2.1 Syntactic Knowledge

The grammatical information stored in the Lexicon under each word can generally be classified into seven categories. They are part of speech, number type, determiner type, verb pattern, verb tense, root, and phrases. The usage of each category is explained as follows:

1. Part of Speech : The part of speech of a word is used to determine its syntactic role in a sentence.
2. Number Type : The information in this category is used to verify the subject - verb agreement of a sentence. It includes the number form of a noun, a pronoun, a proper noun, or the subject that a verb can take.
3. Determiner Type : The information is to denote that a determiner is definite or indefinite.
4. Verb Pattern : The verb pattern information is helpful to predict what kind of structure a sentence can be during the parsing process. Verb patterns basically can be divided into four classes :

    A. Intransitive Verbs : which can not take a direct object
    B. Transitive Verbs : which can take a direct object
    C. Bitransitive Verbs : which can take a direct object plus an indirect object

D. Others, such as verb which need to be followed by an infinitive

5. Verb Tense : The tense of a verb can be present, present participle, past, and past participle. They are applied to define a sentence's tense, voice, and aspect.
6. Root : The root information is to indicate the root of a verb when it is not in the present form. In addition, the root also allows a word to share the information stored under other words. This will be discussed later in this section.
7. Phrases : The information is to tell the PreProcessor to combine the current word with the following words into a phrase.

Finally, the possible syntactic information required to be stored under each type of part of speech is summarized in the Table 5.

4.2.2. Semantic and World Knowledge

In the Lexicon, there are three types of information stored under each word, which are related to the semantic and world knowledge. One of them is the modifier which belongs to the world knowledge. The other two types of information are the feature and the semantic representation structure, which belong to the semantic knowledge. The functions and structures of them are described in the following paragraphs.

4.2.2.1. Semantic Representation Structure

The design principle of the semantic representation structure of a word is based on the case grammar and the modeling concepts. Keep in mind that the structure is used not only to describe the meaning of a word, but is also used to formulate a simulation model.

Generally, the meaning of natural words can be represented by complex concepts built out of primitive concepts and/or other concepts. For example, the noun phrase "the red Volkswagen" is composed of the concepts, "the color of the car is red" and "the brand

Table 5. A Summary of the Syntactic Information Stored under each Type of Part of Speech

Part Of Speech	Category	Possible Values
n (Noun)	Number	s (Single) pl (Plural) 1s (First Person Single) 2s (Second Person Single) 3s (Third Person Single) 1pl (First Person Plural) 2pl (Second Person Plural) 3pl (Third Person Plural).
	Root	
pronoun	Number	s, pl, 1s, 2s, 3s, 1pl, 2pl,3pl.
propnoun (proper noun)	Number	s, pl.
det (determiner)	Type	def (Definite) indef (Indefinite).
	Number	s, pl.
adj (Adjective)	--------	
adv (Adverb)	--------	
v (Verb)	Verb Pattern	vi (Intransitive Verb) vt (Transitive Verb) vb (Bitransitive Verb) vinf (Verb + infinite clause).
	Number	s, pl, 1s, 2s, 3s, 1pl, 2pl, 3pl.
	Tense	present (Default) presp (Present Participle) past pp (Past Participle)
	Root	
prep (Preposition)	---------	
conj (Conjunction)	---------	

of the car is Volkswagen." Moreover, a primitive concept is defined as a concept which does not consist of the other concepts. However,the degree of the fineness of a primitive concept depends on the requirement of an application. A primitive concept in our application can be a concept in the others.

Usually, a primitive concept, in the NLME system, is used to stand for the meaning of a word. Its structure is the pair of a semantic marker and the word. However, in order to keep the representation structures consistent to all the synonymous words, we usually pick up one word from them to represent the others in the structures. Furthermore, the semantic marker assigned to a word is to indicate that the word describes a modeling concept or a generic concept of a particular semantic class. For example,the words "red," "blue," and "yellow," belong to the same generic concept, color, and the structures used to denote them are (color red), (color blue), and (color yellow). Moreover, "idle" and "available" describe the same modeling concept, state, and the conceptual structures of both them are (state idle).

A concept is used to represent the meaning of a phrase or a clause or a sentence. The structure of a concept is a semantic marker and a set of slots or cases. Each slot is to signify a supporting primitive concept or concept. The structure used to describe a concept is called conceptual frame and it is similar to a case frame when the case frame is used to represent a sentence. Furthermore, slots in a conceptual frame play an important role in differentiating the meanings of words referring to the same conceptual frames.

In addition to the structures used to represent the meanings of words, it is also essential to know when a specific sense of a word should be used. Generally, more than one interpretation is associated with a word, and a phrase or a sentence can only be understood if we choose the correct meanings for words in the phrase or the sentence. Several methods can be applied to decide a word sense in a sentence. The first one is to

narrow down the number of possible interpretations according to the application domain. Then, the interpretations can be filtered using the information of the word's syntactic role in the sentence and the surrounding word. For instance, in the noun phrases "Poisson process" and "operation process," both have the same main noun "process." However, their interpretations are quite different. In the first noun phrase, "Poisson process," because the modifier "Poisson" has the notion of function, the "process" can be interpreted as distribution or function. As to the second phrase, "operation process," the "process" can be interpreted as activity based on the its modifier "operation."

Words can also be used to propose conceptual frames for the high level constructions such as phrases and sentences. According to the case grammar, sentence structures can be signified by verbs in sentences. Sentences can be described by verbs with sets of cases that may be found in the sentences, and verbs can be categorized by sentence types. Following the case grammar principle, we apply the same methodology to the noun as case frame to the analysis of a verb, and let nouns indicate the potential noun phrase structures. For each noun we tried to find a corresponding conceptual frame according to the domain of application. The conceptual frames have semantic markers based on main nouns of noun phrases, and a roster of optional and mandatory slots or cases. The slots represent the possible supporting concepts for the main noun. Moreover, all surface forms in the noun phrases having the equivalent conceptual content could be mapped into the slots. For example, the noun station can imply the conceptual frame :

```
(station
        slot function
        slot type
        slot name   ).
```

If we fill the conceptual frame with the noun phrase "a milling station," it becomes :

```
(station
        slot function : milling   ).
```

Furthermore, if we instantiate it with the noun phrase "a TDK drilling station," it becomes :

```
(station
        slot function : drilling
        slot name : TDK ).
```

To summarize above discussion, the semantic representation structure of a word in the Lexicon should be equipped with three kind of capabilities. The structure should be able to :

1. Present the semantic representation of a word.
2. Decide the proper word sense for a word based on its surroundings.
3. Propose the potential conceptual frame for the higher level structures.

In order to accomplish the functions described above, the semantic representation structure of a word consists of a set of rules, and each rule corresponds to a specific meaning of the word. The rule is composed of 3 components. They are a condition, a proposition, and a semantic representation. The function and structure of each component are described as follows.

1. Condition : The condition of a rule is to specify the situation that the corresponding word sense in the rule can be used. The structure of a condition is a LISP function which can be evaluated to be true or false. The functions used most frequently here are **feature** and **getr**. The function **feature** is to test the generic concepts or the modeling concepts associated with other words in a sentence. The **getr** is to retrieve the values of the system registers which used by the Syntactic Analyzer to store words according to their syntactic roles in the sentence. For instance, in a noun phrase, the surrounding words of the main noun should have the syntactic roles of adjectives, or determiners, or quantifiers, etc. Basically, the registers provide an easy way to identify a word's surrounding words through their

grammatical categories in a phrase or sentence.

2. Proposition : The proposition represents the proposed higher level conceptual frame. The basic format of a proposition is a special type of conceptual frame and it is signified by a special character "!". The structure of the proposition is described as follows.

Format :

!(<main concept> ?(<supporting concept 1> <default or delay> <syntactic information>)
?(<supporting concept 2> <default or delay> <syntactic information>)
?(<supporting concept 1> <default or delay> <syntactic information>)
. . .

)

Example : A conceptual frame for the verb "go"

!(event (Type transfer)
?(part nil subj)
?(from nil from)
?(to nil to))

i. <main concept> : it is to denote the main concept associated with the proposed higher level construction, and it is indicated by a semantic marker.

ii. <supporting concept> : it is to represent the expected concept which can be found in the current level constructions to support the higher level <main concept>. The <supporting concept> is also indicated by a semantic marker.

iii. <default or delay> : Two types of information can be specified in this filed. One is used as the default semantic representation corresponding to the <supporting concept> in the prior field, when the concept cannot be found in the current level. However, the key word delay is to preserve the <supporting concept> to the next higher level of grammatical construction if the <supporting concept> cannot be found in the current level. In addition to the default value and key word delay, "nil" can also be used in here to indicate that if the <supporting concept> can not be found, it is deleted from the conceptual frame.

iv. <syntactic information> : It is to tell the Semantic Synthesizer that under which grammatical categories, the corresponding <supporting concept> can be located. The value of the <syntactic information> can be det (determiner), adj (adjective), subj (subject), obj (object), etc.

3. Semantic Representation : The structure represents a particular word sense corresponding to the condition in a rule. Sometimes, the semantic representation can be incorporated into the proposition in a rule and the value in this field is "nil."

Finally, the semantic representation structures of a word is cataloged according to its potential syntactic roles in a sentence, not according to its parts of speech. Although, they are pretty much the same, there are still some differences. For example, a noun in a noun phrase can be used as a main noun or an adjective, and their responsibilities in the noun phrase are also different. The main noun in a noun phrase need to propose a conceptual frame for the noun phrase.

4.2.2.2. Features

The feature information of a word in the Lexicon is basically applied by other words to decide their meaning in a sentence. Particularly, the information is used by the feature functions specified in the conditions of semantic representation structures of words. The information includes the generic concepts and/or modeling concepts associated with a word. For example, the word "move" has the event and transfer, two features.

4.2.2.3. Modifiers

The modifiers are to indicate what kinds of concepts can be used to modify a word, and the information is used by the Syntactic Analyzer to decide the proper syntactic structure of a sentence. In the Parsing process, the most difficult task is usually to determine the receivers of modifiers in sentences when several alternatives are available, and the information of modifiers is applied to resolve such confusions. For instance, in sentence 4-2, the modifier "for 10 minutes" is assigned to the verb "process" is based on the modifiers information stored under the word "process."

4.2.3. The Inheritance Property of the Lexicon

For a natural language processing system, the problem that a large number of words and their associated information need to be kept is generally expected. Therefore, how to reduce the size of a dictionary in order to fit it into a limited volume of computer memory that is essential to the design of a language system. Generally, there are several alternatives available to achieve such an objective. The most effective one is to restrict the applications of the system to a small number of similar domains. This approach can not only dramatically reduce the number of the words required, but also can limit the number of possible interpretations of words in a dictionary. The other one is to create customized lexicons according to individual users. This is because people tend to use only a certain

number of words in their applications. Another one is to impose restrictions on users to allow them only to use certain number of words.

Besides trying to reduce the number of words in a lexicon, there is another approach which can also reduce the size of it, and that is to reduce the volume of information stored under each word. Basically, the approach is to allow different words to share their common information. As it is discussed previously, usually different words can be used to describe the same concept. Thus, they should have the same semantic representation, and most of the time they also have the same grammatical properties. Furthermore, according to the case grammar, verbs can be classified by sentence types or case frames, i.e., semantic representations. For example, the verbs "transfer," "move," "transport," "send," "deliver," "ship," and "drag" have the same grammatical properties and they share the same semantic representation.

The inheritance property of the Lexicon permits a word to share information with the other words. The information can be syntactic and semantic information, or semantic information only. Furthermore, although a word's information can be obtained through the other words, the information retrieval device always gives the first priority to the information stored under the current word. Only when the current word fail to provide the information requested, the device then tries to locate the information from the other words. The linkage between the word which possesses information and the word which inherits the information is set up by storing the possession word into the root slot or the semantic representation slot of the inheritance word in the Lexicon. The control flow of the dictionary search device is presented in the Figure 10, and an example of the Lexicon is given in Table 6.

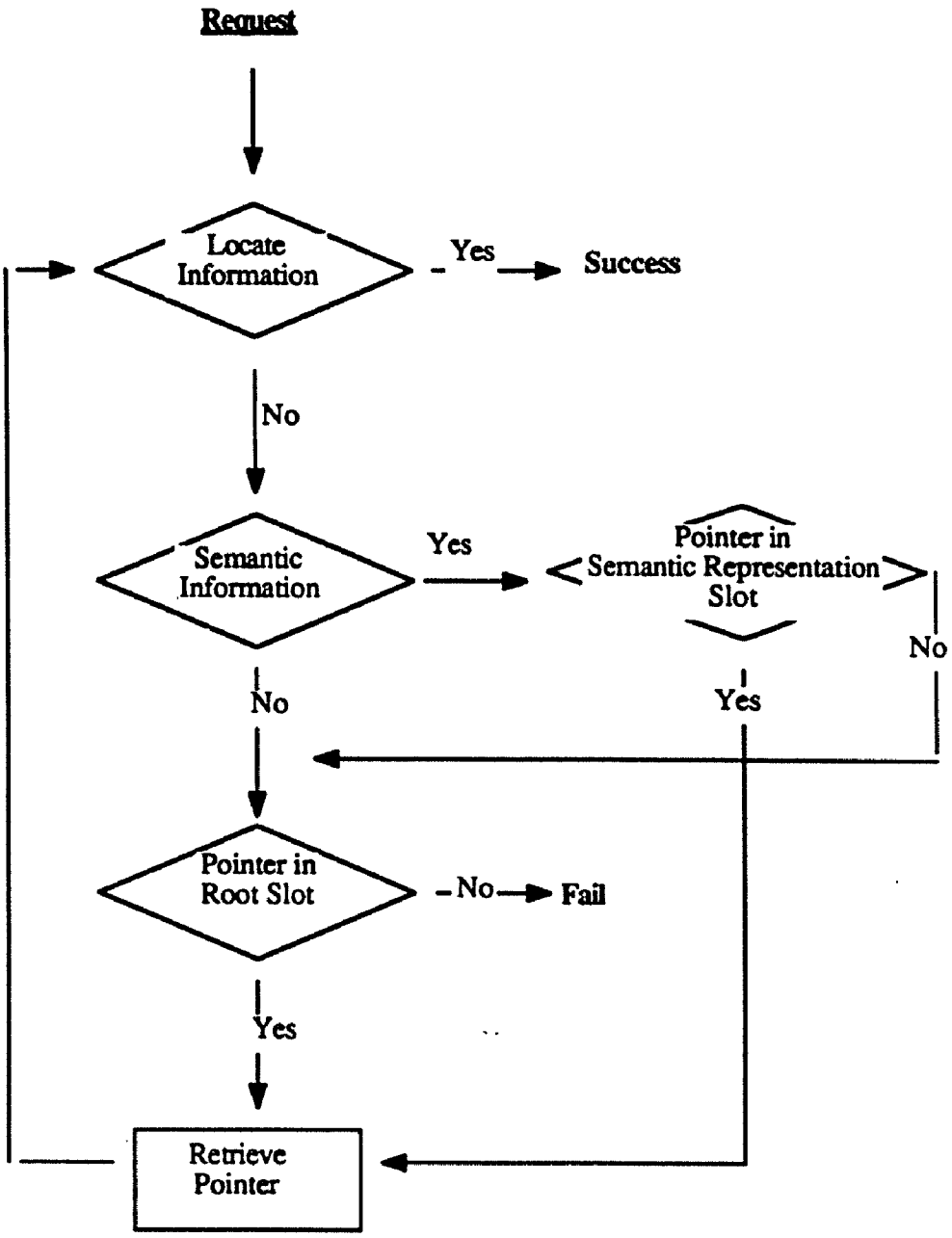


Figure 10. The Control Flow of the Dictionary Search Process

Table 6. An Example of the Lexicon

```
<----- Part of Speech : Determiner ----->

(word the (grammar (det ) ) (art def ) (number (s pl ) ) )]

(word a (grammar (det ) ) (art indef ) (number (s ) ) )]

<----- Part of Speech : Noun ----->

(word minutes (grammar (n ) ) (feature (time.unit ) ) (number (pl ) )
      (n ( ( (feature function quant )
          ! (distribution ? (function nil quant )
                        (number ? (figure nil nbr )
                                  (unit <minute> ) ) ) nil )
          ( (and (nullr quant ) ^nbr )
          ! (number ? (figure nil nbr )
                    (unit <minute> ) ) nil ) ) ) )]

(word item (grammar (n ) ) (number (s ) ) (feature (part unit ) )
      (n ( (! (part (description ? (name (name <default> ) )
                                   ? (type ) )
                                   ? (condition ) ) nil ) ) )]

(word part (root item ) )]

(word work (grammar (n v vi ) ) (number (s ) )
      (phrases ( (work center ) (work station ) ) ) )]

(word station (grammar (n ) ) (number (s ) ) (feature (station ) )
      (n ( (! (station (description ? (type )
                                      ? (name ) )
                  ? (state delay ) ) nil ) ) ) )]

<----- Part of Speech : Adjective ----->

(word busy (grammar (adj ) ) (feature (state ) )
      (adj ( ( ( (state <busy> ) ) ) ) ) )]
```

Table 6. Continued.

```
<----- Part of Speech : Adverb ----->

(word exponentially (grammar (adv ) ) (feature (function ) )
      (adv ( ( ( (function <exponential> ) ) ) ) )]

<----- Part of Speech : Proper Noun ----->

(word DIP (grammar (n ) )
      (adj ( ( ( (name DIP ) ) ) ) )]

(word Poisson (grammar (n ) ) (feature (function ) )
      (adj ( ( ( (function <poisson> ) ) ) ) )]

<----- Part of Speech : Preposition ----->

(word from (grammar (prep ) )
      (prep ( (! (from ? (station nil np )
                        ? (number nil np ) ) nil ) ) )]

(word to   (grammar (prep ) )
      (prep ( (! (to ? (station nil np )
                      ? (number nil np ) ) nil ) ) )]

<----- Part of Speech : Verb ----->

(word transfer (grammar (n v vt past pp ) ) (number (pl ) )
          (feature (event transfer ) )
          (adj ( ( ( (event transfer ) ) ) ) )
          (v ( (! (event (type transfer )
                        ? (part nil obj )
                        ? (from nil from )
                        ? (to nil to )
                        ? (condition ) ) nil ) ) )]

(word transferred (grammar (v vt past pp ) ) (root transfer )]

(word move (grammar (v vt ) ) (root transfer )]
```

4.3. Syntactic Processor

The major role of the Syntactic Processor is to support the Semantic Processor with the information of the grammatical constructions of sentences. The Syntactic Processor consists of two components, the PreProcessor and the Syntactic Analyzer. The PreProcessor, the frontier of the system, is to confirm words in a sentence with the Lexicon, and the Syntactic Analyzer based on the augmented transition networks is to breaks up a sentence into grammatical categories.

4.3.1. PreProcessor : The Frontier of the System.

The functions of the PreProcessor is two fold:

1. To confirm words with the Lexicon : The function is to secure the processor from encountering unrecognizable words in the processes.
2. To Combine words into phrase : Sometimes, a single word in a sentence is meaningful, only when it is combined with the surrounding words into a phrase. The PreProcessor uses the information in the phrases category of the Lexicon to check whether the current word and the following words can be combined into a phrase or not. If it can, these words is replaced by a phrase word. A phrase word is a word that is formed by connect words in a phrase with hyphens. For example, the phrase "as well as" is replaced by the word "as-well-as."

4.3.2. Syntactic Analyzer : An Augmented Transition Network (ATN) Based Parser

A parser is a formalism which can determine whether a sentence conforms to the constraints of the syntax of a language, and also build a representation of the syntactic structure. The Syntactic Analyzer in the NLME is modeled after the augmented transition

networks formalism which has been discussed in the chapter two. The formalism is to describe grammar as transition networks and each network consists of a set of nodes (or states) and arcs between the nodes. The arcs are labeled by word classes or syntactic constructs.

Basically, the ATN parser operates top down, making implicit expectations of what will be found next in the sentence, based on what has been found. The approach hypothesizes a high level structure (e.g., a phrase), then attempts to match (recursively) lower level structures (e.g., words) to it. Each arc in a network represents an expectation. If an arc is followed and later proves not to head to a successful parse of the sentence, the ATN parser backs up to the last choice point and tries an alternative choice.

Furthermore, the way to define grammatical rules as augmented transition networks is through a set of predefined arc types and a set of registers. The registers are used as temporary storages to store the syntactic information collected during the parsing processes, and traditionally, they are named after lexical categories such as subject, object, noun, adjective,... . Moreover, an arc in a network basically is used to test a word's grammatical property against the condition specified in the arc. Generally, when the arc is taken, the parser is automatically advanced to the next word in a sentence and the next arc in a network. The arcs used in the Syntactic Analyzer generally can be categorized into the following six types:

1. CAT arc.

   Format :

   (CAT [category or categories] [test] [destination node] [action]s)

   The CAT arc is to test the current word to see if it can be a member of the lexical category [category or categories]. Examples of lexical categories are noun, pronoun, verb, adjective, adverb,etc. Furthermore, the [test], [destination node],

and [action]s are three common attributes for each type of arc. The [test] is to specify which extra conditions need to be satisfied so that an arc can be taken. Usually, the [test] is to check the structure of a sentence, which up to now has been generated against certain grammatical requirements. As to the [destination node], it is to indicate the end node of an arc. The last common attribute is a set of actions. Actions are frequently used along with registers to construct portions of the parse syntactic tree or other data structure that reflect the content of input.

2. WRD arc.

Format :

(WRD [word or words] [test] [destination node] [action]s)

The WRD arc is to check whether the current word matches one of the words specified in [word or words].

3. PUSH arc.

Format :

(PUSH [node] [test] [destination node] [registers] [action]s)

The PUSH arc is to execute a subnetwork starting at [node]. Before the control is passed to the subnetwork, all the registers at the current network are saved and made invisible to the subnetwork. Also, some of the registers of the subnetwork are initialized according to the register name and value pairs in the [registers]. Furthermore, if the subnetwork parses successfully and the [test] is satisfied, the [action]s are taken and the control is transferred to the [destination node].

5. JUMP arc.

Format :

(JUMP [node] [test] [action]s)

The JUMP arc causes the ATN parser to advance to the [node]. However, unlike

the other arcs, the JUMP arc does not causes the processing word to be advanced to the next word in the sentence.

6. TEST arc.

   Format :

   (TEST [test] [destination node] [action]s)

   The TEST arc is usually used to perform a designated test against the current word, which cannot be performed by the other types of arc.

7. ROOT arc.

   Format :

   (ROOT [root or roots] [test] [destination node] [actions])

   The ROOT arc is to compare the root form of the current word with words specified in the [root or roots]. Generally, the ROOT arc is used to identify the present tense of verbs in sentences. Especially, when the parser needs to decide the aspects and voices of sentences, the root arc is used to check whether the auxiliary verbs of the sentences are either the verb "Be" or "Have."

In addition to the different arc types used to construct networks, system registers can be used in the [action] field of each arc type to store the information collected. These registers are classified according to the constructs of sentences and they are described as follows:

1. Sentence level :
   subj (subject), v (verb), adv (adverb), aspect, obj (object), indobj (indirect object), adjunct, conj (conjunction), ss (subordinate sentence), modR (modifier).

2. Noun phrase level :
   det (determiner), quant (quantifier), n (main noun), nbr (number), adj (adjective), modR (modifier).

3. Preposition phrase :
   prep (preposition), np (noun phrase).

Moreover, a set of function is available to manipulate registers, and they are setr, getr, addr, and nullr.

1. Function setr.

   Format : (setr [register] [value])

   The getr is to reset the register [register] with the value [value]. Frequently, the system variable "*" which represents the word currently under processing, is used in the field of [value].

2. Function getr.

   Format : (getr [register])

   The getr is to retrieve a value from the register [register]. The short-hand form of the function is ^[register].

3. Function addr.

   Format : (addr [register] [value])

   The addr is to add a value to the register [register]. The difference between the setr and the addr is that the setr will reset the register before the value is stored.

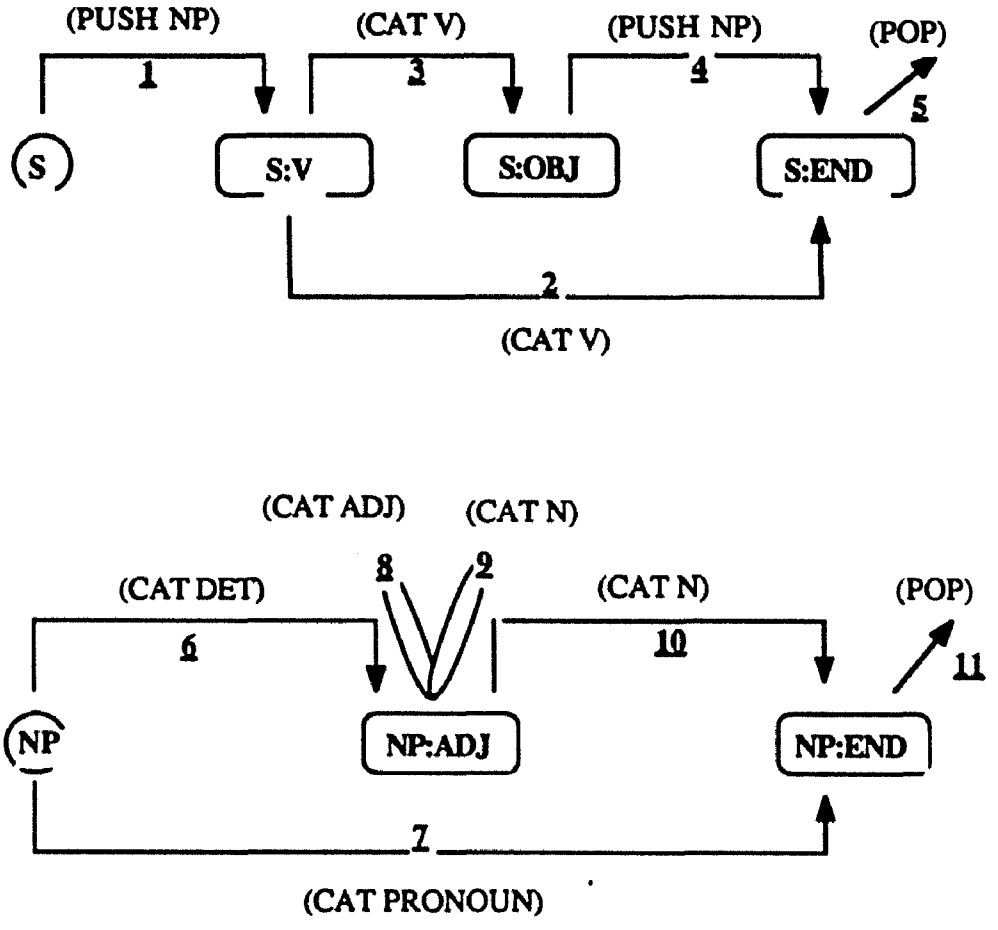
4. Function nullr.

   Format : (nullr [register])

   The nullr is to examine whether the register [register] is vacant or not.

Figure 11 presents a simple version of augment transition networks and the definition corresponding to each arc. Now, let's consider to process the following sentence with the networks.

The PCB leaves the TDK station. (4.5)



Arc	Definition
1	(push np t s:v nil (setr subj *))
2	(cat v (catf vi) s:end (setr v *))
3	(cat v (catf vt) s:obj (setr v *))
4	(push np t s:end nil (setr obj *))
5	(pop (form.cd 's '(subj v obj) (null *))
6	(cat det t np:adj (setr det *))
7	(cat pronoun t np:adj (setr n *))
8	(cat adj t np:adj (addr adj *))
9	(cat n t np:adj (addr adj *))
10	(cat n t np:end (setr n *))
11	(pop (form.cd 'ph '(det adj n)) t)

Figure 11 An Example of Augmented Transition Networks with Their Definitions

The process begins with the starting node S and the PUSH arc will call the noun phrase subnet, starting at the NP node. There are two alternatives in front of the NP node. One is arc 6 which looks for a determiner in the sentence, and the other one is the arc 7 which looks for a pronoun. However, only the arc 6 is satisfied by the first word "the," and the test of the arc is examined. Because the condition specified here is "t" which stands for always true, the action is taken to store the current word "the" which is kept in the system variable "*", into the register det.

Then, the parser moves the process from the current word "the" to the next word, "pcb" in the sentence, and moves the process node from the current node, NP to the next node, NP:ADJ in the networks. The NP:ADJ node is to decide whether the current processing word is an adjective (arc 8) or a noun which used as an adjective (arc 9), or the main noun in a noun phrase (arc 10). The arcs are tested following a accenting sequence. Arc 8 is fail and arc 9 is taken. The whole process is then moved to the next word "leaves" and node NP:ADJ again. However, this time none of the arcs coming out of node NP:ADJ are satisfied. So, the process is backtracked to the previous stage through arc 9 to node NP:ADJ. This time arc 9 is successfully taken, and the process is moved to node NP:END. The pop arc on arc 11 is to construct the semantic representation of the noun phrase "the pcb." The process of constructing semantic representation will be discussed in the next section.

Next, the process is returned to the sentence network and moved to node S:V. The catf function on arc 2 and arc 3 is to examine the grammatical property of the current processing word, and vi and vt are stand for intransitive verb and transitive verb. Because the verb "leave" can be both a transitive verb and a intransitive verb. However, arc 2 is tested first; therefore, it is taken and the process is moved to node S:END. The word now under processing is "the." However, the test (null *) on the pop arc is to indicate that the

arc can only be taken when the process reaches the end of the sentence. The condition causes the fail of arc 8, and the process is backtracked to node S:V again. The next step of the process is to try arc 3. Eventually the process will reach the last node and successfully parse the sentence.

4.4. Semantic Processor

The goal of the Language Processor is to extract the meanings of sentences from the surface structures of them. The nucleus of the processor, the Semantic Processor, is mainly designed to accomplish this job with the help of the Syntactic Processor and the Lexicon. The functions performed by the Semantic Processor include :

1. Forming semantic representations for sentences using the Semantic Synthesizer,
2. Resolving referential ambiguities using the Short Term Memory, and
3. Resolving Semantic ambiguities using the contextual rules and the contextual memory.

The strategies used to achieve these functions are described in the following sections.

4.4.1. Semantic Synthesizer

Unlike the augmented transition networks' top down approach, the Semantic Synthesizer takes the bottom up approach to generate the semantic representation for a sentence. The Syntactic Analyzer operates on a sentence from the sentence level down to the phrase level and then to the lowest word level. Several kinds of information can be acquired form the analysis. For a sentence, we know its structure. For a phrase, we know its syntactic role in a sentence and its constructions. For a word, we know its role in a phrase. Then, according to this information plus the semantic information from the Lexicon, the Semantic Synthesizer merges the semantic representations of words into

phrases and merges phrases into a sentence. A framework of transforming a sentence into its semantic representation is given in the Figure 12.

In the Language Processor, the processes of the syntactic analysis and the semantic analysis are not executed sequentially, but overlap each other. There are two alternatives to arrange the order of the two processes. The first one is to delay the semantic process until a complete syntactic structure tree of a sentence is generated. Then, according to the structure tree, the semantic process is proceeded. However, the second approach used in the NLME system is to allow these two processes to be mixed with each other. Whenever a construct of a sentence is determined by the syntactic analysis, the semantic analysis is initialed to construct the representation of the construct. In our case, phrase is the basic unit which can trigger the Semantic Synthesizer.

The advantage of the second method over the first one is to provide the syntactic analysis with some semantic information. Without semantic knowledge, a syntactic parser can only generate a grammatical correct sentence structure, but it doesn't guarantee that the interpretation of the sentence from the structure is meaningful. And this issue has been explored in the section of world knowledge. Furthermore, the mixed processes are also more close to humans' behavior which proceed the semantic process and the syntactic process in parallel.

As it was discussed previously, the strategy of the synthesizer is to construct semantic representations of sentences from bottom up. The process started from assembling the semantic representations of words into a phrase and representations of phrases into a sentence. Most of the operational principles of the Semantic Synthesizer have been mentioned in the semantic representation structure of last section. Some of the principles are further summarized as follows:

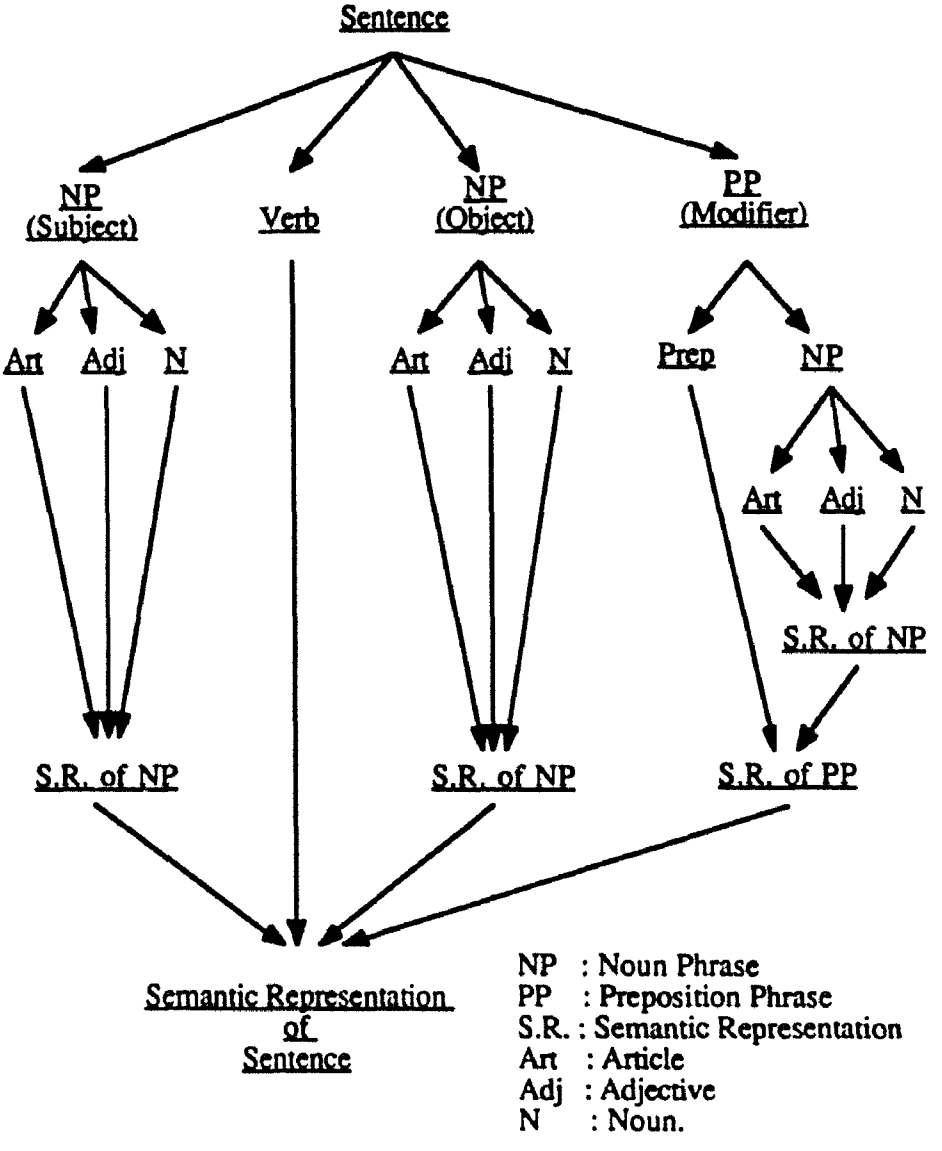


Figure 12. A Framework of Transforming a Sentence into Its Semantic Representation.

1. For each word in grammatical constructions of sentences, its possible semantic representations are retrieved from the Lexicon, and the conditions corresponding to the representations are evaluated in order to decide the proper word sense according to context.
2. Basically, words in lower level grammatical constructions need to propose semantic structures for higher level constructions.
3. The proposed semantic structures are conceptual frames and they are instantiated with semantic representations in lower level constructions.
4. The instantiating processes are based on the information specify in each slot of conceptual frames. The information includes what kind of concept is required and under which syntactic categories the concept can be found.

4.4.2. Short Term Memory

The function of the Short Term Memory is to resolve the referential ambiguities which originate from pronouns and determiners in noun phrases. Basically, the Short Term Memory is a storage used to keep track of noun phrases in input sentences. The circumstances to use the memory are whenever pronouns or nouns leaded by determiners are encountered. Then, the Syntactic Analyzer will search the Short Term Memory to locate a proper noun phrase to replace the pronoun or the noun.

The structure of the Short Term Memory and the search strategy are based on semantics rather than syntax. Each noun phrase, the main noun, plus its modifiers are stored in the memory by its semantic representation, not by its surface structure. The representations stored in the memory are also categorized according to their main concepts, semantic markers corresponding to them, and they are updated when a noun phrase representing the same concept as one in the Short Term Memory is received.

The advantage of such a structure is to permit the search process to locate the referred nouns from the Short Term memory based on the corresponding pronouns' semantic roles in sentences Moreover, the information about a pronoun's semantic role in a sentence can usually be acquired from the conceptual frame of the sentence, which is signified by its verb. For example, consider the following sentence:

It is processed for 10 minutes. (4.6)

Because in our model, only parts can be processed in a station, we can conclude that the pronoun "it" in sentence 4.6 is referred to a noun with a work piece concept, and it is used to locate the proper noun phrase in the Short Term Memory.

4.4.3. Context Applier : Contextual Rules and Contextual Memory

Context is essential to the understanding of natural languages in several areas. First, in a sentence, context can be applied to decide the meaning of a word in a sentence from a number of possible interpretations. Context also contributes in resolving the referential ambiguity which is caused by pronoun and determiner. Besides these two areas, there is another area that context can be very helpful, and that is when certain information is left out from a sentence, and it can only be understood by looking through other sentences.

Considering the following sentence:

The transfer time is 10 minutes. (4.7)

What is transferred, from where, and to where ? The sentence itself doses not give any answers. However, if we examine the previous sentences and we find the sentence :

Then, the PCB is transferred to the TDK station, (4.8)

and further we find the sentence :

The Burg station processes the PCB for 10 minutes. (4.9)

Then, we can conclude that it is the PCB to be transferred, the starting location is the Burg station, and the destination is the TDK station. Furthermore, the sentence 4.7 is to give the transfer time of the PCB from the Burg station to the TDK station. On the other hand, if we do not use the context, we are unable to make such inference.

Basically, the Context Applier is designed to deal with the above situation. The objective of the Context Applier is to disambiguate a sentence with context. It is to collect additional information for a semantic ambiguous sentence in order to make it self-contained and well defined. In our applications, a semantic ambiguous sentence usually does not clearly define which object the sentence's information should be associated with. Here, an object can be an event, an activity, or an entity in a model.

For instance, sentence 4.7 is an uncertain sentence. Because the sentence is used to describe the duration of a transportation activity and the activity is not defined in the sentence. Moreover, sentence 4.8 is still an ambiguous sentence. Even though the sentence does specify the part participating the transfer activity and the destination, there is still more than one activity which meets the description of sentence 4.8, but their starting location are different. The activity in sentence 4.8 is still undeterminable.

How to define objects in sentences ? Basically, objects are defined according to their description. Entities such as parts, stations, and material handling equipment, can be described by their names, functions, or types. For example, cast part and PCB represent part entities, and the first one is referred by its type and the second one is referred by its name. Furthermore, drilling station and TDK station represent station entities, and.the former one is called by its function and the later one is called by its name. As to events and

activities, they are defined differently according to their types. An operation activity can be determined by the part entity and the station entity that involve the activity. Moreover, an arrival event is decided by the arrival part entity and its destination, and an transfer activity is decided by the part entity transferred and the staring and ending stations. Thus, only when the objects referred to in sentences are not ambiguous and well defined, the sentences are self-contained.

The strategy adopted by the Context Applier to disambiguate a sentence is to apply a set of contextual rules and they are set up using contextual knowledge. The contextual rules are used to examine sentences, whether they are ambiguous or not; and if they are, these rules can further be used to pick up the missing information of the sentences from their proceeding sentences. Here, we assume that when a sentence is ambiguous, it can be disambiguated by its prior sentences. Moreover, the semantic representation of each input sentence is stored into the contextual memory which can later be used by the Context Applier. Currently, the contextual rules are mainly designed to handle activity related sentences.

Basically, the contextual rules are drawn from the contextual knowledge about simulation modeling concepts. A simulation model consists of a sequence of activities or events. The happening of an activity or an event will change the state of a system, and the new state will further cause other activities or events to take place. In order words, if the Language Processor currently receives a sentence which is to describe an activity, the activity should be directly or indirectly triggered by the other activities described in previous sentences. Here, the "indirectly" means that the NLME system does not expect system descriptions to be described following a strict events/activities occurring sequence. A well defined and self-contained sentence can always be understood, no matter where it is placed in a system description. However, the processor does expect that ambiguous

sentences can be made certain by their prior sentences.

Consider the following contextual rule :

IF (Part A is transferred from station X to station Y) (4.10)
OR
(Part A arrives at station Y) (4.11)
THEN (Part A is processed at station Y). (4.12)

By comparing the initiating activities, sentence 4.10 and 4.11, and the initiated activity, sentence 4.12, several kinds of information can be revealed from their relations and common attributes. For example, if a sentence is used to describe an operation activity, but it fails to provide the part or the station participating the activity. They can still be identified by applying the example rule to the previous input sentences.

Generally, The structure of context rules follows the format of production rules. The premise of a rule are composed by a number of conceptual frames which represent a set of activities, and the conclusion of a rule is the conceptual frame of an activity that can be fired by one of the activities in the premise. Therefore, the conceptual frames in the premise denote a set of disjointed conditions to initiate the activity in the conclusion. Moreover, in order to make the access of rules more convenient, contextual rules are indexed according to the types of conceptual frame in the conclusion. A set of contextual rules, along with their English translations are presented is presented in Table 7.

Furthermore, the inference process for the Context Applier to use rules can be described as backward reasoning and breadth first search. The backward reasoning is to confirm a conclusion activity, input sentence, by proving the existence of one of the condition activities, prior input sentences, in the contextual memory. The breadth first search is only when all the condition activities are tested against a semantic representation of a sentence in the contextual memory, the process then moves to the next representation

in the Working Memory. However, the opposite approach, depth first search, is to use a condition activity to test against all the representations in the contextual memory in one time. The control flow of the Context Applier is given in Figure 13.

Table 7. An Example of Contextual Rules

[Note : a word started with a question mark is represented as a variable]

```
(<Contextual Rules> transfer

        (if (event (type transfer)
                   (part ?part)
                   (from (station ?station0))
                   (to   (station ?station1)))

            (event (type operation)
                   (part ?part)
                   (station ?station1))

            (event (type arrival)
                   (part ?part)
                   (to (station ?station1)))
        then

            (event (type transfer)
                   (part ?part)
                   (from (station ?station1))
                   (to   (station ?station2)))
```

[If one of the previous sentences mentions that part ?part is transferred from station ?station0 to station ?station1, or part ?part is processed at station ?station1, or part ?part arrives at station ?station1, we can conclude that part ?part is transferred from station ?station1 to station ?station2]

Table 7. Continued

(<Contextual Rules> Operation

(if (event (type transfer)
(part ?part)
(from (station ?station0))
(to (station ?station1)))

(event (type arrival)
(part ?part)
(to (station ?station1)))

then

(event (type operation)
(part ?part)
(station ?station1))

[If one of the previous sentences mentions that part ?part is transferred from station ?station0 to station ?station1, or part ?part arrives at station ?station1, we can conclude that part ?part is processed at station ?station1.]

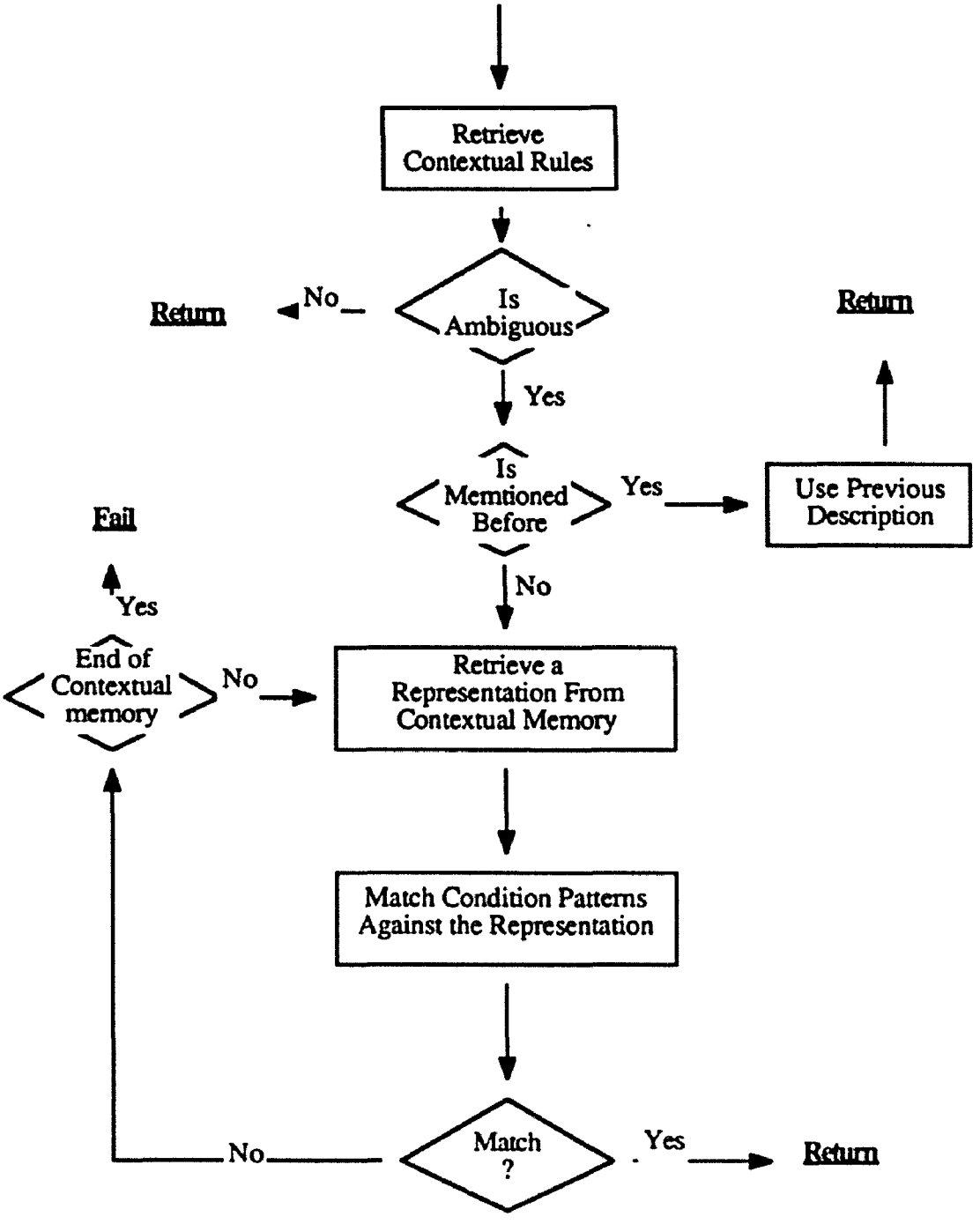


Figure 13. The Control Flow of the Context Applier

The process is started from retrieving the appropriate rule according to the input semantic representation structure. Then, the conceptual frame in the conclusion of the rule is matched against the input representation to check whether it is ambiguous or not. If it is, the conceptual frame is further used to confirm whether the same representation has appeared before or not. In other words, this process is to find out that if the object described by the input sentence has been described by previous sentences or not. If an equivalent representation can be found, it is used to disambiguate the input sentence. However, if the process fails, the conceptual frames in the condition of the rule are applied to locate the appropriate representations in the contextual memory to disambiguate the input sentence.

The choice of the backward reasoning and breadth first search is based on the formulation of contextual rules and the nature of the problem. Basically, the reasoning approach of the Context Applier is a confirmation process. First, the applier forms a hypothesis activity based on an ambiguous sentence. Then, the hypothesis is proved by proving the truth (existence) of its precondition activities. This approach is unlike the forward reasoning strategy which uses an input activity to predict which activities will happen next. Moreover, the backward reasoning process is supported by the breadth first search. The reason to adopt the search method is so that more efficiency can be gained by it than by the others. Generally, people omit a piece of information from a sentence when the information has been expressed in a previous sentence, and they just do not want to state it repeatedly. Therefore, a shadow search is just what we need to locate the information. Under such circumstances, the breadth first search provides us more efficiency than the depth first search.

CHAPTER V

THE STRUCTURE AND FUNCTIONS OF THE CENTRAL UNIT

The functions of the Central Unit are to preserve the semantic representations generated from the Language Processor and to use them to construct simulation programs. The structure of the Central Unit is given in Figure 14 and the control flow of the unit is given in Figure 15. Basically, the tactic used to save the semantic representations of input sentences is to store them into the Working Memory. However, the Working Memory is not just a flat storage. It is formatted as a hierarchical structure. Therefore, the storage keeper, the Semantic Representation Interpreter, is used to examine incoming representations and decide how and where to store them according to the organization of Working Memory.

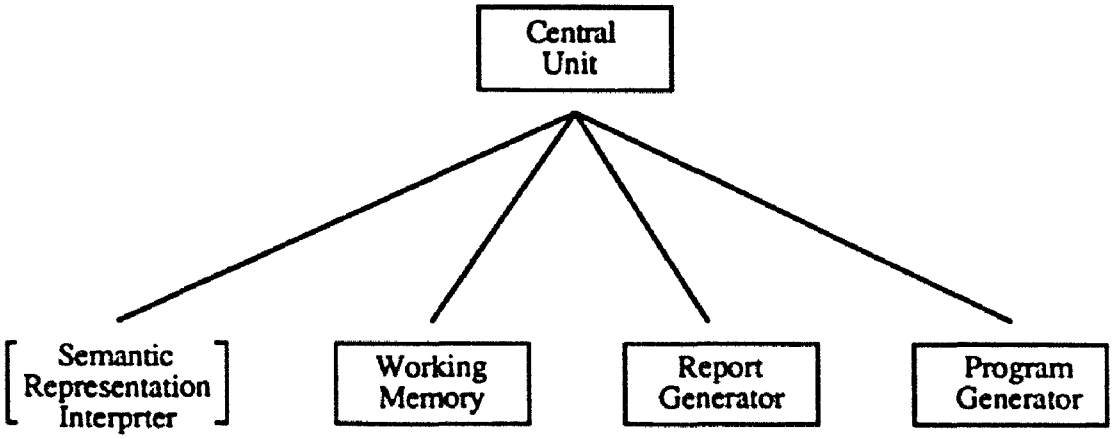


Figure 14. The Structure of the Central Unit.

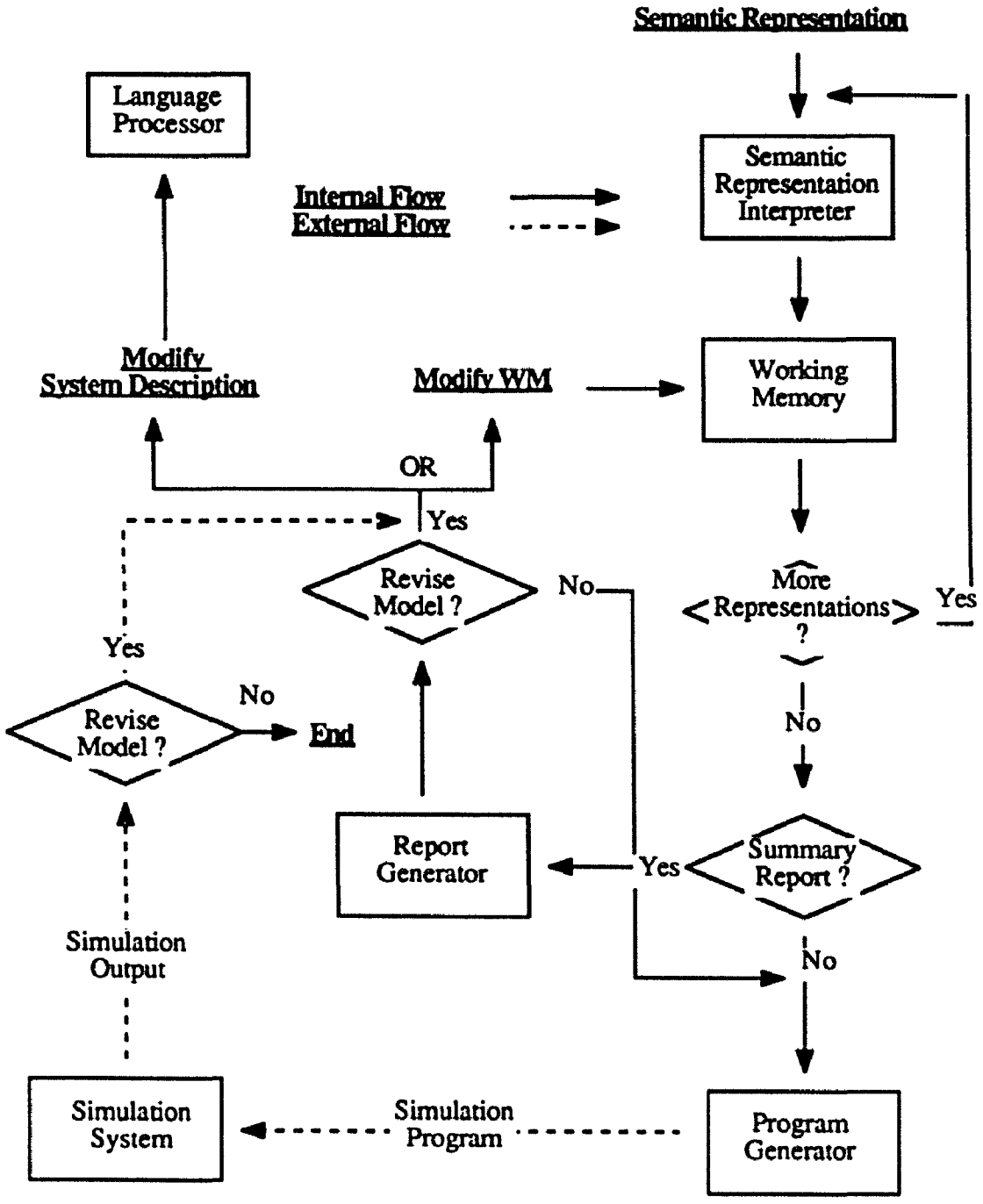


Figure 15. The Control Flow of the Central Unit

After all the representations are processed, a summary of the current contents of the Working Memory can be reported by the Report Generator. The summary report is mainly used to confirm the correctness of the model built by the NLME. If modifications are needed, system analysts can either revise the original system descriptions or the contents of the Working Memory. Finally, the Program Generator is called to construct the simulation programs. Based on the outputs of the programs, the system analysts can decide whether further modifications of the models are necessary, or if more experiments are required.

5.1 Working Memory (WM) : The Heart of the Central Unit.

The Function of the Working Memory is to provide a structure way to manage the information collected from a system description. In a system description, each statement stands for an individual piece of information. Usually, the relationships which exist among different pieces of information are not clearly indicated. Also, the information is generally not presented in groups according to their functions and usages. In order to let the information be effectively used by the application programs in the Central Unit, we need to organize and integrate all the informations. Moreover, the relationships among different information need to be sorted out, and information should be categorized according to their objectives.

The Working Memory in the Central Unit is not just served as a storage. It also offers a systematic way to arrange the data in the memory. And, the objectives are accomplished through a frame system. Moreover, the frame system of the Working Memory and the manufacturing system it presents are hierarchical in nature. The structure is divided into three levels: the root level, the configuration level, and the activity level. An example of the Working Memory structure is presented in Figure 16.

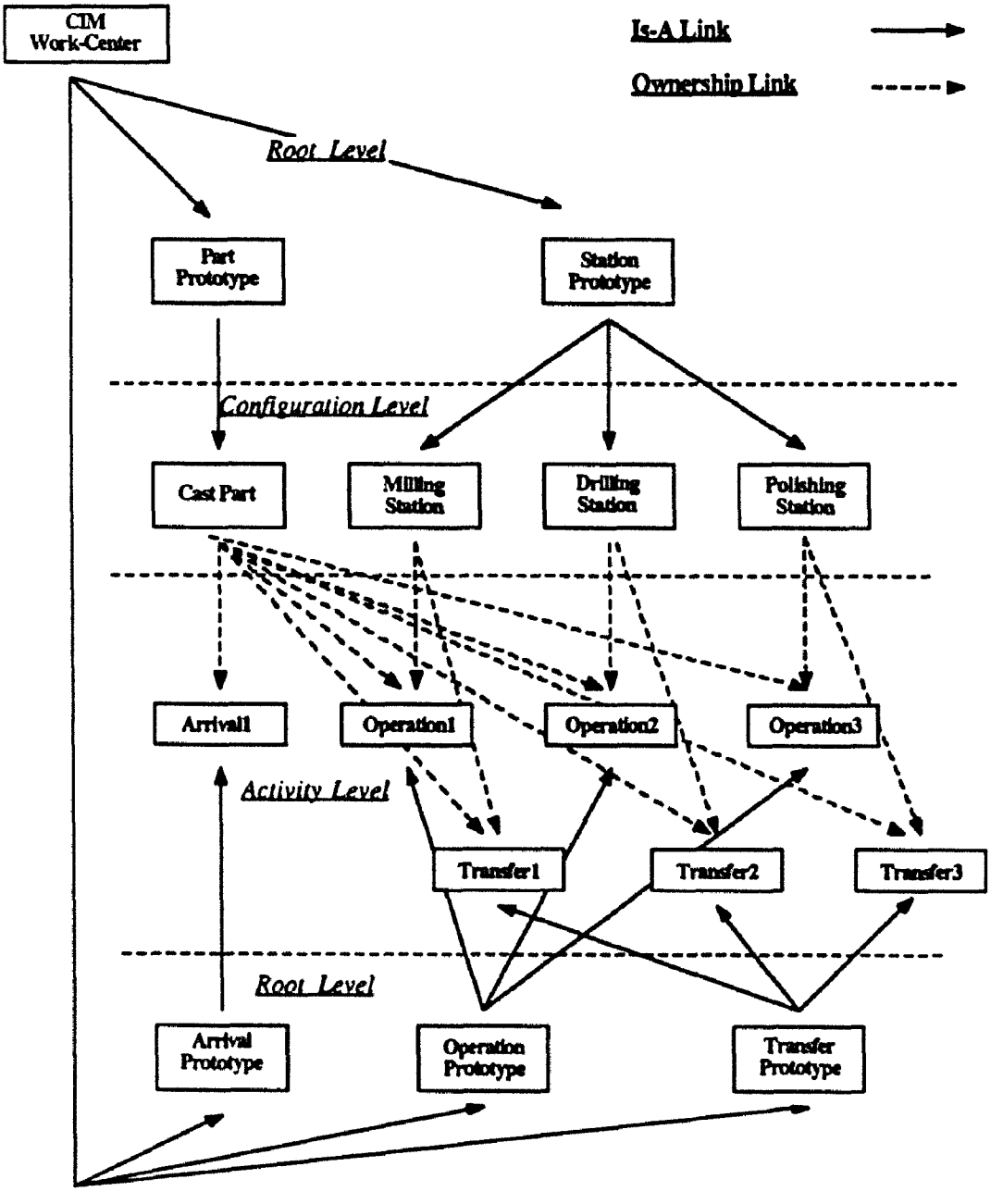


Figure 16. The Working Memory Structure Corresponding to the System Description in Table 3

The root level stands for the initial structure of the memory, which consists of predefined prototypes of frames used to construct a system. The frames include different entity types, such as part, station, and material handling equipment, and different activity types, such as arrival event, operation activity, and transfer activity. Generally, the root level provides a general view of the common characteristics among manufacturing systems.

Unlike the root level, the configuration level and activity level are system specific. They give a more detailed view of a particular system, and they are created based on the information absorbed from a system description. Moreover, the configuration level is used to describe the facilities and the parts in a system. Each frame in this level specifically corresponds to a facility or a part type in the system, and the frame can either be a station or a kind of material handling equipment or a part type. As opposed to the physical objects, the activity level consists of conceptual objects, events, and activities, and represents the behavior of the physical objects in a system. Each frame in this level stands for an activity or an event occurring in the system, and the frame can be either an arrival event, a transfer activity, or an operation activity.

Except the initial structure, the root level of the Working Memory, the other two levels are generated according to the information gathering from input statements. Furthermore, each frame in one level is related to some frames in the other two levels through two links; part-of and consist-of slots, and they are the common attributes among all types of frame in the memory. These slots in a frame are used to store names of some other frames and they can easily be accessed by their names from the current frame. The interpretations of links between frames are classified into two classes. One is the IS-A link through the part-of slot, and the other one is the ownership link through the consist-of slot. Generally, the links reflect how the information in the memory is organized and how the relation between two frames can be characterized.

The IS-A links are interpreted as a set membership, and they are laid between frames of the root level and frames of the configuration level or activity level. Basically, frames in these two levels -- the configuration and activity levels -- are grouped into sets according to their natures. In addition, each member of a set is connected to the set prototype in the root level. For instance, in Figure 16, the milling station, the drilling station, and the polishing station frames in the configuration level all belong to the same type of facility, station, and they are connected to the station prototype frame at the root level. So, as the operation1, the operation2, and the operation3 frames, they are connected to the operation prototype frame.

The advantages of such a structure are two-fold, and they are from the inheritance property of frame systems. First, the taxonomic hierarchical structure provides a method to categorize frames into classes or sets. For the same class of objects (frames), they have various common attributes and behaviors, and they can be defined in one frame, the class prototype or set prototype. Then, those attributes and behaviors can be shared by all the members in that class. In this way, these attributes need not be redefined individually in each member of the class, and the consistency of the attributes can easily be maintained among same type of frames. Secondly, the structure also provides a degree of "taxonomic" inference based on the system's representation of prototype and instance hierarchies. In other words, the taxonomic inference property allows the system to identify the nature of incoming objects by matching their attributes against those attributes of frames in the Working Memory. For objects belonging to the same type, they should have common attributes among them. The match process is started from the root level down to the lower level.

Furthermore, the second kind of link is the ownership link used to describe the relations between objects in the configuration and activity levels. Generally, it characterizes

the behaviors of physical objects in terms of activities. For instance, a part in the configuration level is connected to an arrival event , and several operation and transfer activities through the part's consist-of slot. Basically, these events and activities are used to describe the process routing of the particular type of part in a manufacturing system . Likewise, a station frame is associated with a set of operation activities performed in the station.

Several benefits can be gained from the ownership links. First of all, it is easy to identify what activities correspond to a particular part type or facility. Second, the relationships among physical objects can be recognized by the co-ownership of an activity. For example, if a particular part type and a station are both connected to the same operation activity frame, we can then conclude that the part is processed at the station. Finally, from the opposite direction of the link -- from activities to objects through the part-of slots -- the participant of activities can be easily located. From the simulation modeling point of view, the entities engaging in an activity can be identified.

In addition to the previous discussion, several frame systems' properties have been adopted to enhance the modeling capabilities of the NLME:

1. A default value can be assigned to an attribute (slot) of a frame. Particularly, when default values are defined in a prototype object, they can be shared by its instance objects. This structure can reduce the volume of information which needs to be stored, and keep the data more consistent.
2. The procedure attachment property brings a static storage to life. It can be used to confirm the data type or the range of data or the number of value required for slots of frames, when the slot is instantiated. On the other hand, procedures can also be activated to search for data when a value of an attribute is needed.

3. The Working Memory is presented in a comprehensible way through the hierarchical structure.
4. The class-subclass or prototype-instance hierarchical representation allows models to have different degrees of aggregation and abstraction. Each object in the the structure can be further decomposed into finer classes based on the object's nature. For instance, a station object can have machine tools as subclasses, and an operation activity can further be expanded into a set-up activity and a machine processing activity, two subclasses.

Finally, Table 8 lays out the contents of the initial structure of the Working Memory. Generally, a frame in the memory can have several attributes (slots) associated with it, and each attribute has value, default, if-needed, and max, four different facets. The value and default facets are to define the value and the default value of an attribute. The if-needed facet is to specify procedures used to search for a value for an attribute when its value and default facets are null. Furthermore, the max facet is to indicate the largest number of data that an attribute can have.

5.2 Semantic Representation Interpreter

The Function of the Semantic Representation Interpreter is to decide how semantic representations of sentences are stored into the Working Memory. Since the Working Memory is not just plain storage, each semantic representation generated from the Language Processor cannot be directly put into the memory. The information carried by a representation needs to be analyzed and arranged based on the structure of the Working Memory, and then the information is stored.

The strategy adopted to achieve the previous objective is to apply rules to analyze the representations. Basically, the rules are to pull out information from the representations,

Table 8 The Contents of the Initial Structure of the Working Memory

```
(Frame system
        (object-type        (:value system))]

(Frame part
        (object-type        (:value part))
        (description        (:max 1))
        (arrival            (:if-needed find.arrival (f s)))
        (operations         (:if-needed find.operations (f s)))
        (transfers          (:if-needed find.transfers (f s)))
        (lot-size           (:max 1))
        (enter-time         (:max 1))]

(Frame station
        (object-type        (:value station))
        (description        (:max 1))
        (operations         (:if-needed find.operations (f s)))
        (transfers          (:if-needed find.transfers (f s)))
        (machine-no         (:max 1)
                            (:default 1)).
        (buffer-size        (:max 1)
                            (:default infinitive))]

(Frame operation
        (object-type        (:value operation))
        (part               (:max 1)
                            (:if-needed find.part (f s))
        (station            (:max 1)
                            (:if-needed find.station (f s))
        (operation-time     (:max 1))]

(Frame transfer
        (object-type        (:value transfer))
        (part               (:if-needed find.part (f s))
        (from               (:if-needed find.station(f s))
        (to                 )
        (tf-time            )]

(Frame arrival
        (object-type        (:value arrival))
        (part               (:max 1)
                            (:if-needed find.part (f s))
        (to                 (:max 1))
        (arrival-interval   (:max 1)]
```

and to formulate the store functions of the Working Memory. Then, the functions are executed to put the information into the proper locations in the memory. The store rules employed here generally follow the same format as the contextual rules in the Semantic Processor. Each store rule consists of two parts, a condition part and an action part.

< Store Rule>

(IF
[Conceptual Frame]
THEN
(<store> [Object Identifier]
[Values]))

The condition part of a rule is a conceptual frame which represents a possible structure of semantic representation structures. The action part of a rule is a formulation of function <store>. Two parameters are needed in the function:

1. [Object Identifier]

   The [Object Identifier] is used to decide where to store the information from the semantic representation which is matched by the conceptual frame of the rule's condition. Moreover, the structure of the identifier for physical objects are different from events and activities. The format for a physical object is as the following list:

   (<object> ([Object type] [Description]))

   The list starts with the key word <object> and then a nested list. The nested list is used to indicate the type of the object and its description. Here, the object type can be station, part , etc. As to the identifier structures for events and activities, they are similar to the physical objects. However, the identifier list starts with the key

word <event>, and then a type list to indicate the event or activity type of the corresponding semantic representation. Following the type list, there are several lists to declare which objects possess the ownerships of the activity or event. For example, the following structure is the identifier for an operation activity.

```
(<event>  (type     operation)
          (part     (description (name PCB)))
          (station  (description (name TDK))))
```

2. [values]

The [values] field specifies what data is to be stored into the frame identified by the [Object Identifier] in the function <store>. The format of [values] is a list as the follows.

```
(<value>  (Attribute1  Value1)
          (Attribute2  Value2)
          (Attribute3  Value3)
          .....)
```

The list starts with the key word <value> and is followed by several sub-lists. Each sub-list stands for an attribute and value pair.

Specifically, rules are grouped according to the conceptual frame type in the condition part. Thus, whenever the rules are applied to examine semantic representations of the sentences, only the proper rules need to be retrieved according to the frame type of the representations. This approach can greatly increase the processing efficiency. An example of the store rules is given in Table 9, and in the rules; a word starting with a "?" is recognized as a variable.

Table 9. An Example of Store Rules for a Transfer Frame.

```
(<Store.Rules> transfer

    (if (event   (type transfer)
                 (condition <null>)
                 (duration <null>)
                 (part ?part)
                 (from (station ?station1))
                 (to   (station ?station2)))

   then (<store> (<event>  (type transfer)
                           (part ?part)
                           (station ?station1))
                 (<value>  (to   (to (station ?station2))))))
    (if (event   (type transfer)
                 (condition <null>)
                 (part ?part)
                 (from (station ?station1))
                 (to   (station ?station2))
                 (duration ?duration))

   then (<store> (<event>  (type transfer)
                           (part ?part)
                           (station ?station1))
                 (<value>  (to   (to (station  ?station2)))
                           (tf-time (to (station ?station2)
                           (duration ?duration))))))

    (if (event   (type transfer)
                 (condition (station <null>)
                 (attribute (type queue)
                 (station ?station3)
                 (state ?state)))
                 (part ?part)
                 (from (station ?station1))
                 (to   (station ?station2)))

   then (<store> (<event>  (type transfer)
                           (part ?part)
                           (station ?station1))
                 (<value>   (to   (to (station ?station2)
                           (condition (station ?station3)
                           (queue-state ?state)))))))
```

Table 9. Continued.

```
(if (event   (type transfer)
             (condition (attribute <null>)
             (station (description ?description)
             (state ?state)))
             (part ?part)
             (from (station ?station1))
             (to    (station ?station2)))

then (<store> (<event>  (type transfer)
                        (part ?part)
                        (station ?station1))
              (<value>  (to  (to (station ?station2)
                        (condition
                          (station (description ?description))
                          (station-state ?state)))))))]
```

Unlike the confirmation process of the Context Applier, the Semantic Representation Interpreter uses conceptual frames specified in the conditions of rules and matches them against incoming representations. Then, the interpreter can conclude what kind of actions should be taken. Generally, the rules applying process is similar to the forward reasoning of production systems, and the process is divided into four steps. They are the following:

1. Retrieve the rules according to the frame type of the input semantic representation.
2. For each rule from the previous step, match its conceptual frame -- the condition part -- against the semantic representation, and bind variables in the frame with the corresponding values in the representation. If a match is found, go to the next step. Otherwise, the process has failed.

3. Use the bindings from step 2 to instantiate the variables in the action part of the matched rule, and formulate a <store> function.
4. Execute the <store> function to store information into the Working memory.

5.3 Program Generator : The Program Writer of the System

The function of the Program Generator is to produce different simulation programs based on the contents of the Working Memory. The programs are generated through different language drivers. Here, the language drivers are devices designed according to different modeling concepts of different simulation systems. The language drivers are applied to compile the information stored in the Working Memory into programs of different simulation systems. Basically, the idea of the language drivers is similar to the terminal drivers of graphical packages. Terminal drivers can generate different control codes based on the same control statements, and create the same graphs for different types of terminal.

The design of language drivers is based on the modular modeling concepts. A module, in our application, is used to stand for the behavior of an object type in manufacturing systems. For example, a station module represents a station which processes parts and a transfer module represents material handling equipment which move parts from one station to the other stations. Here, the behavior of objects is characterized in terms of modeling concepts, not according to their physical capabilities. In other words, the behaviors are described by the changes of state variables, participant entities, and the delay of simulation time. Although the functions of two objects are different, they can still be represented by the same kind of module if they can be pictured with the same kind of concept. For instance, the functions of a milling station and a drilling station are different, but they are still simulated by the same type of module. Moreover, a module is made up of statements of a particular simulation language, and they resemble the actions of the

corresponding objects in a system. Several types of modules written using the simulation language SLAM [Prisker and Pegden, 1979] are given in Table 10.

Furthermore, modules can be coupled together in order to form additional module. The hierarchical structure of the Working Memory is divided into three layers. The middle level is composed of the objects of a system, and the lowest level consists of the activities associated with the objects in the upper level. Generally, the strategy of constructing a model using the modular modeling approach is to map objects in the configuration level of the memory to the pre-defined modules according to the object types. And the parameters of the modules can be instantiated using the data from the objects and their activities in the lowest level of the memory. Finally, based on the hierarchical structure of the Working Memory, modules are combined into a simulation program starting from bottom-up.

Actually, the outputs of the Program Generator are not executable programs, but are model templates. The difference between a program and a template is that in a program, the process flow and statements are fully defined and can be executed immediately, but not in a template. Generally, programs consist of statements. Among statements, their relations are to indicated the logic flow of the programs, and within a statement, it is to specify what kinds of actions should be taken and the parameters to be applied by the actions. Like programs, model templates consist of statements which lay out the control flow of the process. However, for each statement in the templates, the values of its parameters have been stripped out and replaced by functions of the Working Memory. The functions are to signify where and how the values can be obtained from the memory. Therefore, when a program is requested, the corresponding model template is evaluated, functions are executed to acquire the proper data, and then a program is generated.

The relationships among system descriptions, the contents of the Working Memory, and model templates and programs are not always one-to-one. Basically, a copy of the

Table 10. An Example of Defining Modules in SLAM

```
; Loading Module :
;         To Generate Parts of a Particular Part Type into the System.
;
;         Attribute Description
;             ATRIB(1) : Part Arrival Time.
;             ATRIB(2) : Part Type Identification Number.
;             ATRIB(3) : Processing Time at a Station.
;
PART1   CREATE, [part arrival time interval],,,1;
;
;             assign part type identification number.
          ASSIGN, ATRIB(2) = [part type #];
;
;             send the part to its first process station.
          ACT,,,[first process station];

          - - - - - - - - - - - - - - - -

; Station Module :
;         To Perform Operations on Different Part Types.
;
;   Identification Block :
;         To Identify the Part Types of Incoming Parts.
;
          ACT/1,,ATRIB(2) .EQ. [part type 1], A1;
          ACT/2,,ATRIB(2) .EQ. [part type 2], A2;

          - - - - - - - - - - - - - - - -
;
;    Assignment Block :
;         To Assign Operation Times to the Incoming Parts According to
;         Its Part Type.
;
A1        ASSIGN, ATRIB(3) = [operation time of part type1];
          ACT,,, Q1;  send the part to be processed
;
A2        ASSIGN, ATRIB(3) = [operation time of part type 2];
          ACT,,,Q1;

          - - - - - - - - - - - - - - - -
```

Table 10. Continued.

```
;   Station Block :
;         To Process the parts.  If the Station Is Busy, Keep the Incoming Parts
;         Waiting in the Buffer
;
;             buffer of the station.  If the buffer is full, the station is blocked.
Q1        QUEUE(1),,[buffer size];BLOCK;
;
;             perform the operation
          ACT([machine no.])/3, ATRIB(3);
;
;
; Transfer Module :
;         To Perform the Material Handling Functions.
;
          GOON,1;
          ACT/4,[transfer time], [part type 1] .AND. [transfer condition],
                [destination station];
          ACT/5,[transfer time], [part type 2] .AND. [transfer condition],
                [destination station];

          - - - - - - - - - - - - - - - - - -
```

Working Memory is created by a system description, and based on the structure of the memory, a model template can be constructed. Then, the template can be fed with data from the Working Memory to generate a program. Moreover, the contents of frames in the memory are subject to change, but not the relationships among the frames. For different contents of the memory, but with the same overall structure, different programs can be generated from the same structure.

The advantage of such a design is to permit system analysts to conduct different experiments for a system under study. An analyst can first use a system description to generate a copy of the Working Memory and to generate the corresponding model template.

Then, the analyst can test out his or her manufacturing system model through varying the values of its parameters such as the number of machine tools in a station, operation times, and buffer size. The changes of parameters can be done through modifying the contents of the corresponding objects or events or activities in the memory. For different parameters, different programs are constructed, and the performances of the manufacturing system can be observed from the outputs of different program runs. Later, the analyst can decide a set of appropriate parameters for his or her system, which meets his or her criterion.

5.4. Outputs of the Central Unit

Four different outputs can be generated from the Central Unit. They are model templates, program files, data files, and summary reports. The first two kinds of outputs, model templates and program files, have been discussed in the previous section. As for the data files and summary reports, both of them represent the contents of the Working Memory.

The data files are used to save the values of frames in the Working Memory, and the files can later be used to restore the contents of the memory. Generally, the language processing is the most time consuming step among all the processes in the NLME, and the process usually accounts for more than 80% of the total processing time. Therefore, the advantage of keeping data files is to reduce the processing time required to regenerate programs or to revise models, when the corresponding contents do not exist in the Working Memory. The data files provide a short cut to restore contents of the memory rather than through the system description. Thus, every time a program is needed, it can be generated from the data file, not from the scratch. Furthermore, when a model needs a minor modification or is used to perform different experiments, the corresponding contents of the memory can be restored. The alteration can be made directly on the memory, not on the input statements. However, in order to maintain the consistency between the system

description and the model, the modification of the model should be limited to its parameters, not the logical structure of it.

Unlike the data file which is used by the NLME, a summary report is presented to system analysts with the information stored in the Working Memory in a structured format. Basically, the report is used by analysts as a tool to verify the model built from a system description with his or her conceptual model of the system under study. The information contained in a report is organized into the following categories:

1. Job Descriptions:

   The job descriptions provide information about all the part types in a model. The information includes part description, arrival time, and lot size.

2. Facility Descriptions :

   Information in this category is related to the facilities described in a model. For instance, for a station, the information includes station description, buffer size, and machine number.

3. Activity Descriptions

   The activity descriptions basically provide four different kinds of information. The first one is the process routing of parts in a system, and the routing shows the operation sequence of the parts and the stations to perform those operations. In addition, the conditions required for parts to be processed in particular stations are also indicated. This kind of situation usually occurs when alternative stations are capable of performing the same kind of operation for a part. The decision of choosing a station from them are basically based on the state of the stations. Consequently, if the operation processes of parts reach a dead end in a system, then those processes will also be warned in the report. Most of the time, the dead ends are caused by incomplete information about parts' operation processes in

system descriptions.

The format of the routing are showed as follows.

```
( ( [ part #] --- >
            ( ( <<opt. 1>> [station #] [condition] ) )
             --- >
              ( ( <<opt 2 >> [station #] [condition] ) )
               --- >
                   - - - - - - - - - - - - - - - - - -
))
```

(1). Parts and stations are denoted by part number and station number, and their full descriptions are in the part and facility description.

(2). The << opt. # >> indicates the operation number corresponding to the a part 's process in a system.

(3). The [condition] specifies the condition that a station is allowed to perform a particular operation for a part.

(4). Besides the above information, the following three kinds of messages are also provided in the routing sheets:

a. <exit> : To indicate that a part is complete its operations and exits a system.

b. <dead-end> : To indicate that the process of a part is terminated before it exits a system.

c. <merge-to [station #]> : to indicate that the previous alternative process flow of a part is currently joined together at the station [station #].

Finally, the other three kinds of information provided by the activity description is to present the time required to perform each type of activity in a system. The information includes the arrival time intervals of parts, the process times of parts in stations, and the transfer times of parts between stations.

CHAPTER VI

THE NATURAL LANGUAGE MODELING ENVIRONMENT PROTOTYPE

6.1. Current Implementation

In order to demonstrate the capability of the proposed natural language processing system, a prototype of the system has been developed. Basically, the construction of the NLME prototype follows two principles. First, the development of the system is breadth first instead of depth first. In other words, we try to equipped the prototype with all the system elements discussed in the previous two chapters. However, their functions may be only capable of handling problems to certain degrees of complexity. Secondly, in order to reduce tedious and time consuming programming work, we tried to adopt the available computer codes from literatures to the prototype system. For instance, the ATN compiler is written based on [Amsterdam, 1986] and [Christaller, 1983], and it is used to translate the augmented transition networks of sentence structures into LISP functions. In addition, the frame system manipulation functions are adopted from [Finin, 1986a,1986b], [Winston and Horn, 1984], and [Charniak, Riesbeck and McDermott, 1980].

Currently, the computer system used to implement the prototype system is the Domain system, Apollo work station which works in AGIS operation system. The computer codes of the system are written in Franz lisp which is one of the most distributed dialects of Lisp and the programs are stored in ten different files. Finally, the status of the current implementation of the NLME prototype is discussed in the following subsections.

6.1.1. Sentence Structures

In the prototype of the NLME, only parts of available sentence structures have been implemented, and they mainly consist of simple sentences and a few complex sentences. The current acceptable sentence structures of the NLME are given in Table 11. It should be pointed out, however, that the table merely lists the active voice sentence structures, and that the passive voice structures are also recognized by the Language Processor of the NLME. Following the sentence structures, Figure 17 gives the augmented transition networks which are defined based on the structures and are used to parse the input sentences. The definitions of arcs in the networks are also given in Appendix 1. The first network, SENTENCE represents the highest level constructions of a sentence. Each arc in the network stands for a major grammatical category of a sentence, such as subject, verb, object, etc. As for the NP and the PREP networks, they are used to support the SENTENCE networks to identify noun phrases and preposition phrases.

6.1.2 Modeling Capabilities

The current modeling capabilities of the NLME are somewhat limited. It is able to model manufacturing systems in terms of two kinds of entities (part and station), and three kinds of activities (arrival, transfer and operation). Each of them is described further in the following.

1. Part :

   A part in a NLME model stands for a specific type of work piece in a system. Parts can be assigned with attributes, such as lot size and the first arrival time.

2. Station:

   A station is referred to as a cluster of machine tools and they are grouped based on their functions or corresponding projects. A station is equipped with several

Table 11. The Acceptable Sentence Structures by the NLME Prototype

---

Sentence --> Adverb + Simple Sentence.
--> Complex Sentence.
Complex Sentence --> Subordinate Sentence + "," + Main Sentence.
--> Main Sentence + Subordinate Sentence.
Main Sentence --> Simple Sentence.
Subordinate Sentence --> Conjunction + Simple Sentence.
Simple Sentence --> Noun Phrase(subject)+ Verb Phrase + Adverb(time,location).
Noun Phrase --> Be + Preposition Phrase.
--> Be + Noun Phrase.
--> Be + [Adverb][1] + Adjective.
--> Intransitive Verb + Preposition Phrase.
--> Intransitive Verb + To-infinitive.
--> Transitive Verb + Noun Phrase (object).
--> Bitransitive Verb + Noun Phrase(indirect object)
+ Noun Phrase(direct object).
--> Bitransitive Verb + Noun Phrase(direct object) + "to" or "for"
+ Noun Phrase(indirect object).
Noun Phrase --> [Determiner or Quantifier] + [Number] + [Adjectives or Nouns]
+ Main Noun + [Preposition Phrases] + [To-infinitive].
--> Proper Noun.
--> Pronoun.
Preposition Phrase --> Preposition + Noun Phrase.
--> Preposition Phrase + Noun Phrase + "and" + Noun Phrase.
To-infinitive --> "to" + Verb Phrase.
Adverb(location, time) --> Noun Phrase.
--> Preposition Phrase.

1. The category specified in a pair of bracket is optional.

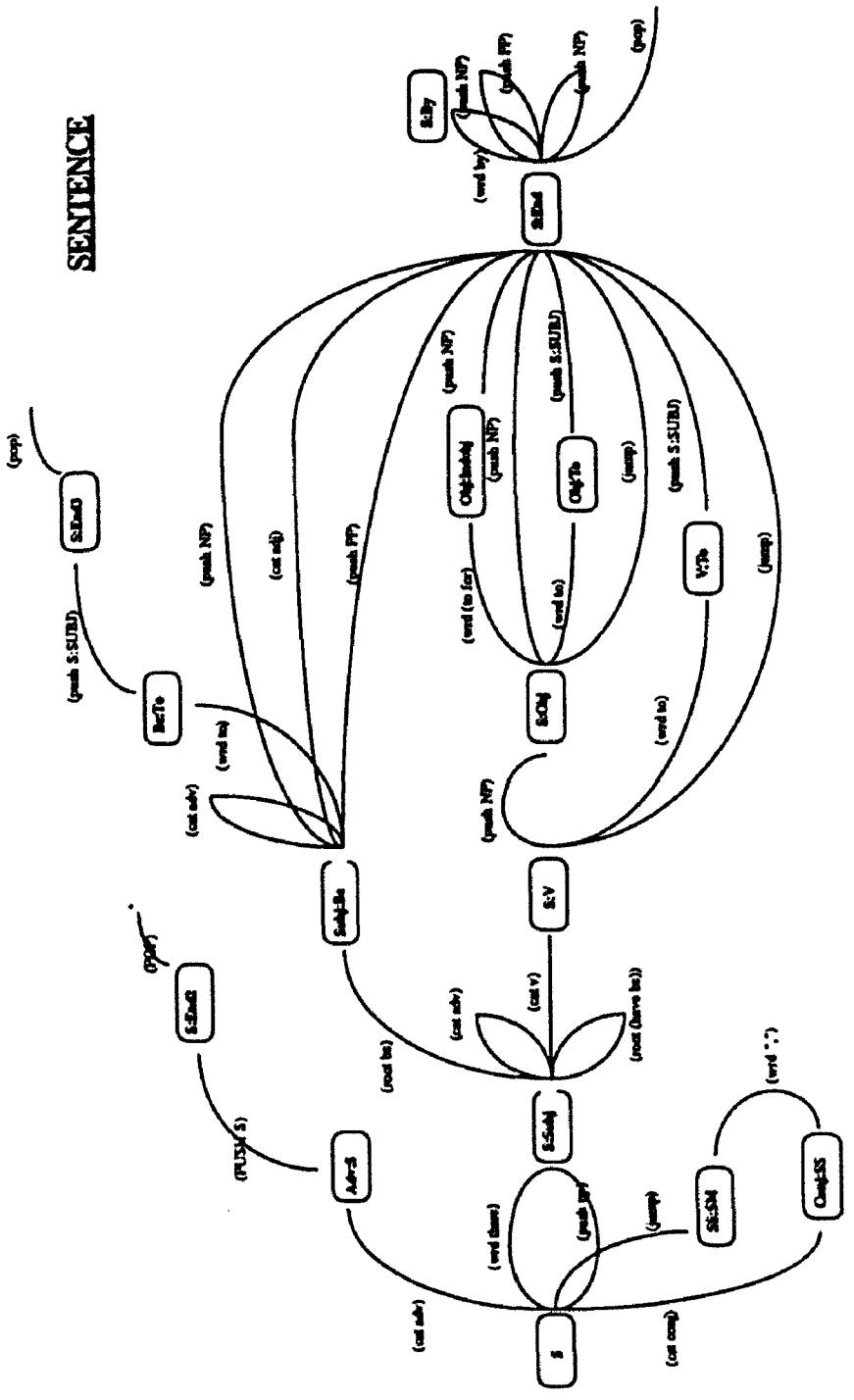
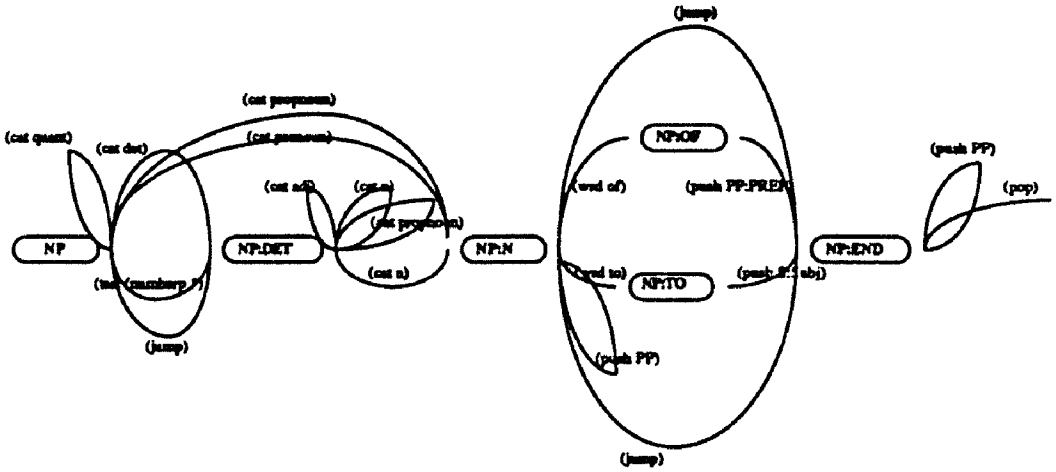


Figure 17 The Augmented Transition Networks Corresponding to the Sentence Structures in Table 11

NOUN PHRASE



PREPOSITION PHRASE

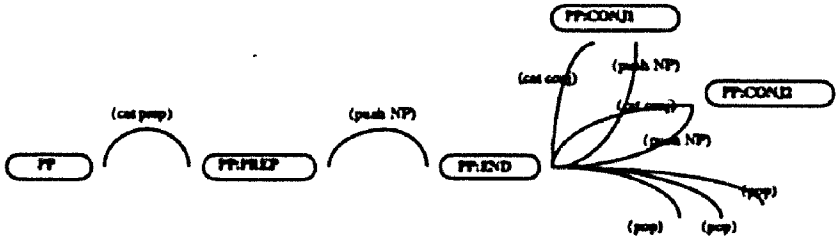


Figure 17. Continued.

machines and a buffer, and the number of machines and the size of the buffer can be specified. When the number of machines in a station is one, it can be treated as a single machine tool. The state of a station can be classified into available and busy. When a station is available, it means that the station has at least one machine available to process an incoming part. While a station is busy, all of its machines are occupied. Furthermore, The buffer in a station can also has available and full.

3. Arrival Event:

The arrival event describes the frequency of arrival of a particular part type, which is loaded into a system. The arrival time interval is its main attribute.

4. Operation Activity:

A part processed at a machine in a station is modeled by an operation activity. The station entity can have an operation time attribute.

5. Transfer Activity.

The transfer activity stands for the process of moving a part from one station to another. A conditional transfer is allowed in a model. Conditional transfers are to simulate the decisions of selecting stations to process incoming parts. The decisions are made based on the states of the stations which are capable of processing the parts, or the states of their buffers.

6.1.3. System Descriptions

System descriptions are the portrayals of system analysts' conceptual models of their manufacturing systems. For the problem description to be acceptable, its statements need to follow the sentence structures discussed previously, and the words in the statements should have been defined in the Lexicon. Moreover, the models described by the system descriptions should also be within the limit of the NLME's modeling capabilities described beforehand.

In addition, some restrictions are imposed on the ways to address the system descriptions. These restrictions are mainly from the incomplete implementations of the NLME. However, some of the constraints are to reduce the possible ambiguities of input sentences, and some are from the nature of the language. The restrictions are:

1. Adverbs, such as "next," "then," and "also" can be placed at the beginning of sentences. These adverbs are not used to modify any one part of sentences. Instead, they are used to help the sentences flow from one to another more smoothly.
2. The NLME makes a clear distinction among words used to express the behavior of a part's entering a system, transferring between stations, and leaving a system. Words with the meanings of "come in," such as "enter," and "arrive," can only be used to describe the situations that parts are moved into systems from outside. On the other hand, words with the meanings of departure, such as "exit," and "leave," can only be used to express the activities that parts are removed from systems. Finally, words, such as "transfer," "move," "go", and "transport," are used to indicate that parts are transferred between stations.
3. Conditional transfers are allowed in models. A conditional transfer is stated in a pair of sentences. The first sentence is an "if" complex sentence. The subordinator of the sentence is to specify the condition required to execute the transfer activity mentioned in the main sentence. The following sentence begins with the word "otherwise," and is to indicate the alternative station to go if the condition in the previous sentence is not satisfied. The following two sentences are an example of describing a conditional transfer.

    If the milling station is busy,
        the pcb is transferred to the drilling station. (6.1)

Otherwise, it is moved to the milling station. (6.2)

4. Pronouns and determiners can only be used to refer to stations and parts in the system descriptions. This is because the Sort Term Memory currently maintains only semantic representations of these two types of entity.
5. The contextual rules in the NLME prototype are capable of handling ambiguous sentences used to described events or activities, and/or their attributes.
6. Each sentence in the system description files should be put into a pair of parentheses, and the first letter of a sentences should be in the lower case, not in upper case. This is only the temporary treatment and the constraint can be released in the further implementation of the system.

6.1.4. Users' Interface

Currently, the process of applying the NLME can be divided into two steps. The first step is to create a system description based on the manufacturing system under study. The creation process is through the text editor of the NLME resident computer system, Apollo work stations The second step is to operate the NLME and the processes are under the LISP environment in the computer system. Generally, commands are issued to the NLME through a series of menus. These menus are responsible for functions, such as to process system descriptions, to trace operation processes, to manipulate the Working Memory, and to generate reports and programs. The functions of each menu are further described as follows.

1. Main Menu (Figure 18).

   This is the root menu of the menu system. From this menu, users can access the lower level menus.

```
* ---------------------------------------------------------- *
                    << Main Menu >>
          < Options >

              1. Language Processor Utilities.
              2. Trace or Untrace the Processes.
              3. Working Memory Utilities.
              4. Reports and Model Verification.
              5. Program Generator Utilities.
              6. Exit.
* ---------------------------------------------------------- *
```

Figure 18. Main Menu

2. Language Processor Menu (Figure 19).

In this menu, five actions can be taken to command the Language Processor, and they are :

(1). To request for importing a system description from an external file,
(2). To add more information to a model from sentences stored in an external file,
(3). To add a sentence to the current system description under the interactive model,
(4). To confirm words in a system description with the Lexicon.
(5). To initialize the Language processor, and to erase the contents of the Short Term Memory and the Contextual Memory.

Moreover, in the menu system, if a number greater than the number of option in a menu is entered, that will cause a lower level menu to return to the root menu.

```
* ---------------------------------------------------------------- *
                  << Language Processor Menu >>
          < Options >

              1. Input a System Description.
              2. Add More Statements to the Current Description.
              3. Add a Statement to the Current Description.
              4. Check Words in a System Description.
              5. Re-initialize the Language Processor.
* ---------------------------------------------------------------- *
```

Figure 19. Language Processor Menu

3. Trace Utility Menu (Figure 20).

This menu is used to keep track of the operation processes of the NLME. The menu provides the options to display or to stop displaying the intermediate results of the processes. Moreover, an option menu is associated with the trace utilities menu to provide the alternatives to display :

(1). input sentences after they are read from an external file,
(2). processed sentences when they are generated from the PreProcessor,
(3). the contents of the Short Term Memory whenever it is accessed,
(4). the semantic representations used to disambiguate sentences when the contextual rules are applied,
(5). the semantic representations of input sentences after they are processed by the Language Processor,
(6). <store> functions when the corresponding semantic representations are examined by the Semantic Representation Interpreter,
(7). the contents of frames after they are accessed by <store> functions,
(8). semantic representations when they are generated by the ATN parser.

```
* ---------------------------------------------------------- *
                    << Trace Utilities >>
          < Options >

              1. Trace the Processes.
              2. Untrace the Process.
* ---------------------------------------------------------- *


* ---------------------------------------------------------- *
                  << Trace, Untrace Options >>

              1. Input Sentence.
              2. PreProcessor Output.
              3. Short Term Memory
              4. Contextual Analysis.
              5. Semantic Representations Generated.
              6. <Store> Functions generated.
              7. Contents of the frame being accessed.
              8. ATN Parser Output.
* ---------------------------------------------------------- *
```

Figure 20. Trace Utility Menu

4. Working Memory Utility Menu (Figure 21).

The utilities provided by this menu are basically used to manipulate the Working Memory. Functions of the utilities include :

(1). To display the current contents of the Working Memory.

(2). To save the current contents of the Working Memory into an external file.

(3). To restore contents of the Working Memory with the data in an external file.

(4). To erase the current contents of the Working Memory and to set up its initial structure.

(5). To store a semantic representation into the Working Memory.

```
* ---------------------------------------------------------- *
        << Working Memory (WM) Utilities >>
      < Options >

          1. Display the Contents of the Current WM.
          2. Save the Contents of the Current WM.
          3. Restore the Contents of WM from a File.
          4. Re-initialize the Contents of WM.
          5. Enter a Semantic Representation into WM.
* ---------------------------------------------------------- *
```

Figure 21. Working Memory Utility Menu

5. Report Utility Menu (Figure 22).

Two options are in this menu. The first one is to generate the process flow charts of all the parts in the current model. The flow charts can be used to verify the current model. The second option is to generate a summary report of the current model.

```
* ---------------------------------------------------------- *
              << Report Menu>>
      < Options >

          1. Verify the Current Model.
          2. Generate a Summary Report.
* ---------------------------------------------------------- *
```

Figure 22. Report Utility Menu.

6. Program Generator Menu (Figure 23).

The utilities in this menu are used to create model templates and program files. The first option of the menu is to generate a model template based on the current structure of the Working Memory. However, the second option is to create a program file according to the data in the memory and its model template.

```
* ---------------------------------------------------------- *
          << Program Generator Menu>>
      < Options >

          1. Generate a Model Template.
          2. Generate a Program File
* ---------------------------------------------------------- *
```

Figure 23. Program Generator Menu

6.2 An Annotated Example

Basically, the objective of the previous discussions was to present the strategies applied by the NLME's components to accomplish their functions. The emphasis was on individual techniques. However, in this section, we would like to provide an integrated view of how all the components function together. The whole picture of the NLME is projected through the trace of its operation processes, which is provided in the following descriptions. Hopefully, this will facilitate better understanding of the system.

The following trace corresponds to the processing of the system description given in Table 3. The whole operations are started from requesting the NLME to perform all levels trace and to input a system description from file Table3.

```
(**** Trace Started ****)
    File Name>> table3
 >>>>>> |Input Sentence : |
 (the cast part enters the CIM work center)                    (6.3)
 ------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |
 nil
 ------------------------------------------------------------
```

The above message is to indicate the current contents of the Short Term Memory before the input sentence is to be parsed. Currently, the contents of the memory is empty, nil.

```
 >>>>>> |PreProcessor Output : |
 (the cast part enters the CIM work-center)
 ------------------------------------------------------------
```

Here, the PreProcessor is to confirm words in the above sentence with the Lexicon and to combine "work center" into a single word "work-center" as a word phrase.

```
 >>>>>> | Updated Short Term Memory|
 ((part (description (type cast))))
 ------------------------------------------------------------
```

During the process, the contents of the Short Term Memory are updated with the noun phrase "the cast part" in the input sentence.

```
 >>>>>> |Semantic Structure from Parser|
 (event (type arrival) (part (description (type cast))) (system (name CIM)))
 ------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |
 ((event (type arrival) (part (description (type cast))) (system (name CIM))))
 ------------------------------------------------------------
```

The difference between above two kinds of information is that the first one is direct from the parser and the representation has not been examined by the Context Applier. However, the second one is generated from the Context Applier using the first one as source.

```
>>>>>> |Function Generated|
(<store> (<event> (type arrival) (part (description (type cast))))
         (<value> (interval)))
-----------------------------------------------------------------
```

Basically, the first sentence does not contribute to the model formulation. It is only to indicate a specific part type to be processed in the model. According to the information , the corresponding frame in the Working Memory is created and is listed as follows. However, the information displayed not only includes the information stored in a frame, but also includes the information can be accessed from this frame through its part-of link.

```
>>>>>> Contents of Frame : arrival1
frame arrival1
 part = part1
 system = (name CIM)
 consist-of = arrival1
 object-type = arrival
 part-of = arrival part1
-----------------------------------------------------------------

>>>>>> |Input Sentence : |
(the part arrives at the milling station according to a Poisson process
 with a mean of 3 minutes)                                          (6.4)
-----------------------------------------------------------------

>>>>>> |Contents of Short Term Memory : |
((part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |PreProcessor Output : |
(the part arrives at the milling station according-to a Poisson process
 with a mean of 3 minutes)
-----------------------------------------------------------------
```

During the parsing sentence 6.4, the noun phrase "the part" is encountered. In order to resolve the referential problem, the parser looks up the Short Term Memory.

```
>>>>>> Search Short Term Memory for : part
>>>>>> |Contents of Short Term Memory : |
((part (description (type cast))))
-----------------------------------------------------------------
```

Later, the new noun phrase "the milling station" is parsed and is stored into the Short Term memory.

```
>>>>>> | Updated Short Term Memory|
((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |Semantic Structure from Parser|
(event (type arrival)
       (part (description (type cast)))
       (to (station (description (function milling))))
       (duration
       (distribution (function <poisson>)
                     (mean (number (figure 3) (unit <minute>))))))
-----------------------------------------------------------------

>>>>>> |Semantic Structure Generated : |
((event (type arrival)
        (part (description (type cast)))
        (to (station (description (function milling))))
        (duration
        (distribution (function <poisson>)
                      (mean (number (figure 3) (unit <minute>)))))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<event> (type arrival) (part (description (type cast))))
         (<value> (to (to (station (description (function milling)))))
                  (interval
                     (distribution (function <poisson>)
                                   (mean (number (figure 3)
                                                 (unit <minute>)))))))
-----------------------------------------------------------------
```

```
>>>>>> Contents of Frame : arrival1
frame arrival1
 part = part1
 system = (name CIM)
 consist-of = arrival1
 object-type = arrival
 interval = (distribution (function <poisson>)
                          (mean (number (figure 3) (unit <minute>))))
 to = (station1)
 part-of = arrival part1
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |
(the station processes it for 5 minutes)                          (6.5)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |
 ((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |
 (the station processes it for 5 minutes)
-----------------------------------------------------------------
```

In sentence 6.5, the noun phrase "the station" and the pronoun "it" require the parser to resolve the referential situation by searching the Short Term memory.

```
 >>>>>> Search Short Term Memory for : station
 >>>>>> |Contents of Short Term Memory : |
 ((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> Search Short Term Memory for : part
 >>>>>> |Contents of Short Term Memory : |
 ((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|
 (event (type operation)
        (part (description (type cast)))
        (station (description (function milling)))
        (duration (number (figure 5) (unit <minute>))))
-----------------------------------------------------------------
```

```
>>>>>> |Semantic Structure Generated : |
((event (type operation)
        (part (description (type cast)))
        (station (description (function milling)))
        (duration (number (figure 5) (unit <minute>)))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<event> (type operation)
                  (part (description (type cast)))
                  (station (description (function milling))))
         (<value> (op-time (number (figure 5) (unit <minute>))))))
-----------------------------------------------------------------

>>>>>> Contents of Frame : operation1
frame operation1
 station = station1
 part = part1
 system = (name CIM)
 consist-of = operation1
 machine-no = 1
 object-type = operation
 op-time = (number (figure 5) (unit <minute>))
 part-of = operation part1 station1
 ---------------------------------------------------------------

 >>>>>> |Input Sentence : |
 (the milling station has a waiting space for 1 part)                    (6.6)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |
 ((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |
 (the milling station has a waiting space for 1 part)
-----------------------------------------------------------------

 >>>>>> | Updated Short Term Memory|
 ((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|
 (attribute (type queue)
            (station (description (function milling)))
            (value (number (figure 1) (unit <part>))))
-----------------------------------------------------------------
```

```
>>>>>> |Semantic Structure Generated : |
((attribute (type queue)
            (station (description (function milling)))
            (value (number (figure 1) (unit <part>)))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<object> (station (description (function milling))))
         (<value> (buffer-size (number (figure 1) (unit <part>)))))
-----------------------------------------------------------------

>>>>>> Contents of Frame : station1
frame station1
 machine-no = 1
 operations = operation1
 object-type = station
 buffer-size = (number (figure 1) (unit <part>))
 consist-of = operation1
 description = (description (function milling))
 part-of = station
-----------------------------------------------------------------

>>>>>> |Input Sentence : |
(the station has 2 machines)                                    (6.7)
-----------------------------------------------------------------

>>>>>> |Contents of Short Term Memory : |
((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |PreProcessor Output : |
(the station has 2 machines)
-----------------------------------------------------------------

>>>>>> Search Short Term Memory for : station
>>>>>> |Contents of Short Term Memory : |
((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |Semantic Structure from Parser|
(attribute (type machine-no)
           (station (description (function milling)))
           (value (number (figure 2) (unit <machine>))))
-----------------------------------------------------------------
```

```
>>>>>> |Semantic Structure Generated : |
((attribute (type machine-no)
            (station (description (function milling)))
            (value (number (figure 2) (unit <machine>)))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<object> (station (description (function milling))))
         (<value> (machine-no (number (figure 2) (unit <machine>)))))
-----------------------------------------------------------------

>>>>>> Contents of Frame : station1
frame station1
 operations = operation1
 object-type = station
 machine-no = (number (figure 2) (unit <machine>))
 buffer-size = (number (figure 1) (unit <part>))
 consist-of = operation1
 description = (description (function milling))
 part-of = station
-----------------------------------------------------------------

>>>>>> |Input Sentence : |
(then the cast part goes to the drilling station)                    (6.8)
-----------------------------------------------------------------

>>>>>> |Contents of Short Term Memory : |
((station (description (function milling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |PreProcessor Output : |
(then the cast part goes to the drilling station)
-----------------------------------------------------------------

>>>>>> | Updated Short Term Memory|
((station (description (function drilling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |Semantic Structure from Parser|
(event (type transfer)
       (part (description (type cast)))
       (to (station (description (function drilling)))))
-----------------------------------------------------------------
```

After the above representation is generated, the Context Applier is then applied to check the completeness of the information. Because in the representation, the starting station of the transfer is not indicated, a contextual analysis is initiated.

```
>>>>>> |Context Check|
(event (type operation)
       (part (description (type cast)))
       (station (description (function milling)))
       (duration (number (figure 5) (unit <minute>))))
-----------------------------------------------------------------
```

After the contextual rules are applied to check the previous input sentence, the above representation is to indicate that sentence 6.5 is located and is used to disambiguate the current sentence. The final semantic representation of the current sentence is listed as follows.

```
>>>>>> |Semantic Structure Generated : |
((event (type transfer)
        (part (description (type cast)))
        (from (station (description (function milling))))
        (to (station (description (function drilling))))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<event> (type transfer)
                  (part (description (type cast)))
                  (station (description (function milling))))
         (<value> (to (to (station (description (function drilling)))))))
-----------------------------------------------------------------

>>>>>> Contents of Frame : transfer1
frame transfer1
 from = station1
 part = part1
 system = (name CIM)
 consist-of = transfer1
 buffer-size = (number (figure 1) (unit <part>))
 machine-no = (number (figure 2) (unit <machine>))
 object-type = transfer
 to = (station2)
```

```
part-of = transfer part1 station1
-----------------------------------------------------------------

>>>>>> |Input Sentence : |
(the transfer time is uniformly distributed from 1.5 to 2 minutes)        (6.9)
-----------------------------------------------------------------

>>>>>> |Contents of Short Term Memory : |
((station (description (function drilling))) (part (description (type cast))))
-----------------------------------------------------------------

>>>>>> |PreProcessor Output : |
(the transfer time is uniformly distributed from 1.5 to 2 minutes)
-----------------------------------------------------------------

>>>>>> |Semantic Structure from Parser|
(attribute (type duration)
           (event transfer)
           (from (number (figure 1.5)))
           (to (number (figure 2) (unit <minute>)))
          (duration
                (distribution (function <uniform>)
                              (from (number (figure 1.5)))
                              (to (number (figure 2) (unit <minute>))))))
-----------------------------------------------------------------
```

Because, the current sentence does not describe the part and stations associated with the transfer activity, context is applied and sentence 6.8 is found and is used to resolve the ambiguity of the current sentence.

```
>>>>>> |Context Check|
(event (type transfer)
   (part (description (type cast)))
   (from (station (description (function milling))))
   (to (station (description (function drilling)))))
-----------------------------------------------------------------

>>>>>> |Semantic Structure Generated : |
((event (type transfer)
        (part (description (type cast)))
        (from (station (description (function milling))))
        (to (station (description (function drilling))))
       (duration
           (distribution (function <uniform>)
                         (from (number (figure 1.5)))
```

```
                    (to (number (figure 2) (unit <minute>)))))))
-----------------------------------------------------------------

>>>>>> |Function Generated|
(<store> (<event> (type transfer)
                  (part (description (type cast)))
                  (station (description (function milling))))
         (<value> (to (to (station (description (function drilling)))))
                  (tf-time
                      (to (station (description (function drilling)))
                          (duration
                              (distribution (function <uniform>)
                                            (from (number (figure 1.5)))
                                            (to
                                                (number (figure 2)
                                                        (unit <minute>))
 )))))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : transfer1
frame transfer1
 from = station1
 part = part1
 system = (name CIM)
 consist-of = transfer1
 buffer-size = (number (figure 1) (unit <part>))
 machine-no = (number (figure 2) (unit <machine>))
 object-type = transfer
 tf-time = (station2 (distribution (function <uniform>)
                                   (from (number (figure 1.5)))
                                   (to (number (figure 2) (unit <minute>)))))
 to = (station2)
 part-of = transfer part1 station1
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |
 (the process time at the drilling station is normally distributed
  with a mean of 2 and a variance of 0.5 minutes)                    (6.10)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |
 ((station (description (function drilling))) (part (description (type cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |
 (the process time at the drilling station is normally distributed
  with a mean of 2 and a variance of 0.5 minutes)
```

```
------------------------------------------------------------

>>>>>> | Updated Short Term Memory|
((station (description (function drilling))) (part (description (type cast))))
------------------------------------------------------------

>>>>>> |Semantic Structure from Parser|
(attribute (type duration)
          (event operation)
          (station (description (function drilling)))
          (duration
              (distribution (function <normal>)
                            (mean (number (figure 2)))
                            (variance (number (figure 0.5) (unit <minute>))))))
------------------------------------------------------------
```

The part to be processed is not indicated in the current sentence, and contextual analysis is performed. Sentence 6.9 is used to disambiguate the current sentence.

```
>>>>>> |Context Check|
(event (type transfer)
   (part (description (type cast)))
   (from (station (description (function milling))))
   (to (station (description (function drilling))))
   (duration
         (distribution (function <uniform>)
                       (from (number (figure 1.5)))
                       (to (number (figure 2) (unit <minute>))))))
------------------------------------------------------------

>>>>>> |Semantic Structure Generated : |
((event (type operation)
    (part (description (type cast)))
    (station (description (function drilling)))
    (duration
        (distribution (function <normal>)
                      (mean (number (figure 2)))
                      (variance (number (figure 0.5) (unit <minute>)))))))
------------------------------------------------------------

>>>>>> |Function Generated|

(<store> (<event> (type operation)
                  (part (description (type cast)))
                  (station (description (function drilling))))
         (<value>
```

```
                    (op-time
                        (distribution (function <normal>)
                                      (mean (number (figure 2)))
                                      (variance (number (figure 0.5) (unit
<minute>)))))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : operation2
frame operation2
 station = station2
 part = part1
 system = (name CIM)
 consist-of = operation1 operation2
 machine-no = 1
 object-type = operation
 op-time = (distribution (function <normal>)
                         (mean (number (figure 2)))
                         (variance (number (figure 0.5) (unit <minute>))))
 part-of = operation part1 station2
-----------------------------------------------------------------
```

The rest of the processes are similar to the previous processes, and will not be discussed further in this section, but are presented in Appendix 2. The summary report and the program corresponding to the system description are given in Table 12 and 13.

Table 12 A Summary Report Corresponding to the System Description in Table 3.

***** Summary Report For CIM System *****

Simulation Time : 24000 <minute>

A. Jobs Description.

Part : part1

a. Description : (description (type cast))
b. Enter Time : nil
c. Lot Size : nil

B. Facilities Description.

Station : station1

a. Description : (description (function milling))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station2

a. Description : (description (function drilling))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station3

a. Description : (description (function polishing))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 1) (unit <machine>))

C. Activities Description.

1. Routings.

```
((part1 --->
     ((<<opt.1>> station1)
      --->
      ((<<opt.2>> station2)
       --->
       ((<<opt.3>> station3) ---> (<exit>))))))
```

Table 12. Continued

---

2. Arrival Times.

Part	To	Value
part1	station1	(distribution (function <poisson>) (mean (number (figure 3) (unit <minute>))))

3. Process Times.

Part	Station	Value
part1	station1	(number (figure 5) (unit <minute>))
part1	station2	(distribution (function <normal>) (mean (number (figure 2))) (variance (number (figure 0.5) (unit <minute>))))
part1	station3	(number (figure 3) (unit <minute>))

4. Transfer Times.

Part	From	To	Value
part1	station1	station2	(distribution (function <uniform>) (from (number (figure 1.5))) (to (number (figure 2) (unit <minute>))))
part1	station2	station3	(distribution (function <exponential>) (mean (number (figure 2) (unit <minute>))))
part1	station3	<exit>	nil

Table 13. A SLAM Program Corresponding to the System Description in Table 3

```
GEN,NLME,CIM SYSTEM,,1;
LIMITS,3,3,100;
;
NETWORK;
;
;     attribute description:
;
;       attribute 1 : part arrival time
;       attribute 2 : part type
;       attribute 3 : part process time in a station
;
;
;     load part1 into system
;
PAT1  CREATE,NPSSN(3,1), , ,1;
;
;     station1
;
;
STN1  GOON,1;
;
Q1    QUEUE(1),,1,BLOCK;
      ACT(2)/1,5;
      GOON,1;
;
      ACT/4,UNFRM(1.5,2,1),ATRIB(2).EQ.1,STN2;
;
;     station2
;
;
STN2  GOON,1;
;
Q2    QUEUE(2),,2,BLOCK;
      ACT(2)/2,RNORM(2,0.7,1);
      GOON,1;
;
      ACT/5,EXPON(2,1),ATRIB(2).EQ.1,STN3;
;
```

Table 13.Continued

```
;
;     station3
;
;
STN3  GOON,1;
;
Q3      QUEUE(3),,1,BLOCK;
        ACT(1)/3,3;
        GOON,1;
;
        ACT/6, ,ATRIB(2).EQ.1,EXIT;
;
;
;     collect statistics
;
;
EXIT  COLCT,INT(1),PART1 IN SYS;
        TERM;
        ENDNETWORK;
INIT,0,24000;
FIN;
```

CHAPTER VII

AN EMPIRICAL STUDY OF THE SYSTEM

7.1. The Experiment

In order to evaluate the performance of the proposed system, an experiment was designed to compare the time factors of the manual approach and the NLME approach to formulate models. Theoretically, the estimation of the time required to apply these two approaches should include two stages. The first stage is the learning process. For the manual approach, the process includes learning simulation modeling concepts and how to use a particular simulation language to formulate a model. As for the NLME approach, the time includes learning the operation of the NLME system and its limitation. The second stage is the problem solving process. The process involves activities such as interpreting problems, formulating models, coding programs and debugging programs. For a problem, the above process usually needs to be repeated until a correct model is achieved.

However, because of the constraints of available subjects, time required, and the capability of the current NLME prototype, the author was unable to perform an experiment following the processes described above. First, it is difficult to find subjects willing to spend such a long length of time on the experiment. Secondly, the NLME system is still a prototype. The system will not be able to demonstrate its full strength. Nevertheless, it is important to evaluate the performance of the proposed system to see if its application is justified. However, the experiment focused only on the problem solving process.

The experiment involved six subjects and three different problems. All six subjects were graduate students, and each of them had previously taken at least one simulation

course for one semester. In the simulation class, they were taught simulation modeling techniques and a special simulation language, SLAM. As for the problems used, they were created and based on the acceptable sentence structures, the available words in the Lexicon, and the modeling capability of the NLME prototype. The first problem described a manufacturing system with four machine stations and one part type. The process of the part consisted of three operations and one of them could be performed in either of the two stations with the capabilities, and the decision made was according to the queue status of a particular station. Furthermore, twenty-four sentences were used to describe the system and they are given in Table 14. The second problem involved five machine stations and two different part types. The processes of these two part types included three and four different operations. The description of this problem was composed of thirty-five sentences and is given in Table 15. The last problem was a manufacturing system with six machine stations and two kinds of parts. Each part type needed to go through four operations in order to complete its process and one of the four operations could be performed by two different stations. The system was described using forty-eight sentences and they are given in Table 16. Furthermore, the summary reports and the programs generated by the NLME corresponding to these three problems are provided in Appendix 3.

The process of the experiment started from randomly assigning each problem to two subjects. First, the subjects were asked manually to formulate the corresponding models using SLAM network approach. This process stopped when the simulation model could be executed by a computer without any problem. The second step of the experiment was to ask the same subjects to use the NLME system to generate simulation programs. Finally, both of the times required to perform these two processes were recorded in order to make comparisons.

Table 14.Experiment.Problem Number One

PROBLEM ONE

The cast part enters the DOMAIN work center. First, it arrives at the drilling station according to a Poisson process with a mean of 3 minutes. The process time at the station is normally distributed with a mean of 5 and a variance of 1 minute. There is a waiting space for 3 parts in the station. The station has 3 machines. If the buffer of the turning station is full, the cast part is moved to the milling station. Otherwise, it is moved to the turning station. The time to transfer the cast part to the turning station is 1.2 minutes. Also, the service time at the station is exponentially distributed with a mean of 4 minutes. The turning station has 3 machines. The station has a waiting space for 2 parts. Next, the part is transferred to the polishing station. The transfer time is 1.5 minutes. The process time at the polishing station is uniformly distributed from 2 to 3 minutes. There are 2 machines in the station. Also, the station has a waiting space for 2 parts. The time to transfer the cast part from the drilling station to the milling station is 1 minute. The process time at the milling station is normally distributed with a mean of 6 and a standard deviation of 0.6 minutes. There is 1 machine in the milling station. The station has a waiting space for 1 part. Next, the part is moved to the polishing station. The transfer time is 1.2 minutes. Then it leaves the DOMAIN work center.

The DOMAIN work center is to be simulated for 1200 minutes.

Table 15. Experiment Problem Number Two

PROBLEM TWO

The steel part arrives at the FRANZ system. First, it is provided to the turning station every 5 minutes. The time to process the part in the station is normally distributed with a mean of 7 and a variance of 1 minute. The turning station has 2 machines. The station has a waiting space for 2 parts. Next, the part goes to the welding station. The transfer time is 0.5 minutes. Then the part is processed for 5 minutes. There are 5 machines in the welding station. There is a waiting space for 4 parts in the station. Next, the steel part is moved to the coating station. The time to transfer the part to the station is 1 minute. The process time at the station is uniformly distributed between 5 and 6 minutes. The coating station has 2 machines. Also, there is a waiting space for 1 part in the station. Next, the steel part is transferred to the painting station. Then it is processed at the station. The transfer time is 0.5 minutes. The service time is 3 minutes. There are 3 machines in the painting station. There is a waiting space for 3 parts in it. Next, the steel part leaves the system.

The aluminum part enters the FRANZ system. The part arrives at the shearing station according to a Poisson process with a mean of 3 minutes. It is processed for 4 minutes in the station. The shearing station has 3 machines. Also the station has a waiting space for 2 parts. Next, the aluminum part is moved to the welding station. The transfer time is 0.8 minutes. The service time at the welding station is exponentially distributed with a mean of 5.5 minutes. Next, the part goes to the painting station. The time to transfer the aluminum part to the painting station is 1.2 minutes. The process time at the painting station is 4 minutes. Then the part leaves the FRANZ system.

The FRANZ system is to be simulated for 1200 minutes.

Table 16. Experiment Problem Number Three

PROBLEM THREE

The steel part enters the APOLLO work center. First, it arrives at the drilling station according to a Poisson process with a mean of 10 minutes. The drilling station processes it for 5 minutes. There is a space for 2 items at drilling station. Also, the drilling station has 2 machines. Then the steel part goes to the adjustment station. The transfer time is normally distributed with a mean of 5 and a variance of 1 minute. The time to process the steel part at the adjustment station is uniformly distributed between 2 and 5 minutes. The adjustment station has a space for 2 items. There are 2 machines in the station. If the buffer of the turning station is full, the steel part is moved to the milling station. Otherwise, it is moved to the turning station. The time to transfer steel part to the turning station is exponentially distributed with a mean of 2 minutes. Then the steel part is processed for 2 minutes in the turning station. Next, it is moved to the polishing station . The transfer time is normally distributed with a mean of 5 minutes and a standard deviation of 2 minutes. The polishing station processes the steel part for 4 minutes . The transportation time of the steel part from the adjustment station to the milling station is exponentially distributed with a mean of 2 minutes. The time to process the steel part at the milling station is 5 minutes. The milling station has 1 machine. Also, the station has a space for 1 item . Next, the steel part is transferred to the polishing station . The transfer time is 2 minutes. Then it leaves the system.

Table 16. Continued

The nickle part enters the APOLLO work center. The nickle part is provided to the drilling station every 5 minutes. First, it is processed for 3 minutes. Then it is moved to the adjustment station. The transfer time is uniformly distributed from 2 to 3 minutes. The service time of the nickle part at the adjustment station is normally distributed with a mean of 5 and a variance of 1 minute. Next, it goes to the turning station. The time to transfer the nickle part to it is exponentially distributed with a mean of 3 minutes. The turning station processes it for 5 minutes. There are 2 machines in the turning station. Also, there is a space for 2 parts in it. If the polishing station is busy, it is moved to the washing station . Otherwise, it is moved to the polishing station . The transfer time to the washing station is exponentially distributed with a mean of 2 minutes . The process time at the washing station is normally distributed with a mean of 7 and a variance of 2 minutes . Then it leaves the system. There is a space for 1 part at the washing station. Also, there is 1 machine in it. The time to transfer the nickle part from the turning station to the polishing station is uniformly distributed between 3 and 4 minutes. The polishing station processed the nickle part for 6 minutes. Then the nickle part leaves the system. The polishing station has 2 machines. The station has a space for 2 parts.

The APOLLO work center is to be simulated for 1200 minutes .

7.2. Experimental Result

The result of the experiment is given in Table 17. Under the manual approach category, the data corresponding to subject three is the minimum time required to generate a model, and the data corresponding to subject four was not recorded. The minimum time required indicates that the subject failed to come out with an executable program because of syntax and logic error. After the program was examined, it was decided that the model needed to be re-built, and the experiment was terminated. As for the missing data, this is because the subject was unable to continue to participate the experiment after he spent more than two hours on problem three which is the most complicated one among the three problems.

Table 17. Experimental Result

Subject	Problem	Manual (min)	NLME (min)
1	2	125.	35.
2	1	65.	30.
3	2	150.*1	25.
4	3	.*2	43.
5	3	225.	50.
6	1	80.	18.

*1. Experiment was terminated due to unrecoverable error in the program.

*2. The subject was unable to finish the experiment.

The result of the experiment shows a significant time saving when the NLME system is applied. The average time spent on the three problems is presented in Table 18, and the plot is shown in Figure 24. Generally speaking, the NLME approach is three times to five time faster than the manual approach. Especially, the time saving is increased along with the complexity and size of problems.

7.3. Some Remarks About the Experiment

Several observations from this experiment are worthwhile mentioning here. For the manual approach, the subjects generally spent around 85% of the total time on formulating the models and around 15% of the time on coding and debugging the programs. However, during the model formulation period, a large amount of effort was used to model some special conditions described in the problems, such as alternative operations, or two different part types processed in the same station. Sometimes, these conditions themselves

Table 18. The Average Time Spent on each Problem by the Two Approaches

Problem	Manual (Min)	NLME (Min)
1	72.5	24.0
2	125.0*1	30.0
3	225.0*2	46.5

*1. Only the data of subject one is included.
*2. Only the data of subject five is included.

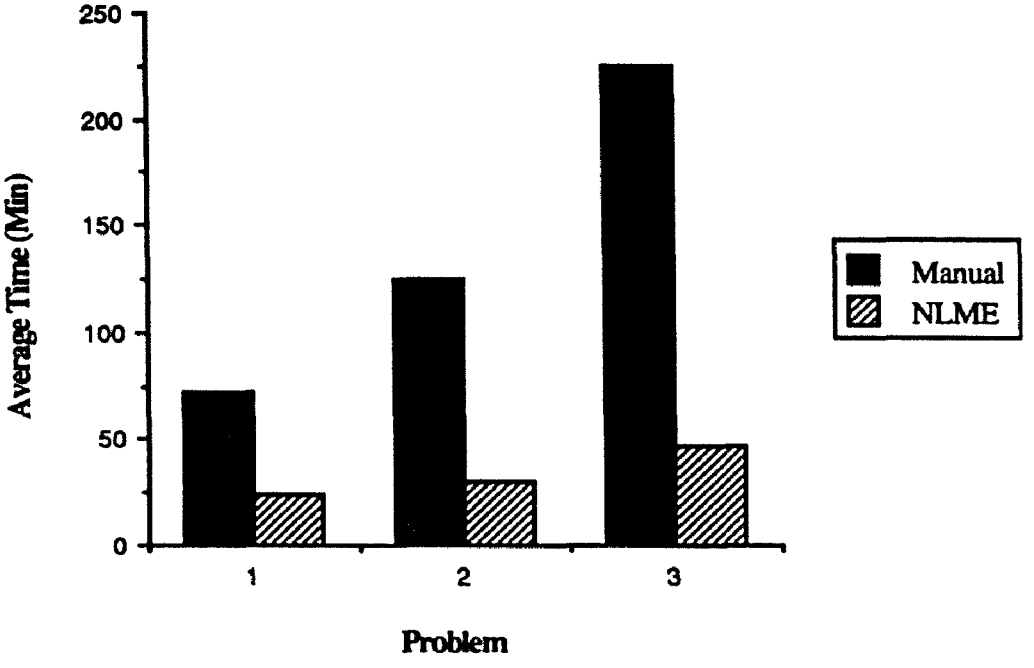


Figure 24. Manual Approach Vs NLME Approach

are not so complicated and difficult to formulate, but the subjects lack of knowledge of certain features of the simulation language or have to model the conditions in a more complicated way.

Recently, the term "expert system" has been mentioned among a variety of research studies. The basic idea of it is to incorporate human expertise of a certain problem domain into computer system. The system can then be used by inexperienced users to solve the problems under their domain. The same concept can also be applied to the area of model formulation process. An expert system can be constructed based on the knowledge of experienced simulation analysts. Then, the system can be used to provide advice about

how to model a particular situation using a simulation language under a specific problem domain. Therefore, a great amount of time saving can be gained from using such a system.

On the other hand, for an experienced system analyst, the NLME type of system can be used to construct the first draft of the simulation model of a system. The preliminary model can provide an overview and framework of the final model, and the analyst can release himself or herself from tedious, but simple tasks and spend his or her time and effort to concentrate on modeling the critical conditions in a system. Hence, The NLME system can also benefit experienced users.

CHAPTER VIII
CONCLUDING REMARKS

8.1. Conclusions

The model development process is usually labor and cost intensive and is error prone. Particularly, as hardware cost continues to decline in recent years, the cost of programming becomes more and more dominant and apparent. In order to simplify the model formulation process, the programming of simulation model has evolved from general purpose computer languages to special simulation languages and up to current menu driven and graphic input approaches. The significance of this research is to improve the existing simulation methodology and to explore a new dimension in the area of model formulation. The study is to provide a new form of modeling approach with an old tool, natural language.

For most people, natural language is the most common way of thinking, learning and communicating. No other means is as acceptable and flexible as it is. The current system can not and perhaps will not accept the full range of human language and interpret all the meanings that people normally use. However, the system can still support a level of communication that is much more natural and convenient. Especially, if the system is operated under interactive mode, the system can immediately ask users to help, to detect errors, and to clarify ambiguities. Such systems can be much easier to use than most conventional systems.

Finally, the specific objectives set forth in the introduction were met, and a prototype of the proposed natural language modeling environment was designed and implemented.

Even the NLME prototype has limited capability in terms of the acceptable sentence structures and modeling, the potential of the NLME can still be foreseen from the demonstration of its prototype.

8.2. Directions for the Further Research

This research is not in anyway conclusive. Several directions for further research emerge form this initial exploration. These possible developments include some extension of the NLME system and its potential usages. They are described as follows.

1. The Natural Language Modeling Environment in this research is only capable of understanding input sentences. However, communication between people and machines should be bi-directional. One of the possible extensions is to enhance the system with sentence generation capability. Then the system can initiate the conversation with users and request additional information about models.
2. Current implementation of the NLME can only relieve system analysts from modeling tasks. The analysts still need some knowledge about simulation in order to analyze the simulation output. Hence, a post processor capable of examining simulation output can be added to the system. The post processor should be able to present the simulation result in a comprehensive format, detect unusual levels of activities from outputs, and issue warning messages when the system states are over the pre-defined levels.
3. Another possible extension is to equip the NLME system with simulation capability. A set of primitive simulation functions can be constructed. Based on the information stored in the Working Memory and simulation functions, simulation experiments can be performed without the assistance of external simulation systems.

4. One of the potential usages of the NLME is the system can be alternated to be an instructional program. The program can be applied to teach students how to interpret descriptions in terms of modeling concepts and how to formulate simulation models.
5. The strategy of the Language Processor can further be applied to other applications of manufacturing systems. For instance, it can be used in control systems which allow system controllers to communicate with computers in terms of natural language, not just a limited set of command. Furthermore, the strategy can also be applied to the area of robot programming, which allows robots to be programmed in general English statements.

REFERENCES

Amsterdam, J. 1986. "Augmented Transition Networks for Natural Language Parsing", Premiser Issue, pp. 15-21.

Balci, O. 1986. "Requirements for Model Development Environments", Computer And Operation Research, Vol. 13, No. 1, pp. 53-67.

Baron, D. E. 1974. Case Grammar and Diachranic English Syntax, Mouton, the Hayue, Paris,
p. 1

Barr,A. & Feigenbaum, E. A. (Eds.). 1981. The Handbook of Artificial Intelligence, Vol. I, William Kaufmann, Inc., Los Altos, Cal.

Bobrow, D. G., Kaplan, R. M., Kay, M., Norman, D. A., Thomspon, H. & Winograd, T. 1977. "GUS, a frame-driven dialog system", Artificial Intelligence, Vol. 8, pp. 155-173.

Bolc, L. (Ed.). 1983 The Design of Interpreters, Compilers, and Editors for Augmented Transition Networks, Springer-Verlag, New York, NY.

Bonnet, A. 1985. Artificial Intelligence : promise and performance, Prentice-Hall International, UK., Limited, London. pp. 120-127.

Brachman, R. J. 1983. "What IS-A is and isn't : an analysis of taxonomic links in semantic networks", IEEE Transaction on Computer, Vol. C-32, pp. 30-36.

Bruce, B. 1975. "Case systems for natural language", Artificial Intelligence, Vol. 6, pp. 327-360.

Buller, Jr. W. I. & Schultz, C. R. 1986. "Production rule-based simulation for job shop scheduling", The proceeding of the 1986 Summer Computer Simulation Conference, pp. 718-723, Reno, Nevada.

Cater, A. 1983. "Request-based parsing with low-level syntactic recognition", In Janes, K. S. & Wilks, Y. (Eds.). 1983. Automatic Natural Language Parsing, Ellis Harwood Limited, England.

Celce, M. 1972. "Paradigms for Sentence Recognition", in SDC Final Report HRT-15092/7907.

Charniak, E., C. Riesbeck & D. McDermott. 1980. Artificial Intelligence Programming, Lawrence Erlbaum Associates, Hillsdale, NJ.

Charniak, E. & McDermott, D. 1985. Introduction to Artificial Intelligence. Addison-Wesley Publishing Company, Reading, Mass. pp. 591-600.

Christaller, T. 1983. "An ATN Programming Environment", in Bolc [1983], pp. 71-148.

Christy, D. P. & Watson, H. J. 1983. "The application of simulation : a survey of industrial practice", Interface, Vol. 13, No. 5, pp. 47-52.

Crite, G. D., Mills, R. I. & Talavage, T. J. 1985. "PATHSIM, a modular simulation for an automatic tool handling system evaluation in FMS", Journal of Manufacturing Systems, Vol. 4, No. 1., pp.15-27.

Fikes, R. & Kehler, F. 1985. "The role of frame-based representation in reasoning", Communication of the ACM, Vol. 18, No. 9, pp. 904-920.

Fillmore, C. J. 1971. "Types of lexical information", In Steinberg, D. & Jokobovits, L. (Eds.). Semantics, Cambridge University Press, Cambridge, England. pp. 370-392.

Finin, T., McAdams, J. & Kleinosky, P. 1984. "FOREST " an expert system for automatic test equipment", Proceedings of IEEE Computer Society Conference on AI Applications, pp. 350-357.

Finin, T. 1986a. "Understanding Frame Languages: Part I", AI Expert, Vol. 1, No. 3, pp. 44-51.

Finin, T. 1986b. "Understanding Frame Languages: Part II", AI Expert, Vol. 1, No. 4, pp. 51-56.

Fox, M. S. & Smith, S. F. 1984. "ISIS - a knowledge-based system for factory scheduling", Expert System, Vol. 1, No. 1, pp 25-49.

Gershman, A. V. 1979. Knowledge-Based Parsing, Computer Science Research Report No. 156, Yale University, New Haven, CT.

Hayes-Roth, F. 1985. "Rule-based systems", Communication of the ACM, Vol. 28, No. 9, pp. 921-932.

Heidorn, G. E. 1973. "An interactive simulation programming system which converses in English", Proceeding 1973 winter Simulation Conference, San Fransisco, Cal. pp. 781-793.

Heidorn, G. E. 1974. "English as a very high level language for simulation programming", Proceeding of Symposium on Very High Level Language, SIGPLAN Notice, Vol. 9, pp. 91-100.

Hendrix, G. G. 1977. "Human engineering for applied natural language processing", Proceeding of the International Joint Conference on Artificial Intelligence, MIT, Cambridge, Mass.

Jackson, P. 1986. Introduction to Expert System, Addison-Wesley Publishing Company, Inc., Wokingham, England. pp. 52-68.

Klahr, P. & Faught, W. S. 1981. "Knowledge-based simulation", Proceeding of the First Annual Conference of the American Association for Artificial Intelligence, Standford, CA. pp. 181-183.

Martin, W. A. 1973. " The things that really matter- A theory of prepositions, semantic cases, and semantic type checking", Automatic programming Group, Internal Memo 13, MIT Project MAC, Massachusetts Institute of Technology, Cambridge, Mass.

Medeiros, D. J., Sadowski, R. P., Starks, D. W. & Smith, B. S. 1980. "A modular approach to simulation of robotic systems", Proceedings of the 1980 Winter Simulation Conference, pp. 207-214.

Miner, R. J. & Rolston, L. J. 1986. MAP/1 User's Manual Version 3.0, Prisker & Associates, Inc., West Lafayette, In.

Minsky, M. 1975. "A framework for representing knowledge", In Winston, P. H. (Ed.). The Psychology of Computer Vision, McGraw-Hill Book Company, New York, NY. pp. 211-277.

Nau, D. S. & Chang, T. 1983. "Prospects for process selection using Artificial Intelligence", Computer in Industry, Vol. 4, pp. 253-263.

Nielsen, N. R. 1986. "Knowledge-based simulation programming", National Computer Conference, Vol. 55, pp. 125-134, Las Vegas, Nevada.

O'Hare, G. H. & Bell, D. A. 1985. "The coexistence approach to knowledge representation", Expert System, Vol.2 No. 4, pp. 230-237.

Paul, R. J. & Doukidis, G. I. 1986. "Further developments in the use of Artificial Intelligence techniques which formulate simulation problems", Journal of Operational Society, Vol. 37, No. 8, pp. 787-810.

Pegden, C. D. 1982. Introduction to SIMAN, System Modeling Corporation, State College, Pen.

Prisker, A. A. & Pegden, C. D. 1979. Introduction to Simulation and SLAM, John Wiley & Sons, NY. p. 65.

Reddy, Y. V., Fox, M. S., Doyle, K. & Arnold, J. 1983. "INET : a knowledge based simulation model of a corporate distribution system", Proceeding of IEEE Conference on Trends and Applications, pp. 109-118, Gaithersburg, Ml.

Ruiz-Mier, S., Talavage, J. & Ben-Arieh, D. 1985. "Towards a knowledge-based network simulation environment", Proceeding of the 1985 Winter Simulation Conference, pp. 232-234.

Sathi, N., Fox, M., Baskaran, V. & Bouer, J. 1986. "An artificial intelligence approach to the simulation life cycle", The proceedings of the 1986 Summer Computer Simulation Conference, pp. 773-778, Reno, Nevada.

Schank, R. C. & Abelson, R. P. 1977. Scripts, Plans, Goals, and Understanding. Lawrence Erlbaum Associates, Publishers, Hillsdale, NJ.

Schank, R. C. & Riesbeck, C. K. 1981. Inside Computer Understanding. Lawrence Erlbaum Associates, Publishers, Hillsdale, NJ. p. 14.

Simmons, R. F. 1973. "Semantic networks : their computation and use for understanding English sentences", In Schank, R. C. & Colby, K. M. (Eds.). Computer Models of Thought and Language. W. H. Freeman & Company, San Fransisco, Cal. pp. 63-113.

Stefik, M. 1979. "An examination of a frame-structured representation system", Proceeding of the Sixth International Conference on Artificial Intelligence, Tokyo, Japan. pp. 845-852.

Waltz, D. L. 1982. "The state of the art in natural-language understanding", In Lehnert, W. G. & Ringle, M. H. (Eds.). Strategies for Natural Language Processing. Lawrence Erlbaum Associates, Publishers, Hillsdale, NJ.

Winograd, T. 1972. Understanding Natural Language. Academic Press, New York, NY.

Winston, P. H. & B. K. Horn. 1984. Lisp. Second Edition, Addison-Wesley Publishing Company, Readings, Mass. Chapter 22, pp. 311-320.

Woods, W. A. 1970. "Transition Network grammars for natural language analysis", Communication of the ACM. Vol. 13, pp 591-606.

Woods, W. A., Kaplan, R. & Nash-Webber, B. 1972. "The LUNAR sciences natural language information system : Final Report", BBN Rep. No. 2378, Bolt Beranek and Newman, Inc., Cambridge, Mass.

APPENDIX I

THE DEFINITIONS OF THE AUGMENTED TRANSITION NETWORKS GIVEN IN FIGURE 17

In this appendix, the definitions of the augmented transition networks in Figure 17 are presented. Following the definitions is the current contents of the Lexicon and the contextual rules used to support the ATN parser. Finally, the rules used to store the semantic representations generated from the Language Processor into the Working Memory are provided.

1.1 SENTENCE STRUCTURE DEFINITIONS

```
;
;
;          ********* SENTENCE NETWORK ********
;
(defnode conj:ss
      (push sentence t ss:ms nil
          (addr ss *)
           (addr modR (cons 'ss (form.cd 'ss '(conj ss))))]
;
;
(defnode ss:ms
      (wrd \, t sentence)]
;
;
(defnode sentence
      (cat conj t conj:ss (setr conj *))
      (cat adv t adv:s  (setr adv *))
      (wrd there t s:subj (setr subj *))
      (cat aux t s:aux (setr v *))
      (push np t  s:subj nil (addr subj *))]
;
;
(defnode s:aux
      (push np t s:subj nil (addr subj * ]
;
;
(defnode s:subj
      (root be (not (catf v *>)) subj:be (setr v *))
      (cat adv t s:subj (setr adv *))
      (root (have be) (catf (adv v) *>) s:subj
          (cond ((eq (getf root) 'be)
                    (hold (car ^subj) np)(setr subj '<someone>))
                ((eq (getf root) 'have)
                  (setr aspect '<perfect>))) )
      (cat v t s:v (setr v *))
      (jump subj:be t]
;
```

```
;
(defnode subj:be
      (push np t s:end nil (addr adjunt *))
      (cat adv t subj:be (setr adv *))
      (cat adj t s:end (setr adj *))
      (wrd to (catf v *>) be:to (hold (car ^subj) np))
      (push pp t s:end nil (addr adjunt *))]
;
;
(defnode s:v
      (push np (catf vt ^v) s:obj
         (list (cons 'indobj (catf indobj ^v)))
         (addr obj *))
      (wrd to (catf vinf ^v) v:to )
      (jump s:end (catf vi ^v)]
;
;
(defnode v:to
      (push s:subj t s:end (list (cons 'subj ^subj)) (addr obj *)]
;
;
(defnode be:to
      (push s:subj t s:end3 (list (cons 'subj ^subj))(addr s *)]
;
;
(defnode s:obj
      (wrd (to for) (catf indobj ^v) obj:indobj)
      (push np (catf indobj ^v) s:end  nil
         (setr indobj ^obj)
         (addr obj *))
      (wrd to t obj:to)
      (jump s:end t]
;
;
(defnode obj:indobj
      (push np (nullr indobj) s:end nil (addr indobj *]
;
;
(defnode obj:to
      (push s:subj t s:end
          (list (cons 'subj ^obj))
          (addr obj *]
;
;
(defnode s:by
      (push np t s:end nil (addr subj *]
;
;
(defnode s:end
      (wrd by t s:by)
      (push pp t s:end nil (addr modR *))
```

```
      (push np t s:end nil (addr modR (cons 'time/loc *)))
      (pop (build.frame) t]
;
;
(defnode adv:s
   (push sentence t s:end2 (list (cons 'type 'ss)) (addr s *))]
;
;
(defnode s:end2
   (pop (form.cd 's '(adv s)) t)]
;
;
(defnode s:end3
   (pop (car ^s) t)]
;
;    build.frame is used to construct a semantic representation
;
(defun build.frame ()
   (let ((type (or ^type 's)))
      (caseq (getf root ^v)
         (be (form.cd type '(subj v adv adj adjunt)))
         (t  (form.cd type '(subj adv v aspect obj indobj) ^modR)))))]


;
;
;
;         ********** NOUN PHRASE NETWORK **********
;
;
(defnode np:
      (cat quant t np (setr quant *))
      (cat det t np:det (setr det *)
                  (if (equal 'def (lookup * 'art)) (setr defp t)))
      (cat propnoun t np:n (setr n *))
      (cat pronoun t np:n (setr n * ))
      (jump np:det t]
;
;
(defnode np:det
      (cat adj t np:det (addr adj *))
      (cat propnoun t np:det (addr adj *))
      (cat pp t np:det (addr adj *))
      (cat presp t np:det (addr adj *))
      (cat n t np:det (addr adj *))
      (test (numberp *) np:n (addr conceptR `(number (figure ,*))))
      (cat n t np:n (setr n *))
      (cat propnoun t np:n (setr n *))]
;
;
```

```
;   The semantic approach is used to deal with the NP + PP or
;   NP + PP + PP problems . (valid.pp )
;
(defnode np:n
      (jump np:end (getr indobj))
      (wrd of t np:of)
      (wrd to (catf v *>) np:to)
      (push pp (valid.pp * ^n) np:n nil (addr modR *))
      (jump np:end t)]
;
;
(defnode np:of
      (push pp:prep t np:end (list (cons 'prep 'of)) (addr adj (cdr *))]
;
;
(defnode np:to
      (push s:subj t np:end
            (list (list 'subj  (form.cd 'nph '(n))) (cons 'type 'ss))
           (addr adj *)]
;
;     when the structure is pp + np, the (if ^pp (list ...)) is
;     used to send the noun of np to the pp level. The purpose is to
;     allow features check.
;
(defnode np:end
      (push pp (valid.pp * ^n) np:end nil (addr modR *))
      (pop (resolv.det (form.cd 'nph '(quant det nbr adj n) ^modR ^conceptR)
                  ^defp)
           t  (if ^pp (list (cons 'n ^n))))]
;
;     valid.pp : to check if wrd can be modify by fs
;       Input : fs (features) = (s (f1 r1..)(f2 ..))
;               wrd (word)
;
(defun valid.pp (fs wrd)
   (let ((headers (foreach f in (cdr fs)(car f)))
         (mods (lookup wrd 'modifiers)))
        (or (intersection headers mods)
            (and (intersection '(station part) mods)
                 (member '<check> headers)))))]
;
;    resolv.det is to resolve the referential ambiguity
;
(defun resolv.det (cds def)
   (if def
      (foreach* cd in cds
         (cond ((null cd) nil)
               ((and (member (car cd) '(part station MH))
                     (null (assoc 'description (cdr cd))))
                (and *trace3*
                     (progn
```

```
                (format t
            "  >>>>>> Search Short Term Memory for : ~s~%" (car cd))
                (ptrace '|Contents of Short Term Memory : |
                    *ShorTerMem*)))
            (list (append cd (list (description (car cd))))))
           (t (list cd))))
     cds )]
;
;
(defun description (object)
  (assoc 'description (cdr (assoc object *ShorTerMem*)))]


;
;
;
;        ********** PREPROSITION NETWORK **********
;
;
(defnode pp
      (cat prep t pp:prep (setr prep *]
;
;     The (cons 'pp t) is used to signal the np that the noun phrase is
;     used for preprosition phrase. (check np:end)
;
(defnode pp:prep
      (push np t pp:end (list (cons 'pp t)) (addr np *))]
;
;
(defnode pp:conj1
      (push np t  pp:end nil (addr np2 *))]
;
;
(defnode pp:conj2
      (push np t pp:end nil
          (setr np (list *))
          (addr cd (form.cd 'ph '(prep np))))]

;
;
(defnode pp:end
      (cat conj (catf prep.conj ^prep) pp:conj1 (setr conj *))
      (cat conj (not (equal ^prep 'of)) pp:conj2
         (setr conj *)
          (addr cd (form.cd 'ph '(prep np))))
      (pop (cons ^prep (apply 'append ^cd)) ^cd)
      (pop (cons ^prep (form.cd 'ph '(prep np))) (nullr np2))
      (pop (cons ^prep (form.cd 'ph '(prep np np2)))
         (catf prep.conj ^prep))]
```

1.2. LEXOCON

```
;
;
;
;  File : atn.d/lexicon
;  Date : 6/10/87
;
;
;  <----- Part of Speech : Determiner ----->
;
;
(word the (grammar (det))(art def)(number (s pl))]
;
(word a (grammar (det))(art indef)(number (s))]
;
;
;  <----- Part of Speech : Quantifier ----->
;
;
(word every (grammar (quant))(feature (function))
        (quant (((feature time.unit n)'((function <constant>)))
                ('((quantifier <every>))]
;
;
;  <----- Part of Speech : Adjective ----->
;
;
(word new (grammar (adj))(adj (('((condition new))))))]
;
(word full (grammar (adj)) (feature (state))
     (adj (('((state <full>))))))]
;
(word busy (grammar (adj)) (feature (state))
     (adj (('((state <busy>))))))]
;
(word idle  (grammar (adj)) (feature (state))
     (adj (('((state <idle>))))))]
;
(word available (root idle)]
;
;
;  <-----  Part of Speech : Adverb ----->
;
;
(word there (grammar (adv))(feature (location exit))]
;
(word then (grammar (adv)) (adv ((!s nil)))]
;
(word next (grammar (adv)) (adv ((!s nil)))]
;
```

```
(word also (grammar (adv)) (adv ((!s nil)))]
;
(word first (grammar (adv)) (adv ((!s nil)))]
;
(word otherwise (grammar (adv))
      (adv ((!(option ?(event nil s)))))]
;
(word exponentially (grammar (adv))(feature (function))
      (adv (('((function <exponential>)))))]
;
(word normally (grammar (adv))(feature (function))
     (adv (('((function <normal>)))))]
;
(word uniformly (grammar (adv))(feature (function))
     (adv (('((function <uniform>)))))]
;
;
; <----- Part of Speech : Noun ----->
;
;
(word minutes (grammar (n)) (feature (time.unit))(number (pl))
         (n (((feature function quant)
            !(distribution ?(function nil quant)
                           (number ?(figure nil nbr)
                                   (unit <minute>))) nil)
           ((and (nullr qunat) ^nbr)
            !(number ?(figure nil nbr)
                     (unit <minute>)) nil)))]
;
(word minute (grammar (n)) (feature (time.unit))(number (s))
         (n ((!(number ?(figure (figure 1) nbr)
                       (unit <minute>)) nil)))]
;
(word service (grammar (n))(number (s))(feature (event operation))
     (adj (('((event operation))))]
;
(word transportation (grammar (n))(number (s))(feature (event transfer))
     (adj (('((event transfer))))]
;
(word time (grammar (n))(number (s))(feature (attribute))
       (modifiers (event station part distribution from to))
       (n (((feature operation adj)
           !(attribute (type duration)
                       ?(event nil adj)
                       ?(station nil adj at)
                       ?(part nil adj)
                        (duration ?(distribution delay)
                                  ?(number delay))) nil)
          ((feature transfer adj)
           !(attribute (type duration)
                       ?(event nil adj)
```

```
                    ?(part nil adj)
                    ?(from delay from)
                    ?(to delay to)
                     (duration ?(distribution delay)
                               ?(number delay))) nil)
          ((feature arrival adj)
           !(attribute (type duration)
                       ?(event nil adj)
                       ?(part nil adj)
                       ?(to nil to)
                        (duration ?(distribution delay)
                                  ?(number delay))) nil)
          (!(attribute (type duration)
                       ?(event nil adj)
                       ?(part nil adj)
                        (duration ?(distribution delay)
                                  ?(number delay))) nil)
))]
;
(word mean (grammar (n)) (modifiers (number))
       (n ((!(mean ?(number)) nil)))]
;
(word variance (grammar (n)) (modifier (number))
       (n ((!(variance ?(number)) nil)))]
;
(word items (grammar (n))(number (pl))(feature (part unit))
        (n ((^nbr !(number ?(figure nil nbr)
                           (unit <part>)) nil)
           (!(part (description ?(name)
                                ?(type)
                                ?(property)
                                ?(function))
                   ?(condition)) nil))]
;
(word item (grammar (n))(number (s))(feature (part unit))
        (n ((^nbr !(number ?(figure (figure 1) nbr)
                           (unit <part>)) nil)
           (!(part (description ?(name)
                                ?(type)
                                ?(property)
                                ?(function))
                   ?(condition)) nil))]
;
(word part (root item)]
;
(word parts (root items)]
;
(word work (grammar (n v vi))(number (s))
      (phrases ((work center)(work station)))]
;
```

```
(word station (grammar (n))(number (s))(feature (station))
      (n ((!(station (description  ?(type)
                                   ?(name)
                                   ?(function))
                  ?(state delay)) nil)))]
;
(word machine (grammar (n))(number (s))(feature (station))
      (modifiers (station)) (n machines)]
;
(word machines (grammar (n))(number (pl))(feature (machine station unit))
      (modifiers (station))
      (n (( ^nbr !(attribute (type machine-no)
                   ?(station delay at in subj)
                    (value (number ?(figure)
                                   (unit <machine>)))) nil)
        (!(machine (description ?(name)
                                ?(type)
                                ?(function))) nil)))]
;
(word system (grammar (n))(number (s))(feature (system))
      (n ((!(system ?(name nil adj)
                  ?(type nil adj)) nil)))]
;
(word pin (grammar (n v vt))
      (adj (('((type pin)))))]
;
(word standard (grammar (n))(phrases ((standard deviation)))]
;
;(word space (grammar (n))(feature (queue))(modifiers (number station))
;       (n ((!(type queue)
;            !?(station nil at in)
;            !?(number nil for) nil)))]
;
;
(word queue (grammar (n)) (feature (attribute buffer))
      (n ((!(attribute (type queue)
                  ?(station)
                  ?(number)
                  ?(state delay)) nil)))]
;
(word buffer (grammar (n)) (feature (attribute buffer))
      (modifiers (station number))
      (n ((!(attribute (type queue)
                  ?(station delay at in subj adj)
                   (value ?(number))
                  ?(state delay)) nil)))]
;
(word space (root buffer)]
;
(word arrival (grammar (n))(feature (event arrival))
      (adj (('((event arrival)))))]
```

```
;
(word steel (grammar (n))(feature (property))
     (adj (('((property steel)))))]
;
(word aluminum (grammar (n))(feature (property))
     (adj (('((property aluminun)))))]
;
(word nickle (grammar (n))(feature (property))
     (adj (('((property nickle)))))]
;
(word adjustment (grammar (n))(feature (function))
     (adj (('((function adjustment)))))]
;
;
;  <----- Part of Speech : Proper Nnoun ----->
;
;
;
(word pcb (grammar (propnoun))(number (s))(feature (part))
       (n ((!(part (description (name pcb)))
          nil)))]
;
(word AI (grammar (propnoun))
       (adj (('((name AI)))))]
;
(word CIM (grammar (propnoun))
       (adj (('((name CIM)))))]
;
(word DIP (grammar (propnoun))
       (adj (('((name DIP)))))]
;
(word Poisson (grammar (propnoun))(feature (function))
       (adj (('((function <poisson>)))))]
;
(word VCD (grammar (propnoun))
     (adj (('((name VCD)))))]
;
(word TDK (grammar (propnoun))
     (adj (('((name TDK)))))]
;
;
(word DOMAIN (grammar (propnoun))
     (adj (('((name DOMAIN)))))]
;
;
(word FRANZ (grammar (propnoun))
     (adj (('((name FRANZ)))))]
;
;
(word APOLLO (grammar (propnoun))
     (adj (('((name APOLLO)))))]
```

```
;
;
(word Berg (grammar (propnoun))
     (adj (('((name Berg)))))]
;
;
;  <----- Part of Speech : Pronoun ----->
;
;
(word it (grammar (pronoun))(number (s))
      (n ((!(<check> STM) nil))))]
;
;
;  <----- Part of Speech : Verb ----->
;
;
(word be (grammar (v)) (v (((equal ^subj 'there) !adjunt nil)
                  (!subj nil))))]
;
(word is (root be)(number (s)))]
;
(word are (root be)(number (pl)))]
;
(word have (grammar (v vt)) (number (pl))
      (v ((!obj nil))))]
;
(word has (root have)(number (s)))
;
(word distributed (grammar (v vt past pp))
            (v (((feature function adv)
                !obj '((distribution ?(function)
                                     ?(mean)
                                     ?(stdev)
                                     ?(variance)
                                     ?(from nil from between)
                                     ?(to nil to between)
                                  ))))))]
;
(word transfer (grammar (n v vt past pp))(number (pl))
            (feature (event transfer))
            (adj (('((event transfer)))))
            (v ((!(event (type transfer)
                         ?(part nil obj)
                         ?(from nil from)
                         ?(to nil to)
                         ?(condition)) nil))))]
;
(word transferred (grammar (v vt past pp))(root transfer)]
;
(word move (grammar (v vt))(root transfer)]
;
```

```
(word moved (root transferred))
;
(word goes (grammar (v vi)) (root go) (number (s))
     (v ((!(event (type transfer)
                ?(part nil subj)
                ?(from nil from)
                ?(to   nil to)
                 (duration ?(distribution)
                           ?(number nil for))
              ?(condition)) nil)))]
;
(word provided (grammar (v vt past pp))(feature (event))
          (v ((!(event (type arrival)
                     ?(part nil obj)
                     ?(to nil to)
                      (duration ?(distribution nil time/loc))) nil)))]
;
(word enters (grammar (v vt)) (root enter)(number (s))
       (v ((!(event (type arrival)
                  ?(part nil subj)
                  ?(system nil obj)) nil)))]
;
(word arrives (grammar (v vi))(number (s))(feature (event))
     (v ((!(event (type arrival)
                ?(part nil subj)
                ?(system)
                (to ?(station nil time/loc at))
                (duration ?(distribution nil time/loc))) nil)))]
;
(word process (grammar (n v vt)) (number (pl))
     (feature (event distribution operation))
     (modifiers (mean stdev variance))
     (n (((feature function adj)
        !(distribution ?(function nil adj)
                     ?(mean)
                     ?(stdev)
                     ?(variance)) nil)))
     (adj (('((event operation)))))
     (v ((!(event (type operation)
                ?(part nil obj)
                ?(station nil at in subj)
                 (duration ?(distribution)
                           ?(number nil for)))nil)))]

;
(word processes (grammar (v vt))(root process)(number (s))(feature (event))]
;
(word processed (grammar (v vt past pp))(root process)
     (number (s pl)) (feature (event))]
;
```

```
(word exit (grammar (v vi))(number (pl))
      (v ((!(event (type transfer)
                    ?(part nil subj)
                    ?(system nil time/loc)
                     (to (station <exit>))) nil)))]
;
(word exits (root exit) (number (s))]
;
(word leave  (grammar (v vi vt))(number (pl))
      (v ((!(event (type transfer)
                    ?(part nil subj)
                    ?(system nil obj)
                     (to (station <exit>))) nil)))]
;
(word leaves (root leave) (number (s))]
;
(word simulate (grammar (v vt))(number (pl))(feature (command))
      (v ((!(command (type simulate)
                          ?(system nil obj)
                           (duration ?(number nil for))))))]
;
(word simulated (grammar (v vt past pp))(root simulate)]
;
;
;  <----- Part of Speech : Past and Present Participle ----->
;
;
;
(word cast (grammar (v past pp)) (feature (property))
      (adj (('((property cast)))))]
;
(word milling (grammar (presp)) (feature (function))
      (adj (('((function milling)))))]
;
(word drilling (grammar (presp)) (feature (function))
      (adj (('((function drilling)))))]
;
(word polishing (grammar (presp)) (feature (function))
      (adj (('((function polishing)))))]
;
(word turning (grammar (presp)) (feature (function))
      (adj (('((function turning)))))]
;
(word welding (grammar (presp)) (feature (function))
      (adj (('((function welding)))))]
;
(word coating (grammar (presp)) (feature (function))
      (adj (('((function coating)))))]
;
(word painting (grammar (presp)) (feature (function))
      (adj (('((function painting)))))]
```

```
;
(word shearing (grammar (presp)) (feature (function))
      (adj ((((function shearing)))))]
;
(word washing (grammar (presp)) (feature (function))
      (adj ((((function washing)))))]
;
(word waiting (grammar (presp)) (feature (event queue))]
;
;
;  <----- Part of Speech : Preposition ----->
;
;
(word of (grammar (prep))(prep ((!np nil)))]
;
(word with (grammar (prep))(prep ((!np nil)))]
;
(word for (grammar (prep))(prep ((!np nil)))]
;
(word at (grammar (prep)) (prep ((!np nil)))]
;
(word in (grammar (prep)) (prep ((!np nil)))]
;
(word from (grammar (prep))(prep (((!(from ?(station nil np)
                              ?(number nil np)) nil)))]
;
(word to   (grammar (prep))(prep (((!(to ?(station nil np)
                                        ?(number nil np)) nil)))]
;
(word on (grammar (prep))(prep ((!np nil)))]
;
(word according (grammar (prep conj))
       (phrases ((according to)(according as)))]
;
(word between (grammar (prep prep.conj))
          (prep (((feature number np)
                !(from ?(number nil np))
                !(to   ?(number nil np2)) nil)
               ((feature station np)
                !(between (station1 ?(station nil np))
                          (station2 ?(station nil np2))) nil)))]
;
;
;  <---- Part of Speech : Conjunction ----->
;
;
(word and (grammar (conj))]
;
(word if (grammar (conj))
      (conj (((!(condition ?(attribute)
                           ?(station))nil))))
```

```
;
;
;  <----- Part of Speech : Phrase ----->
;
;
(word work-center (grammar (n))
      (n ((!(system ?(name nil adj)
                ?(det)) nil))))]
;
(word according-to (grammar (prep))
      (prep ((!np nil)))]
;
(word standard-deviation (grammar (n))
     (n ((!(stdev ?(number nil adj)))))]
```

1.3. CONTEXTUAL RULES

```
;
;       File : atn.d/context.rule
;       Date : 7/2/87
;       Description : Rules used by function is.complete to perform
;             context check.
;       Format : (<Context.Rules> ID
;                 (if
;                    ( RHS1
;                      RHS2
;                      . . . )
;                  then
;                     LHS)]
;
(<Context.Rules> operation
  (if ((event (type transfer)
              (part ?part)
              (from (station ?station0))
              (to   (station ?station1)))

       (event (type arrival)
              (part ?part)
             (to (station ?station1))))

  then (event (type operation)
              (part ?part)
              (station ?station1)))]
;
;
(<Context.Rules> transfer
  (if ((event (type transfer)
              (part ?part)
              (from (station ?station0))
              (to   (station ?station1)))

       (event (type operation)
              (part ?part)
              (station ?station1))

       (event (type arrival)
              (part ?part)
              (to (station ?station1))))

  then (event (type transfer)
              (part ?part)
              (from (station ?station1))
              (to   (station ?station2)))))]
```

1.4. STORE RULES

```
;    File : frame.d/kb.rules
;    Date : 4/27/87
;
;    Description : This file contaions rules used to store frames from
;                  NLP into to WM.  The interpreter functions is
;                  stored in file kb.fun.
;
;    Format : (<Store.Rules> ID
;               (if patten1
;                  then action1 action2)
;               (if patten2
;                  then action3 action4 ..))
;
;
(<Store.Rules> arrival
   (if (event (type arrival)
              (system <null>)
              (part ?part)
              (to (station ?station))
              (duration ?duration))
  then (<store> (<event> (type arrival)
                         (part ?part))
                (<value> (to (to (station ?station)))
                         (interval ?duration))))

   (if (event (type arrival)
              (part ?part)
              (system ?description)
              (to <null>)
              (duration ?duration))
  then (<store> (<event> (type arrival)
                         (part ?part))
                (<value> (interval ?duration)))
       (<store> (<object> (system))
                (<value> (system ?description)))))]
;
;
(<Store.Rules> operation
   (if (event (type operation)
              (part ?part)
              (station ?station)
              (duration ?duration))
   then (<store> (<event> (type operation)
                          (part ?part)
                          (station ?station))
                 (<value> (op-time ?duration)))))]
;
;
```

```
(<Store.Rules> transfer
  (if (event (type transfer)
             (condition <null>)
             (duration <null>)
             (part ?part)
             (from (station ?station1))
             (to  (station ?station2)))
  then (<store> (<event> (type transfer)
                         (part ?part)
                         (station ?station1))
                (<value> (to  (to (station ?station2))))))

  (if (event (type transfer)
             (condition <null>)
             (part ?part)
             (from (station ?station1))
             (to  (station ?station2))
             (duration ?duration))
  then (<store> (<event> (type transfer)
                         (part ?part)
                         (station ?station1))
                (<value> (to  (to (station ?station2)))
                         (tf-time (to (station ?station2)
                         (duration ?duration))))))

  (if (event (type transfer)
             (condition (station <null>)
                        (attribute (type queue)
                        (station ?station3)
                        (state ?state)))
             (part ?part)
             (from (station ?station1))
             (to  (station ?station2)))
  then (<store> (<event> (type transfer)
                         (part ?part)
                         (station ?station1))
                (<value> (to  (to (station ?station2)
                         (condition (station ?station3)
                                    (queue-state ?state)))))))

  (if (event (type transfer)
             (condition (attribute <null>)
                        (station (description ?description)
                        (state ?state)))
             (part ?part)
             (from (station ?station1))
             (to  (station ?station2)))
  then (<store> (<event> (type transfer)
                         (part ?part)
                         (station ?station1))
                (<value> (to  (to (station ?station2)
```

```
                        (condition
                                (station (description ?description))
                                 (station-state ?state)))))))]
;
;
(<Store.Rules> queue
   (if (attribute (type queue)
                  (station ?station)
                  (value ?size))
  then (<store> (<object> (station ?station))
                 (<value> (buffer-size ?size)))))]
;
;
(<Store.Rules> machine-no
   (if (attribute (type machine-no)
                  (station ?station)
                  (value ?size))
  then (<store> (<object> (station ?station))
                 (<value> (machine-no ?size)))))]
;
;
(<Store.Rules> simulate
   (if (command (type simulate)
                 (system ?name)
                 (duration ?duration))
   then (<store> (<object> (system ?name))
                 (<value> (run-length ?duration)))))]
```

APPENDIX II

A CONTINUATION OF THE TRACE GIVEN IN CHAPTER SIX

```
 >>>>>> |Input Sentence : |

(there are 2 machines in the drilling station)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function drilling)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

(there are 2 machines in the drilling station)
-----------------------------------------------------------------

 >>>>>> | Updated Short Term Memory|

((station (description (function drilling)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(attribute (type machine-no)
           (station (description (function drilling)))
           (value (number (figure 2) (unit <machine>))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((attribute (type machine-no)
            (station (description (function drilling)))
            (value (number (figure 2) (unit <machine>)))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<object> (station (description (function drilling))))
           (<value> (machine-no (number (figure 2) (unit <machine>)))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : station2

frame station2
  operations = operation2
  system = (name CIM)
  object-type = station
  machine-no = (number (figure 2) (unit <machine>))
  consist-of = operation2
  description = (description (function drilling))
```

```
 part-of = station
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

 (also the station has a waiting space for 2 parts)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function drilling)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

 (also the station has a waiting space for 2 parts)
-----------------------------------------------------------------

 >>>>>> Search Short Term Memory for : station

 >>>>>> |Contents of Short Term Memory : |

((station (description (function drilling)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(attribute (type queue)
          (station (description (function drilling)))
          (value (number (figure 2) (unit <part>))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((attribute (type queue)
           (station (description (function drilling)))
           (value (number (figure 2) (unit <part>)))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<object> (station (description (function drilling))))
         (<value> (buffer-size (number (figure 2) (unit <part>)))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : station2

frame station2
 operations = operation2
```

```
system = (name CIM)
object-type = station
buffer-size = (number (figure 2) (unit <part>))
machine-no = (number (figure 2) (unit <machine>))
consist-of = operation2
description = (description (function drilling))
part-of = station
```

>>>>>> |Input Sentence : |

(next the cast part is moved to the polishing station)

>>>>>> |Contents of Short Term Memory : |

```
((station (description (function drilling)))
 (part (description (property cast))))
```

>>>>>> |PreProcessor Output : |

(next the cast part is moved to the polishing station)

>>>>>> | Updated Short Term Memory|

```
((station (description (function polishing)))
 (part (description (property cast))))
```

>>>>>> | Updated Short Term Memory|

```
((station (description (function polishing)))
 (part (description (property cast))))
```

>>>>>> |Semantic Structure from Parser|

```
(event (type transfer)
       (part (description (property cast)))
       (to (station (description (function polishing)))))
```

```
 >>>>>> |Context Check|

(event (type operation)
       (part (description (property cast)))
       (station (description (function drilling)))
       (duration
          (distribution (function <normal>)
                        (mean (number (figure 2)))
                        (variance (number (figure 0.5) (unit <minute>))))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((event (type transfer)
        (part (description (property cast)))
        (from (station (description (function drilling))))
        (to (station (description (function polishing))))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<event> (type transfer)
                  (part (description (property cast)))
                  (station (description (function drilling))))
         (<value> (to (to (station (description (function polishing)))))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : transfer2

frame transfer2
 from = station2
 part = part1
 system = (name CIM)
 consist-of = transfer1 transfer2
 buffer-size = (number (figure 2) (unit <part>))
 machine-no = (number (figure 2) (unit <machine>))
 description = (description (property cast))
 object-type = transfer
 to = (station3)
 part-of = transfer part1 station2
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

(the transfer time is exponentially distributed with a mean of 2 minutes)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
```

```
---------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

(the transfer time is exponentially distributed with a mean of 2 minutes)
---------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(attribute (type duration)
          (event transfer)
          (duration
              (distribution (function <exponential>)
                            (mean (number (figure 2) (unit <minute>))))))
---------------------------------------------------------------

 >>>>>> |Context Check|

(event (type transfer)
       (part (description (property cast)))
       (from (station (description (function drilling))))
       (to (station (description (function polishing)))))
---------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((event (type transfer)
        (part (description (property cast)))
        (from (station (description (function drilling))))
        (to (station (description (function polishing))))
        (duration
            (distribution (function <exponential>)
                          (mean (number (figure 2) (unit <minute>)))))))
---------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<event> (type transfer)
                  (part (description (property cast)))
                  (station (description (function drilling))))
         (<value> (to (to (station (description (function polishing)))))
                  (tf-time
                      (to (station (description (function polishing)))
                  (duration
                     (distribution (function <exponential>)
                                   (mean
                                       (number (figure 2)
                                       (unit <minute>)))))))))
---------------------------------------------------------------
```

```
 >>>>>> Contents of Frame : transfer2

frame transfer2
 from = station2
 part = part1
 system = (name CIM)
 consist-of = transfer1 transfer2
 buffer-size = (number (figure 2) (unit <part>))
 machine-no = (number (figure 2) (unit <machine>))
 description = (description (property cast))
 object-type = transfer
 tf-time = (station3 (distribution (function <exponential>)
                                   (mean (number (figure 2) (unit <minute>)))))
 to = (station3)
 part-of = transfer part1 station2
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

 (the part is processed for 3 minutes)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

 (the part is processed for 3 minutes)
-----------------------------------------------------------------

 >>>>>> Search Short Term Memory for : part

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(event (type operation)
       (part (description (property cast)))
       (duration (number (figure 3) (unit <minute>))))
-----------------------------------------------------------------
```

```
 >>>>>> |Context Check|

(event (type transfer)
       (part (description (property cast)))
       (from (station (description (function drilling))))
       (to (station (description (function polishing))))
       (duration
           (distribution (function <exponential>)
                         (mean (number (figure 2) (unit <minute>))))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((event (type operation)
        (part (description (property cast)))
        (station (description (function polishing)))
        (duration (number (figure 3) (unit <minute>)))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<event> (type operation)
                  (part (description (property cast)))
                  (station (description (function polishing))))
         (<value> (op-time (number (figure 3) (unit <minute>))))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : operation3

frame operation3
 station = station3
 part = part1
 system = (name CIM)
 consist-of = operation1 operation2 operation3
 description = (description (property cast))
 object-type = operation
 op-time = (number (figure 3) (unit <minute>))
 part-of = operation part1 station3
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

 (then the cast part leaves the CIM work center)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------
```

```
 >>>>>> |PreProcessor Output : |

(then the cast part leaves the CIM work-center)
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(event (type transfer)
       (part (description (property cast)))
       (system (name CIM))
       (to (station <exit>)))
-----------------------------------------------------------------

 >>>>>> |Context Check|

(event (type operation)
       (part (description (property cast)))
       (station (description (function polishing)))
       (duration (number (figure 3) (unit <minute>))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((event (type transfer)
        (part (description (property cast)))
        (from (station (description (function polishing))))
        (to (station <exit>))
        (system (name CIM))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<event> (type transfer)
                  (part (description (property cast)))
                  (station (description (function polishing))))
         (<value> (to (to (station <exit>))))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : transfer3

frame transfer3
 from = station3
 part = part1
 system = (name CIM)
 consist-of = transfer1 transfer2 transfer3
 description = (description (property cast))
 object-type = transfer
 to = (<exit>)
 part-of = transfer part1 station3
-----------------------------------------------------------------
```

```
 >>>>>> |Input Sentence : |

(the polishing station has 1 machine)
------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

(the polishing station has 1 machine)
------------------------------------------------------------

 >>>>>> | Updated Short Term Memory|

((station (description (function polishing)))
 (part (description (property cast))))
------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(attribute (type machine-no)
           (station (description (function polishing)))
           (value (number (figure 1) (unit <machine>))))
------------------------------------------------------------

 >>>>>> |Semantic Structure Generated : |

((attribute (type machine-no)
            (station (description (function polishing)))
            (value (number (figure 1) (unit <machine>)))))
------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<object> (station (description (function polishing))))
           (<value> (machine-no (number (figure 1) (unit <machine>)))))
------------------------------------------------------------

 >>>>>> Contents of Frame : station3

frame station3
  transfers = transfer3
  operations = operation3
  system = (name CIM)
  object-type = station
  machine-no = (number (figure 1) (unit <machine>))
  consist-of = operation3 transfer3
```

```
 description = (description (function polishing))
 part-of = station
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

(also there is a waiting space for 1 part in the station)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

(also there is a waiting space for 1 part in the station)
-----------------------------------------------------------------
 >>>>>> Search Short Term Memory for : station

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> Search Short Term Memory for : station

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------
 >>>>>> Search Short Term Memory for : station

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(attribute (type queue)
          (station (description (function polishing)))
          (value (number (figure 1) (unit <part>))))
-----------------------------------------------------------------
```

```
 >>>>>> |Semantic Structure Generated : |

((attribute (type queue)
            (station (description (function polishing)))
            (value (number (figure 1) (unit <part>)))))
-----------------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<object> (station (description (function polishing))))
          (<value> (buffer-size (number (figure 1) (unit <part>)))))
-----------------------------------------------------------------

 >>>>>> Contents of Frame : station3

frame station3
  transfers = transfer3
  operations = operation3
  system = (name CIM)
  object-type = station
  buffer-size = (number (figure 1) (unit <part>))
  machine-no = (number (figure 1) (unit <machine>))
  consist-of = operation3 transfer3
  description = (description (function polishing))
  part-of = station
-----------------------------------------------------------------

 >>>>>> |Input Sentence : |

 (the CIM work center is to be simulated for 24000 minutes)
-----------------------------------------------------------------

 >>>>>> |Contents of Short Term Memory : |

((station (description (function polishing)))
 (part (description (property cast))))
-----------------------------------------------------------------

 >>>>>> |PreProcessor Output : |

 (the CIM work-center is to be simulated for 24000 minutes)
-----------------------------------------------------------------

 >>>>>> |Semantic Structure from Parser|

(command (type simulate)
            (system (name CIM))
            (duration (number (figure 24000) (unit <minute>))))
-----------------------------------------------------------------
```

```
 >>>>>> |Semantic Structure Generated : |

((command (type simulate)
          (system (name CIM))
          (duration (number (figure 24000) (unit <minute>)))))
-----------------------------------------------------------

 >>>>>> |Function Generated|

(<store> (<object> (system (name CIM)))
         (<value> (run-length (number (figure 24000) (unit <minute>)))))
-----------------------------------------------------------

 >>>>>> Contents of Frame : system

frame system
 consist-of = part MH station operation transfer arrival
 run-length = (number (figure 24000) (unit <minute>))
 system = (name CIM)
 object-type = system
 part-of = factory
-----------------------------------------------------------
```

APPENDIX III

SUMMARY REPORTS, MODEL TEMPLATES AND PROGRAMS CORRESPONDING TO EXPERIMENT PROBLEMS

3.1.1. SUMMARY REPORT OF EXPERIMENT PROBLEM ONE

***** Summary Report For DOMAIN System *****

Simulation Time : 1200 <minute>

A. Jobs Description.

Part : part1

a. Description : (description (property cast))
b. Enter Time : nil
c. Lot Size : nil

B. Facilities Description.

Station : station1

a. Description : (description (function drilling))
b. Buffer Size : (number (figure 3) (unit <part>))
c. Machine Number : (number (figure 3) (unit <machine>))

Station : station2

a. Description : (description (function milling))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 1) (unit <machine>))

Station : station3

a. Description : (description (function turning))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 3) (unit <machine>))

Station : station4

a. Description : (description (function polishing))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

C. Activities Description.

1. Routings.

```
((part1 --->
     ((<<opt.1>> station1)
      --->
      ((<<opt.2>> station2
            (condition station3 (queue-state <full>)))
      --->
        ((<<opt.3>> station4) ---> (<exit>)))
      ((<<opt.2>> station3) ---> (<merge-to> station4)))))
```

2. Arrival Times.

Part	To	Value
part1	station1	(distribution (function <poisson>) (mean (number (figure 3) (unit <minute>))))

3. Process Times.

Part	Station	Value
part1	station3	(distribution (function <exponential>) (mean (number (figure 4) (unit <minute>))))
part1	station4	(distribution (function <uniform>) (from (number (figure 2))) (to (number (figure 3) (unit <minute>))))
part1	station2	(distribution (function <normal>) (mean (number (figure 6))) (stdev (number (figure 0.6) (unit <minute>))))
		(mean (number (figure 6))) (stdev (number (figure 0.6) (unit <minute>))))

4. Transfer Times.

Part	From	To	Value
part1	station1	station2	(number (figure 1) (unit <minute>))
part1	station1	station3	(number (figure 1.2) (unit <minute>))
part1	station3	station4	(number (figure 1.5) (unit <minute>))
part1	station2	station4	(number (figure 1.2) (unit <minute>))
part1	station4	<exit>	nil

3.1.2. MODEL TEMPLATE OF EXPERIMENT PROBLEM ONE

```
(|GEN,NLME,DOMAIN SYSTEM,,1;|)
(|LIMITS,4,3,100;|)
(|;|)
(|NETWORK;|)
(|;|)
(|;      attribute description:|)
(|;|)
(|;        attribute 1 : part arrival time|)
(|;        attribute 2 : part type|)
(|;        attribute 3 : part process time in a station|)
(|;|)
(|;|)
(|;      load part1 into system|)
(|;|)
("PAT1" (CREATE (fslam.val 'arrival1 'interval) (fslam.val 'arrival1 'enter-time) 1
(fslam.val 'arrival1 'lot-size) 1))
(|;|)
(|;      station1|)
(|;|)
(|;|)
("STN1" (GOON 1))
(|;|)
(Q1 (|QUEUE(1)| nil (fslam.val 'station1 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station1 'machine-no) '|)/| '|1|) (fslam.val 'operation1 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/5 (fslam.val2 (cadr (assoc 'station2 (fvalues 'transfer1 'tf-time)))) (concat
'|NNQ(3).GE.| (fslam.val 'station3 'buffer-size)) "STN2"))
(nil (ACT/6 (fslam.val2 (cadr (assoc 'station3 (fvalues 'transfer1 'tf-time)))) || "STN3"))
(|;|)
(|;      station2|)
(|;|)
(|;|)
("STN2" (GOON 1))
(|;|)
(Q2 (|QUEUE(2)| nil (fslam.val 'station2 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station2 'machine-no) '|)/| '|2|) (fslam.val 'operation4 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/7 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer3 'tf-time)))) || "STN4"))
(|;|)
(|;      station3|)
(|;|)
(|;|)
```

```
("STN3" (GOON 1))
(|;|)
(Q3 (|QUEUE(3)| nil (fslam.val 'station3 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station3 'machine-no) '|)/| '|3|) (fslam.val 'operation2 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/8 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer2 'tf-time)))) || "STN4"))
(|;|)
(|;      station4|)
(|;|)
(|;|)
("STN4" (GOON 1))
(|;|)
(Q4 (|QUEUE(4)| nil (fslam.val 'station4 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station4 'machine-no) '|)/| '|4|) (fslam.val 'operation3 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/9 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer4 'tf-time)))) || EXIT))
(|;|)
(|;|)
(EXIT (COLCT |INT(1)| |PART1 IN SYS|))
(nil (TERM))
(nil (ENDNETWORK))
(|INIT,0,1200;|)
(|FIN;|)
```

3.1.3. PROGRAM OF EXPERIMENT PROBLEM ONE

```
GEN,NLME,DOMAIN SYSTEM,,1;
LIMITS,4,3,100;
;
NETWORK;
;
;     attribute description:
;
;       attribute 1 : part arrival time
;       attribute 2 : part type
;       attribute 3 : part process time in a station
;
;
;     load part1 into system
;
PAT1  CREATE,NPSSN(3,1), ,1, ,1;
;
;     station1
;
;
STN1  GOON,1;
;
Q1      QUEUE(1),,3,BLOCK;
        ACT(3)/1,RNORM(5,1.0,1);
        GOON,1;
;
        ACT/5,1,NNQ(3).GE.2,STN2;
        ACT/6,1.2,,STN3;
;
;     station2
;
;
STN2  GOON,1;
;
Q2      QUEUE(2),,1,BLOCK;
        ACT(1)/2,RNORM(6,0.6,1);
        GOON,1;
;
        ACT/7,1.2,,STN4;
```

```
;
;     station3
;
;
STN3  GOON,1;
;
Q3    QUEUE(3),,2,BLOCK;
      ACT(3)/3,EXPON(4,1);
      GOON,1;
;
      ACT/8,1.5,,STN4;
;
;     station4
;
;
STN4  GOON,1;
;
Q4    QUEUE(4),,2,BLOCK;
      ACT(2)/4,UNFRM(2,3,1);
      GOON,1;
;
      ACT/9, ,,EXIT;
;
;
EXIT  COLCT,INT(1),PART1 IN SYS;
      TERM;
      ENDNETWORK;
INIT,0,1200;
FIN;
```

3.2.1. SUMMARY REPORT OF EXPERIMENT PROBLEM TWO

***** Summary Report For FRANZ System *****

Simulation Time : 1200 <minute>

A. Jobs Description.

Part : part1

a. Description : (description (property steel))
b. Enter Time : nil
c. Lot Size : nil

Part : part2

a. Description : (description (property aluminun))
b. Enter Time : nil
c. Lot Size : nil

B. Facilities Description.

Station : station1

a. Description : (description (function turning))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station2

a. Description : (description (function welding))
b. Buffer Size : (number (figure 4) (unit <part>))
c. Machine Number : (number (figure 5) (unit <machine>))

Station : station3

a. Description : (description (function coating))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station4

a. Description : (description (function painting))
b. Buffer Size : (number (figure 3) (unit <part>))
c. Machine Number : (number (figure 3) (unit <machine>))

Station : station5

a. Description : (description (function shearing))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 3) (unit <machine>))

C. Activities Description.

1. Routings.

```
((part1 --->
     ((<<opt.1>> station1)
      --->
      ((<<opt.2>> station2)
       --->
       ((<<opt.3>> station3)
        --->
        ((<<opt.4>> station4) ---> (<exit>))))))
 (part2 --->
     ((<<opt.1>> station5)
      --->
      ((<<opt.2>> station2)
       --->
       ((<<opt.3>> station4) ---> (<exit>))))))
```

2. Arrival Times.

Part	To	Value
part1	station1	(distribution (function <constant>) (number (figure 5) (unit <minute>)))
part2	station5	(distribution (function <poisson>) (mean (number (figure 3) (unit <minute>))))

3. Process Times.

Part	Station	Value
part1	station1	(distribution (function <normal>) (mean (number (figure 7))) (variance (number (figure 1) (unit <minute>))))
part1	station2	(number (figure 5) (unit <minute>))
part1	station3	(distribution (function <uniform>) (from (number (figure 5))) (to (number (figure 6) (unit <minute>))))
part1	station4	(number (figure 3) (unit <minute>))
part2	station5	(number (figure 4) (unit <minute>))

part2 station2 (distribution (function <exponential>)
(mean (number (figure 5.5) (unit <minute>))))
part2 station4 (number (figure 4) (unit <minute>))

4. Transfer Times.

Part	From	To	Value
part1	station1	station2	(number (figure 0.5) (unit <minute>))
part1	station2	station3	(number (figure 1) (unit <minute>))
part1	station3	station4	(number (figure 0.5) (unit <minute>))
part1	station4	<exit>	nil
part2	station5	station2	(number (figure 0.8) (unit <minute>))
part2	station2	station4	(number (figure 1.2) (unit <minute>))
part2	station4	<exit>	nil

3.2.2. MODEL TEMPLATE OF EXPERIMENT PROBLEM TWO

```
(|GEN,NLME,FRANZ SYSTEM,,1;|)
(|LIMITS,5,3,100;|)
(|;|)
(|NETWORK;|)
(|;|)
(|;      attribute description:|)
(|;|)
(|;        attribute 1 : part arrival time|)
(|;        attribute 2 : part type|)
(|;        attribute 3 : part process time in a station|)
(|;|)
(|;|)
(|;      load part1 into system|)
(|;|)
("PAT1" (CREATE (fslam.val 'arrival1 'interval) (fslam.val 'arrival1 'enter-time) 1
(fslam.val 'arrival1 'lot-size) 1))
(nil (ASSIGN |ATRIB(2) = 1|))
(nil (ACT nil nil "STN1"))
(|;|)
(|;      load part2 into system|)
(|;|)
("PAT2" (CREATE (fslam.val 'arrival2 'interval) (fslam.val 'arrival2 'enter-time) 1
(fslam.val 'arrival2 'lot-size) 1))
(nil (ASSIGN |ATRIB(2) = 2|))
(nil (ACT nil nil "STN5"))
(|;|)
(|;      station1|)
(|;|)
(|;|)
("STN1" (GOON 1))
(|;|)
(Q1 (|QUEUE(1)| nil (fslam.val 'station1 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station1 'machine-no) '|)/| '|1|) (fslam.val 'operation1 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/6 (fslam.val2 (cadr (assoc 'station2 (fvalues 'transfer1 'tf-time)))) || "STN2"))
(|;|)
(|;      station2|)
(|;|)
("STN2" (GOON 1))
(nil (ACT/7 nil |ATRIB(2).EQ.1| A1))
(nil (ACT/8 nil |ATRIB(2).EQ.2| A2))
(A1 (ASSIGN (assign.atb 3 (fslam.val 'operation2 'op-time))))
(nil (ACT nil nil Q2))
```

```
(A2 (ASSIGN (assign.atb 3 (fslam.val 'operation6 'op-time))))
(|;|)
(Q2 (|QUEUE(2)| nil (fslam.val 'station2 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station2 'machine-no) '|)/| '|2|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/9 (fslam.val2 (cadr (assoc 'station3 (fvalues 'transfer2 'tf-time))))
|ATRIB(2).EQ.1| "STN3"))
(nil (ACT/10 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer6 'tf-time))))
|ATRIB(2).EQ.2| "STN4"))
(|;|)
(|;      station3|)
(|;|)
(|;|)
("STN3" (GOON 1))
(|;|)
(Q3 (|QUEUE(3)| nil (fslam.val 'station3 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station3 'machine-no) '|)/| '|3|) (fslam.val 'operation3 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/11 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer3 'tf-time)))) || "STN4"))
(|;|)
(|;      station4|)
(|;|)
("STN4" (GOON 1))
(nil (ACT/12 nil |ATRIB(2).EQ.1| A4))
(nil (ACT/13 nil |ATRIB(2).EQ.2| A5))
(A4 (ASSIGN (assign.atb 3 (fslam.val 'operation4 'op-time))))
(nil (ACT nil nil Q4))
(A5 (ASSIGN (assign.atb 3 (fslam.val 'operation7 'op-time))))
(|;|)
(Q4 (|QUEUE(4)| nil (fslam.val 'station4 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station4 'machine-no) '|)/| '|4|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/14 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer4 'tf-time))))
|ATRIB(2).EQ.1| EXIT))
(nil (ACT/15 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer7 'tf-time))))
|ATRIB(2).EQ.2| EXIT))
(|;|)
(|;      station5|)
(|;|)
(|;|)
("STN5" (GOON 1))
(|;|)
(Q5 (|QUEUE(5)| nil (fslam.val 'station5 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station5 'machine-no) '|)/| '|5|) (fslam.val 'operation5 'op-
time)))
(nil (GOON 1))
(|;|)
```

```
(nil (ACT/16 (fslam.val2 (cadr (assoc 'station2 (fvalues 'transfer5 'tf-time)))) || "STN2"))
(|;|)
(|;|)
(EXIT (GOON 1))
(nil (ACT nil |ATRIB(2).EQ.1| A7))
(nil (ACT nil |ATRIB(2).EQ.2| A8))
(A7 (COLCT |INT(1)| |PART1 IN SYS|))
(nil (TERM))
(A8 (COLCT |INT(1)| |PART2 IN SYS|))
(nil (TERM))
(nil (ENDNETWORK))
(|INIT,0,1200;|)
(|FIN;|)
```

3.2.3. PROGRAM OF EXPERIMENT PROBLEM TWO

```
GEN,NLME,FRANZ SYSTEM,,1;
LIMITS,5,3,100;
;
NETWORK;
;
;     attribute description:
;
;       attribute 1 : part arrival time
;       attribute 2 : part type
;       attribute 3 : part process time in a station
;
;
;     load part1 into system
;
PAT1  CREATE,5, ,1, ,1;
        ASSIGN,ATRIB(2) = 1;
        ACT,,,STN1;
;
;     load part2 into system
;
PAT2  CREATE,NPSSN(3,1), ,1, ,1;
        ASSIGN,ATRIB(2) = 2;
        ACT,,,STN5;
;
;     station1
;
;
STN1  GOON,1;
;
Q1      QUEUE(1),,2,BLOCK;
        ACT(2)/1,RNORM(7,1.0,1);
        GOON,1;
;
        ACT/6,0.5,,STN2;
;
;     station2
;
STN2  GOON,1;
        ACT/7,,ATRIB(2).EQ.1,A1;
        ACT/8,,ATRIB(2).EQ.2,A2;
A1      ASSIGN,ATRIB(3) = 5;
        ACT,,,Q2;
A2      ASSIGN,ATRIB(3) = EXPON(5.5,1);
```

```
;
Q2      QUEUE(2),,4,BLOCK;
        ACT(5)/2,ATRIB(3);
        GOON,1;
;
        ACT/9,1,ATRIB(2).EQ.1,STN3;
        ACT/10,1.2,ATRIB(2).EQ.2,STN4;
;
;     station3
;
;
STN3  GOON,1;
;
Q3      QUEUE(3),,1,BLOCK;
        ACT(2)/3,UNFRM(5,6,1);
        GOON,1;
;
        ACT/11,0.5,,STN4;
;
;     station4
;
STN4  GOON,1;
        ACT/12,,ATRIB(2).EQ.1,A4;
        ACT/13,,ATRIB(2).EQ.2,A5;
A4      ASSIGN,ATRIB(3) = 3;
        ACT,,,Q4;
A5      ASSIGN,ATRIB(3) = 4;
;
Q4      QUEUE(4),,3,BLOCK;
        ACT(3)/4,ATRIB(3);
        GOON,1;
;
        ACT/14, ,ATRIB(2).EQ.1,EXIT;
        ACT/15, ,ATRIB(2).EQ.2,EXIT;
;
;     station5
;
;
STN5  GOON,1;
;
Q5      QUEUE(5),,2,BLOCK;
        ACT(3)/5,4;
        GOON,1;
;
        ACT/16,0.8,,STN2;
;
```

```
;
EXIT  GOON,1;
      ACT,,ATRIB(2).EQ.1,A7;
      ACT,,ATRIB(2).EQ.2,A8;
A7    COLCT,INT(1),PART1 IN SYS;
      TERM;
A8    COLCT,INT(1),PART2 IN SYS;
      TERM;
      ENDNETWORK;
INIT,0,1200;
FIN;
```

3.3.1. SUMMARY REPORT OF EXPERIMENT PROBLEM THREE

***** Summary Report For APOLLO System *****

Simulation Time : 1200 <minute>

A. Jobs Description.

Part : part1

a. Description : (description (property steel))
b. Enter Time : nil
c. Lot Size : nil

Part : part2

a. Description : (description (property nickle))
b. Enter Time : nil
c. Lot Size : nil

B. Facilities Description.

Station : station1

a. Description : (description (function drilling))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station2

a. Description : (description (function adjustment))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station3

a. Description : (description (function milling))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 1) (unit <machine>))

Station : station4

a. Description : (description (function turning))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station5

a. Description : (description (function polishing))
b. Buffer Size : (number (figure 2) (unit <part>))
c. Machine Number : (number (figure 2) (unit <machine>))

Station : station6

a. Description : (description (function washing))
b. Buffer Size : (number (figure 1) (unit <part>))
c. Machine Number : (number (figure 1) (unit <machine>))

C. Activities Description.

1. Routings.

```
((part1 --->
      ((<<opt.1>> station1)
       --->
       ((<<opt.2>> station2)
        --->
        ((<<opt.3>> station3
              (condition station4 (queue-state <full>)))
         --->
         ((<<opt.4>> station5) ---> (<exit>)))
        ((<<opt.3>> station4) ---> (<merge-to> station5)))))
 (part2 --->
      ((<<opt.1>> station1)
       --->
       ((<<opt.2>> station2)
        --->
        ((<<opt.3>> station4)
         --->
         ((<<opt.4>> station6
               (condition station5
                     (station-state <busy>)))
          --->
          (<exit>))
         ((<<opt.4>> station5) ---> (<exit>)))))))
```

2. Arrival Times.

Part	To	Value
part1	station1	(distribution (function <poisson>) (mean (number (figure 10) (unit <minute>))))
part2	station1	(distribution (function <constant>) (number (figure 5) (unit <minute>)))

3. Process Times.

Part	Station	Value
part1	station1	(number (figure 5) (unit <minute>))
part1	station2	(distribution (function <uniform>) (from (number (figure 2))) (to (number (figure 5) (unit <minute>))))
part1	station4	(number (figure 2) (unit <minute>))
part1	station5	(number (figure 4) (unit <minute>))
part1	station3	(number (figure 5) (unit <minute>))
part2	station1	(number (figure 3) (unit <minute>))
part2	station2	(distribution (function <normal>) (mean (number (figure 5))) (variance (number (figure 1) (unit <minute>))))
part2	station4	(number (figure 5) (unit <minute>))
part2	station6	(distribution (function <normal>) (mean (number (figure 7))) (variance (number (figure 2) (unit <minute>))))
part2	station5	(number (figure 6) (unit <minute>))

4. Transfer Times.

Part	From	To	Value
part1	station1	station2	(distribution (function <normal>) (mean (number (figure 5))) (variance (number (figure 1) (unit <minute>))))
part1	station2	station3	(distribution (function <exponential>) (mean (number (figure 2) (unit <minute>))))
part1	station2	station4	(distribution (function <exponential>) (mean (number (figure 2) (unit <minute>))))
part1	station4	station5	(distribution (function <normal>) (mean (number (figure 5) (unit <minute>))) (stdev (number (figure 2) (unit <minute>))))
part1	station3	station5	(number (figure 2) (unit <minute>))
part1	station5	<exit>	nil
part2	station1	station2	(distribution (function <uniform>) (from (number (figure 2))) (to (number (figure 3) (unit <minute>))))
part2	station2	station4	(distribution (function <exponential>) (mean (number (figure 3) (unit <minute>))))
part2	station4	station6	(distribution (function <exponential>) (mean (number (figure 2) (unit <minute>))))
part2	station4	station5	(distribution (function <uniform>) (from (number (figure 3))) (to (number (figure 4) (unit <minute>))))
part2	station6	<exit>	nil
part2	station5	<exit>	nil

3.3.2. MODEL TEMPLATE OF EXPERIMENT PROBLEM THREE

```
(|GEN,NLME,APOLLO SYSTEM,,1;|)
(|LIMITS,6,3,100;|)
(|;|)
(|NETWORK;|)
(|;|)
(|;      attribute description:|)
(|;|)
(|;        attribute 1 : part arrival time|)
(|;        attribute 2 : part type|)
(|;        attribute 3 : part process time in a station|)
(|;|)
(|;|)
(|;      load part1 into system|)
(|;|)
("PAT1" (CREATE (fslam.val 'arrival1 'interval) (fslam.val 'arrival1 'enter-time) 1
(fslam.val 'arrival1 'lot-size) 1))
(nil (ASSIGN |ATRIB(2) = 1|))
(nil (ACT nil nil "STN1"))
(|;|)
(|;      load part2 into system|)
(|;|)
("PAT2" (CREATE (fslam.val 'arrival2 'interval) (fslam.val 'arrival2 'enter-time) 1
(fslam.val 'arrival2 'lot-size) 1))
(nil (ASSIGN |ATRIB(2) = 2|))
(nil (ACT nil nil "STN1"))
(|;|)
(|;      station1|)
(|;|)
("STN1" (GOON 1))
(nil (ACT/7 nil |ATRIB(2).EQ.1| A1))
(nil (ACT/8 nil |ATRIB(2).EQ.2| A2))
(A1 (ASSIGN (assign.atb 3 (fslam.val 'operation1 'op-time))))
(nil (ACT nil nil Q1))
(A2 (ASSIGN (assign.atb 3 (fslam.val 'operation6 'op-time))))
(|;|)
(Q1 (|QUEUE(1)| nil (fslam.val 'station1 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station1 'machine-no) '|)/| '|1|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/9 (fslam.val2 (cadr (assoc 'station2 (fvalues 'transfer1 'tf-time))))
|ATRIB(2).EQ.1| "STN2"))
(nil (ACT/10 (fslam.val2 (cadr (assoc 'station2 (fvalues 'transfer6 'tf-time))))
|ATRIB(2).EQ.2| "STN2"))
```

```
(|;|)
(|;      station2|)
(|;|)
("STN2" (GOON 1))
(nil (ACT/11 nil |ATRIB(2).EQ.1| A4))
(nil (ACT/12 nil |ATRIB(2).EQ.2| A5))
(A4 (ASSIGN (assign.atb 3 (fslam.val 'operation2 'op-time))))
(nil (ACT nil nil Q2))
(A5 (ASSIGN (assign.atb 3 (fslam.val 'operation7 'op-time))))
(|;|)
(Q2 (|QUEUE(2)| nil (fslam.val 'station2 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station2 'machine-no) '|)/| '|2|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/13 (fslam.val2 (cadr (assoc 'station3 (fvalues 'transfer2 'tf-time)))) (concat
'|ATRIB(2).EQ.1| '.AND. '|NNQ(4).GE.| (fslam.val 'station4 'buffer-size)) "STN3"))
(nil (ACT/14 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer2 'tf-time))))
|ATRIB(2).EQ.1| "STN4"))
(nil (ACT/15 (fslam.val2 (cadr (assoc 'station4 (fvalues 'transfer7 'tf-time))))
|ATRIB(2).EQ.2| "STN4"))
(|;|)
(|;      station3|)
(|;|)
(|;|)
("STN3" (GOON 1))
(|;|)
(Q3 (|QUEUE(3)| nil (fslam.val 'station3 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station3 'machine-no) '|)/| '|3|) (fslam.val 'operation5 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/16 (fslam.val2 (cadr (assoc 'station5 (fvalues 'transfer4 'tf-time)))) || "STN5"))
(|;|)
(|;      station4|)
(|;|)
("STN4" (GOON 1))
(nil (ACT/17 nil |ATRIB(2).EQ.1| A7))
(nil (ACT/18 nil |ATRIB(2).EQ.2| A8))
(A7 (ASSIGN (assign.atb 3 (fslam.val 'operation3 'op-time))))
(nil (ACT nil nil Q4))
(A8 (ASSIGN (assign.atb 3 (fslam.val 'operation8 'op-time))))
(|;|)
(Q4 (|QUEUE(4)| nil (fslam.val 'station4 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station4 'machine-no) '|)/| '|4|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/19 (fslam.val2 (cadr (assoc 'station5 (fvalues 'transfer3 'tf-time))))
|ATRIB(2).EQ.1| "STN5"))
(nil (ACT/20 (fslam.val2 (cadr (assoc 'station6 (fvalues 'transfer8 'tf-time)))) (concat
'|ATRIB(2).EQ.2| '.AND. '|NNACT(5).GE.| (fslam.val 'station5 'machine-no)) "STN6"))
(nil (ACT/21 (fslam.val2 (cadr (assoc 'station5 (fvalues 'transfer8 'tf-time))))
```

```
|ATRIB(2).EQ.2| "STN5"))
(|;|)
(|;     station5|)
(|;|)
("STN5" (GOON 1))
(nil (ACT/22 nil |ATRIB(2).EQ.1| A10))
(nil (ACT/23 nil |ATRIB(2).EQ.2| A11))
(A10 (ASSIGN (assign.atb 3 (fslam.val 'operation4 'op-time))))
(nil (ACT nil nil Q5))
(A11 (ASSIGN (assign.atb 3 (fslam.val 'operation10 'op-time))))
(|;|)
(Q5 (|QUEUE(5)| nil (fslam.val 'station5 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station5 'machine-no) '|)/| '|5|) |ATRIB(3)|))
(nil (GOON 1))
(|;|)
(nil (ACT/24 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer5 'tf-time))))
|ATRIB(2).EQ.1| EXIT))
(nil (ACT/25 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer10 'tf-time))))
|ATRIB(2).EQ.2| EXIT))
(|;|)
(|;     station6|)
(|;|)
(|;|)
("STN6" (GOON 1))
(|;|)
(Q6 (|QUEUE(6)| nil (fslam.val 'station6 'buffer-size) BLOCK))
(nil ((concat '|ACT(| (fslam.val 'station6 'machine-no) '|)/| '|6|) (fslam.val 'operation9 'op-
time)))
(nil (GOON 1))
(|;|)
(nil (ACT/26 (fslam.val2 (cadr (assoc '<exit> (fvalues 'transfer9 'tf-time)))) || EXIT))
(|;|)
(|;|)
(EXIT (GOON 1))
(nil (ACT nil |ATRIB(2).EQ.1| A13))
(nil (ACT nil |ATRIB(2).EQ.2| A14))
(A13 (COLCT |INT(1)| |PART1 IN SYS|))
(nil (TERM))
(A14 (COLCT |INT(1)| |PART2 IN SYS|))
(nil (TERM))
(nil (ENDNETWORK))
(|INIT,0,1200;|)
(|FIN;|)
```

3.3.3. PROGRAM OF EXPERIMENT PROBLEM THREE

```
GEN,NLME,APOLLO SYSTEM,,1;
LIMITS,6,3,100;
;
NETWORK;
;
;     attribute description:
;
;       attribute 1 : part arrival time
;       attribute 2 : part type
;       attribute 3 : part process time in a station
;
;
;     load part1 into system
;
PAT1  CREATE,NPSSN(10,1), ,1, ,1;
      ASSIGN,ATRIB(2) = 1;
      ACT,,,STN1;
;
;     load part2 into system
;
PAT2  CREATE,5, ,1, ,1;
      ASSIGN,ATRIB(2) = 2;
      ACT,,,STN1;
;
;     station1
;
STN1  GOON,1;
      ACT/7,,ATRIB(2).EQ.1,A1;
      ACT/8,,ATRIB(2).EQ.2,A2;
A1    ASSIGN,ATRIB(3) = 5;
      ACT,,,Q1;
A2    ASSIGN,ATRIB(3) = 3;
;
Q1    QUEUE(1),,2,BLOCK;
      ACT(2)/1,ATRIB(3);
      GOON,1;
;
      ACT/9,RNORM(5,1.0,1),ATRIB(2).EQ.1,STN2;
      ACT/10,UNFRM(2,3,1),ATRIB(2).EQ.2,STN2;
```

```
;
;     station2
;
STN2  GOON,1;
      ACT/11,,ATRIB(2).EQ.1,A4;
      ACT/12,,ATRIB(2).EQ.2,A5;
A4    ASSIGN,ATRIB(3) = UNFRM(2,5,1);
      ACT,,,Q2;
A5    ASSIGN,ATRIB(3) = RNORM(5,1.0,1);
;
Q2    QUEUE(2),,2,BLOCK;
      ACT(2)/2,ATRIB(3);
      GOON,1;
;
      ACT/13,EXPON(2,1),ATRIB(2).EQ.1.AND.NNQ(4).GE.2,STN3;
      ACT/14,EXPON(2,1),ATRIB(2).EQ.1,STN4;
      ACT/15,EXPON(3,1),ATRIB(2).EQ.2,STN4;
;
;     station3
;
;
STN3  GOON,1;
;
Q3    QUEUE(3),,1,BLOCK;
      ACT(1)/3,5;
      GOON,1;
;
      ACT/16,2,,STN5;
;
;     station4
;
STN4  GOON,1;
      ACT/17,,ATRIB(2).EQ.1,A7;
      ACT/18,,ATRIB(2).EQ.2,A8;
A7    ASSIGN,ATRIB(3) = 2;
      ACT,,,Q4;
A8    ASSIGN,ATRIB(3) = 5;
;
Q4    QUEUE(4),,2,BLOCK;
      ACT(2)/4,ATRIB(3);
      GOON,1;
;
      ACT/19,RNORM(5,2,1),ATRIB(2).EQ.1,STN5;
      ACT/20,EXPON(2,1),ATRIB(2).EQ.2.AND.NNACT(5).GE.2,STN6;
      ACT/21,UNFRM(3,4,1),ATRIB(2).EQ.2,STN5;
```

```
;
;     station5
;
;
STN5  GOON,1;
      ACT/22,,ATRIB(2).EQ.1,A10;
      ACT/23,,ATRIB(2).EQ.2,A11;
A10   ASSIGN,ATRIB(3) = 4;
      ACT,,,Q5;
A11   ASSIGN,ATRIB(3) = 6;
;
Q5    QUEUE(5),,2,BLOCK;
      ACT(2)/5,ATRIB(3);
      GOON,1;
;
      ACT/24, ,ATRIB(2).EQ.1,EXIT;
      ACT/25, ,ATRIB(2).EQ.2,EXIT;
;
;     station6
;
;
STN6  GOON,1;
;
Q6    QUEUE(6),,1,BLOCK;
      ACT(1)/6,RNORM(7,1.41,1);
      GOON,1;
;
      ACT/26, ,,EXIT;
;
;
EXIT  GOON,1;
      ACT,,ATRIB(2).EQ.1,A13;
      ACT,,ATRIB(2).EQ.2,A14;
A13   COLCT,INT(1),PART1 IN SYS;
      TERM;
A14   COLCT,INT(1),PART2 IN SYS;
      TERM;
      ENDNETWORK;
INIT,0,1200;
FIN;
```